第6章: プロジェクトセットアップ — 書籍管理アプリの土台を作る

> いよいよ実際のアプリ開発が始まります！この章では、書籍管理アプリの「土台」（プロジェクトの雛形、フォルダ構成、初期設定）を作ります。

この章で行うこと

ここまでの章で学んだ技術（TypeScript、React、Next.js、Supabase）を組み合わせて、実際のプロジェクトを作成します。料理に例えると、**材料を揃えて下ごしらえをする段階**です。包丁やまな板、調味料を準備しておくのと同じで、後でスムーズに料理（コード）が書けるように、まずは作業環境を整えます。

1. **Next.js プロジェクトの作成** — `npx create-next-app` コマンド（プロジェクトの雛形を自動生成するコマンド）を実行
2. **Tailwind CSS の理解** — Tailwind CSS（テールウィンドCSS：HTMLのクラス名でスタイルを直接指定する CSSフレームワーク）の使い方
3. **フォルダ構成の設計** — どのファイルをどこに置くか、プロジェクト全体の設計図を決める
4. **Supabase との接続設定** — アプリからデータベースに接続するための設定ファイルを作成
5. **型定義の作成** — 書籍データの「Book型」をTypeScriptで定義

この章を終えると、ブラウザで開発サーバー（Development Server：開発中にアプリを確認するためのローカルサーバー。`http://localhost:3000` でアクセスできる）にアクセスし、ヘッダー付きのトップページが表示される状態になります。

> **用語のミニ解説：**
>
> - **ビルド（build）**：ソースコード（人間が書いたコード）を、ブラウザや Node.js が実行できる形式に変換する作業のこと。TypeScript を JavaScript に変換したり、複数ファイルを1つにまとめたりします。
> - **devサーバ（開発サーバー）**：コードを書きながら動作確認するためのローカル（自分のPC上）で動くWebサーバー。ファイルを保存すると自動的にブラウザの表示が更新されます。
> - **依存関係（dependencies）**：あなたのアプリが動くために必要な外部のライブラリ（React や Next.js など）のこと。`package.json` に一覧が書かれます。
> - **環境変数（environment variable）**：アプリの外側からプログラムに渡す設定値。コードに直接書きたくない秘密情報（API キーなど）や、開発環境と本番環境で値を変えたい設定に使います。

> **ポイント：** この章はコードを書く量が多いですが、一つ一つのファイルの役割を理解しながら進めてください。「なぜこのファイルが必要なのか」を意識すると、後の章で迷わなくなります。

この章のゴールと地図

家を建てるときは、いきなり壁紙を選んだりしません。まず**土地をならし（雛形づくり）**、**資材を運び込み（パッケージ追加）**、**部屋割りを決め（フォルダ構成）**、**水道や電気を引き（Supabase との接続）**、**設計図の寸法を決め（型定義）**、**玄関と看板を付けて（レイアウトとヘッダー）**、最後に**明かりをつけて確認（開発サーバー起動）**します。この章でやることは、まさにこの順番です。

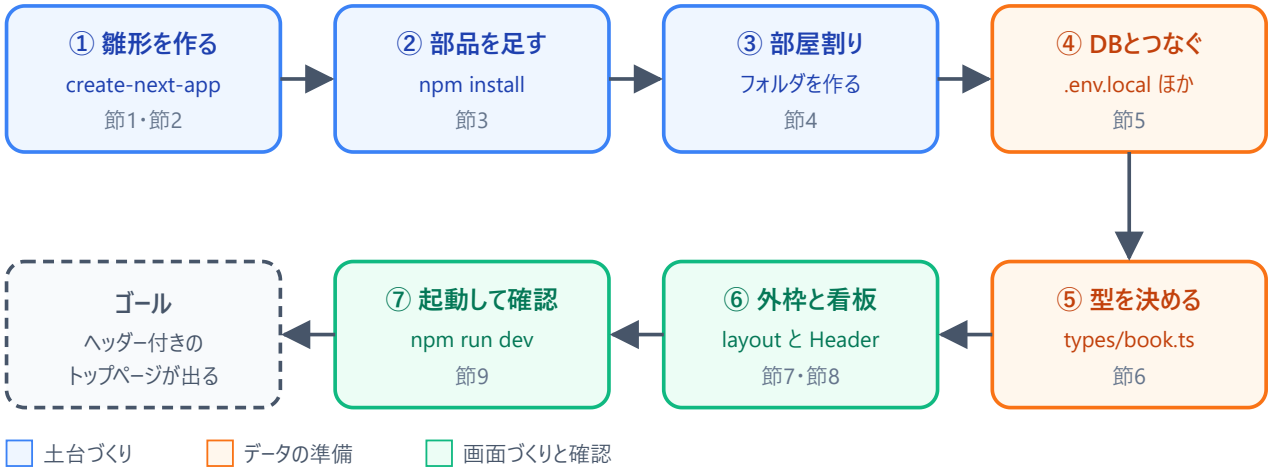


図6-1: この章の地図（①→⑦の順に進み、最後にブラウザで確認する）

この章を読み終えたときにできるようになること（ゴール）：

- `create-next-app` の質問に、意味を理解したうえで答えられる
- 生成されたファイルのうち「自分で書くもの」「設定ファイル」「触らないもの」を区別できる
- `@/components/Header` のような `@/` 付きの import が、どのファイルを指すか説明できる
- `.env.local` に書いた値が、どうやってコード（`process.env`）に届くか説明できる
- 開発サーバーを起動し、うまくいかないとき（ポート使用中・フォルダ違い・import ミスなど）に自分で直せる

> **ヒント:** 各節の最後に「つまずきポイント」を入れてあります。エラーが出たら、まずそのエラーメッセージで本章内を検索してみてください。同じメッセージの実例と直し方が見つかるはずです。

目次

1. Next.js プロジェクトの作成
2. Tailwind CSS の基礎
3. 必要なパッケージのインストール
4. プロジェクト構成の作成
5. Supabase クライアントの設定

6. 型定義ファイルの作成
7. ルートレイアウトの設定
8. 共通ヘッダーコンポーネント
9. 開発サーバーの起動と動作確認
10. データフローの全体像
11. 練習問題（解答・解説つき）

---

1. Next.js プロジェクトの作成

1.1 create-next-app コマンドの実行

ターミナル（コマンドプロンプト、PowerShell、または VS Code のターミナル）を開き、プロジェクトを作成したいディレクトリに移動してから、以下のコマンドを実行します。

> **▼ このコードがやること（先に日本語で）**：Next.js アプリの「雛形（ひながた：最初の土台一式）」を1コマンドで自動生成します。`npx` は「PCにインストールしないで一時的にツールを実行する」仕組みで、ここでは `create-next-app`（プロジェクト作成ツール）の最新版を呼び出しています。`book-management` の部分が作られるフォルダ名なので、好きな名前に変えてもかまいません。各単語の意味は下のコメントを見てください。

コピー

```
# npx: ローカルにインストールしていないコマンドも一時的に実行できるツール
# create-next-app: Next.js プロジェクトの雛形を自動生成するパッケージ
# @latest: パッケージの「最新版」を使うという指定（@を使ってバージョン指定）
# book-management: 作成するプロジェクトのフォルダ名（好きな名前に変えてもOK）
npx create-next-app@latest book-management
```

> **`npx` とは？** `npx` は npm 5.2 以降に同梱されているコマンドで、パッケージをグローバルインストールせずに一時的に実行できます。`create-next-app@latest` は「最新版の create-next-app を使う」という意味です。プロジェクト作成のためだけに使うコマンドを、PCに永久にインストールせずに済むので便利です。

> **`npm` と `npx` の違い**：`npm install` はパッケージを「インストール」して使えるようにするコマンドで、`npx` はパッケージを「実行」するコマンドです。`create-next-app` はプロジェクト作成時に一度だけ使うので、インストールせずに `npx` で実行します。

ヒント（最初に1回だけ聞かれる確認）：そのパソコンで初めて `npx create-next-app@latest` を実行すると、質問の前に `Need to install the following packages:` に続けて `create-next-app@16.4.1`（版の数字）と `Ok to proceed? (y)` が表示されることがあります。「この道具をダウンロードして実行してよいですか？」という確認なので、`y` を押して Enter で進めてください。

注意（バージョン）：この章の画面や出力は、2026年10月時点の最新版である **create-next-app 16.4.1 / Next.js 16.4.1** で実際に確かめたものです。Next.js 16 を動かすには **Node.js 20.9 以上**が必要です（第1章で入れた Node.js のバージョンは `node --version` で確認できます）。`@latest` は「その日の最新版」なので、時期によって質問の文言や数字が少し変わることがあります（執筆中に create-next-app 16.4.2 が出ましたが、質問とその後の表示は 16.4.1 と同じでした）。

つまずきポイント: プロジェクト名に大文字や空白を入れてしまう

プロジェクト名（フォルダ名）は npm のパッケージ名にもなるため、**小文字・数字・ハイフン（`-`）だけ**にするのが決まりです。

NG（大文字が入っている）：

コピー

```
# 大文字の B と M が入っているので作成できない
npx create-next-app@latest BookManagement
```

▼ 実行結果

コピー

```
Could not create a project called "BookManagement" because of npm naming restrictions:
    * name can no longer contain capital letters
```

NG（空白が入っている）：

コピー

```
# 空白（スペース）が入っているので作成できない
npx create-next-app@latest "book management"
```

▼ 実行結果

コピー

```
Could not create a project called "book management" because of npm naming restrictions:
    * name can only contain URL-friendly characters
```

OK（小文字とハイフンだけ）：

コピー

```
# 小文字とハイフンだけなら作成できる
npx create-next-app@latest book-management
```

`name can no longer contain capital letters` は「名前に大文字はもう使えません」、`name can only contain URL-friendly characters` は「名前には URL に使える文字しか使えません」という意味です。どちらもフォルダはまだ作られていないので、名前を直してもう一度実行すれば大丈夫です。

コマンドを実行すると、いくつかの質問が対話形式で表示されます。以下のように回答してください。

1.2 各選択肢の詳細解説

最初に出てくるのは「**おすすめの設定（recommended defaults）をそのまま使いますか？**」という質問です。ここで Enter を押すと推奨設定が選ばれてしまいますが、本書では**推奨設定を使いません**。キーボードの ↓（**下矢印**）**キー**で「**No, customize settings**（いいえ、自分で設定する）」に合わせてから Enter を押してください。

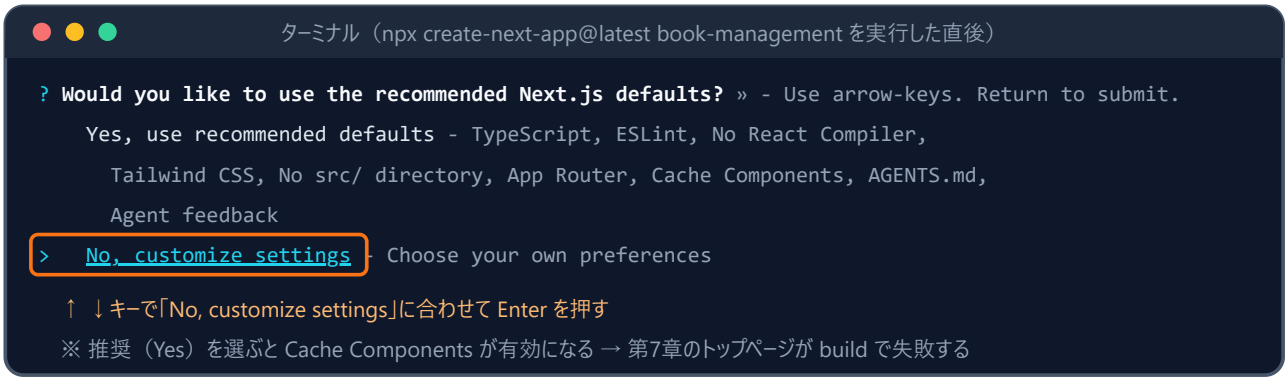


図6-2: 最初の質問の画面イメージ（矢印キーで「カスタマイズ」を選ぶ）

> **なぜ推奨設定を使わないのか:** 推奨設定には「Cache Components（キャッシュ・コンポーネンツ）を有効にする」という項目が含まれています。Cache Components が有効だと、第7章で作るトップページ（サーバーで Supabase からデータを取るページ）が `next build`（本番用ビルド）で失敗します。そのため、1つずつ自分で答える「カスタマイズ」を選び、Cache Components だけは No にします。

カスタマイズを選ぶと、残りの質問が1つずつ表示されます。すべて答え終わると、ターミナルは次のようになります（これは実際の画面を元に描いた画面イメージです）。

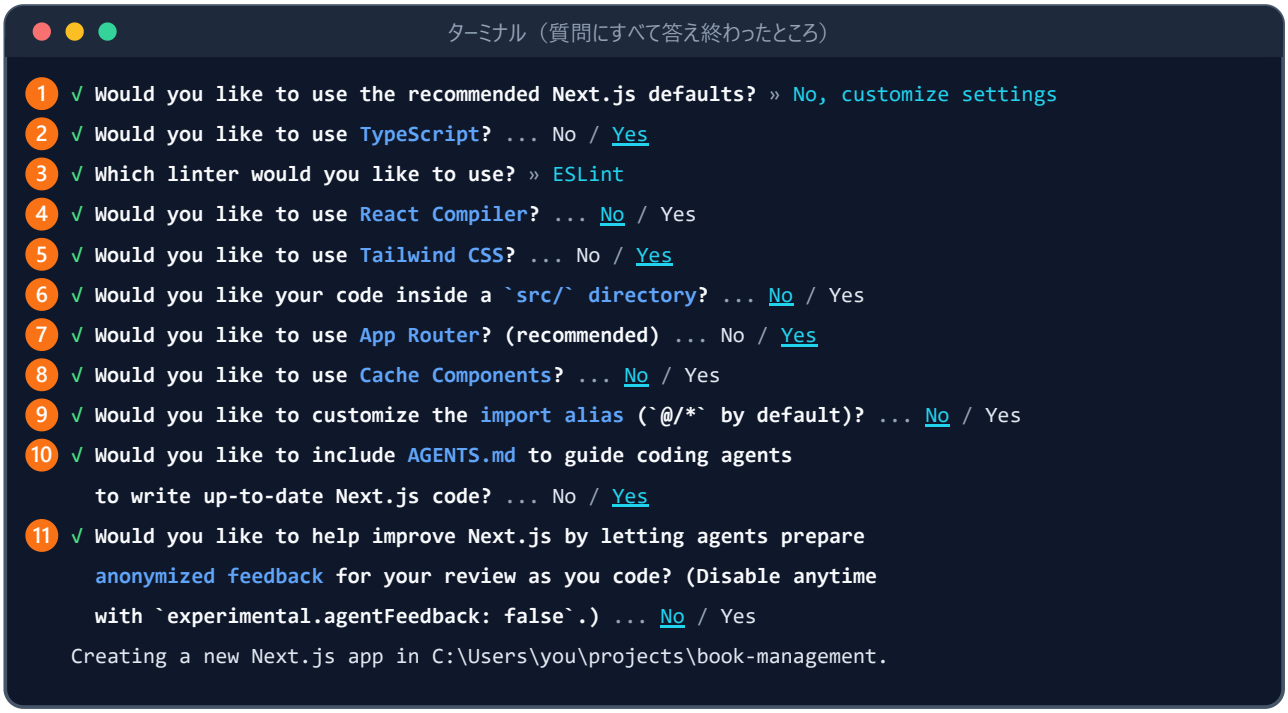


図6-3: create-next-app の質問に答え終わった画面イメージ（水色の下線が選んだ答え。番号は下の表と対応）

文字だけで書くと、実際の表示は次のとおりです（Windows では先頭の印が `√`、macOS では `✓` になります）。

コピー

```
√ Would you like to use the recommended Next.js defaults? » No, customize settings
√ Would you like to use TypeScript? ... No / Yes
√ Which linter would you like to use? » ESLint
√ Would you like to use React Compiler? ... No / Yes
√ Would you like to use Tailwind CSS? ... No / Yes
√ Would you like your code inside a `src/` directory? ... No / Yes
√ Would you like to use App Router? (recommended) ... No / Yes
√ Would you like to use Cache Components? ... No / Yes
√ Would you like to customize the import alias (`@/*` by default)? ... No / Yes
√ Would you like to include AGENTS.md to guide coding agents to write up-to-date Next.js code? ... No / Yes
√ Would you like to help improve Next.js by letting agents prepare anonymized feedback for your review as
you code? (Disable anytime with `experimental.agentFeedback: false`.) ... No / Yes
```

> **ヒント（Yes / No の選び方）**：「No / Yes」の質問は、**←→キー**（または Tab キー）で選択が切り替わり、選ばれている方に下線が付きます。Enter で確定です。文字だけの表示では「No / Yes」と両方が並ぶので、どちらを選んだかは上の画面イメージ（水色の下線）や、次の表で確認してください。

それぞれの選択肢が何を意味するのか、なぜその回答をするのかを詳しく見ていきましょう。番号は図の丸数字と対応しています。

番号	質問	回答	理由
①	**recommended defaults**（推奨設定を使うか）	**No, customize settings**	推奨設定は Cache Components が有効になっていて、本書の手順と合いません。1つずつ自分で答えます。
②	**TypeScript**	Yes	型安全性により、変数や関数の引数に間違った値を渡すミスをコンパイル時に検出できます。特にチーム開発やアプリが大規模になったときに威力を発揮します。本チュートリアルでは、Book 型などを定義してデータの構造を明確にします。
③	**linter**（どのコードチェック道具を使うか）	ESLint	JavaScript/TypeScript のコードを静的解析し、バグの原因になりやすい書き方やスタイルの不統一を自動検出します。例えば、未使用の変数がある場合に警告を出してくれます。Biome という別の道具も選べますが、情報が多い ESLint にします。
④	**React Compiler**	No	React のコードを自動で速くしてくれる仕組みです。便利ですが、学習中は「書いたとおりに動く」ほうが理解しやすいので使いません（後から有効にできます）。
⑤	**Tailwind CSS**	Yes	ユーティリティファーストの CSS フレームワークです。HTML（JSX）の中にクラス名を書くだけでスタイリングできるため、CSS ファイルを別途管理する必要がありません。詳しくは次のセクションで解説します。
⑥	**`src/` directory**	**No**（既定値のまま）	`src/` フォルダを作らず、`app/`・`components/`・`lib/`・`types/` をプロジェクトの直下（`package.json` と同じ場所）に置きます。フォルダが1段浅くなるので、どのファイルがどこにあるか見通しやすくなります。第7章以降のファイルの場所（`components/BookCard.tsx`、`app/page.tsx` など）もこの前提です。
⑦	**App Router**	Yes	Next.js 13 以降で導入された新しいルーティング方式です。ファイルシステムベースのルーティング、Server Components、レイアウトのネストなど、モダンな機能が使えます。Pages Router（旧方式）より推奨されています。
⑧	**Cache Components**	**No**	ページの表示結果をうまく使い回すための新しい仕組みです。有効にすると「データを取りに行く処理は `<Suspense>` で囲む」などの追加ルールが必要になり、第7章のトップページ（サーバーで Supabase からデータを取るページ）が `next build` で失敗します。**既定値は Yes なので、必ず No に切り替えてください。**
⑨	**Import alias**	No（デフォルト）	デフォルトの `@/*` エイリアスをそのまま使います。`@/components/Header` のように、プロジェクト直下のファイルを短いパスでインポートできます。
⑩	**AGENTS.md**	Yes	AI のコーディング支援（第11章の Claude Code など）に「この Next.js は新しい版なので、手元の説明書を読んでから書いてね」と伝えるメモ書きのファイルを作ります。アプリの動作には影響しません。

番号	質問	回答	理由
⑪	**anonymized feedback**（匿名のフィードバック）	No（どちらでも可）	AI のコーディング支援が、Next.js の改善に役立つ匿名の感想を下書きしてくれる機能です。アプリの動作には関係ないので、迷ったら No で構いません。

> **Turbopack について:** 以前の版では「Turbopack を使うか」という質問がありましたが、Next.js 16 では Turbopack（Rust で書かれた高速なバンドラー）が**最初から標準で使われる**ため、質問自体がなくなりました。開発サーバーの起動やホットリロード（コード変更時の自動反映）が従来の Webpack より大幅に高速です。

1.2.1 つまずきポイント: Enter を連打して推奨設定を選んでしまった

> **つまずきポイント: 推奨設定（Yes, use recommended defaults）のまま進めてしまう**
>
> 最初の質問で何も考えずに Enter を押すと、推奨設定が選ばれ、残りの質問は表示されずにすぐプロジェクトが作られます。見た目は成功しますが、**Cache Components が有効**になっています（匿名フィードバックも有効になります）。この章のうちは何も困らないので、第7章で `npm run build` するまで気づかないのが厄介な点です。

質問	推奨設定（Enter 連打） Yes, use recommended defaults	本書の設定（カスタマイズ） No, customize settings
TypeScript	Yes	Yes
Linter	ESLint	ESLint
React Compiler	No	No
Tailwind CSS	Yes	Yes
src/ directory	No	No
App Router	Yes	Yes
Cache Components	**Yes**	**No**
AGENTS.md	Yes	Yes
Agent feedback	**Yes**	**No**

違いは橙色の2行。Cache Components が Yes だと第7章で npm run build が失敗する

図6-4: 推奨設定と本書の設定の違い（推奨設定のままだと Cache Components が有効になる）

推奨設定で作ってしまったかどうかは、`next.config.ts` を開けば分かります。

NG（推奨設定のまま作ったプロジェクトを使い続ける）：

コピー

```
book-management/next.config.ts の中に、次の2行がある
  cacheComponents: true,
  partialPrefetching: true,
```

直し方は2つあります。1つ目は、この2行を消すことです（詳しくは 1.7 で説明します）。2つ目は「作り直す」ことで、まだ何も書いていないので失うものはありません。

OK（フォルダを消して、カスタマイズで作り直す）：

コピー

```
# 1つ上のフォルダに戻る（book-management の中にいる場合）
cd ..
# 推奨設定で作ったフォルダを丸ごと削除する（PowerShell の場合）
Remove-Item -Recurse -Force book-management
# もう一度作成し、最初の質問で「No, customize settings」を選ぶ
npx create-next-app@latest book-management
```

> **ヒント:** macOS や Linux のターミナルでは、削除コマンドは `rm -rf book-management` です。削除は元に戻せないので、フォルダ名をよく確認してから実行してください。

1.2.2 サンプル: 質問に答える代わりにオプションでまとめて指定する

> **▼ このコードがやること（先に日本語で）**：対話の質問に1つずつ答える代わりに、答えを「オプション（ `--` で始まる追加の指定）」としてコマンドに全部書いておく方法です。オプションを1つでも付けると質問は表示されず、書かなかった項目は推奨設定の値になります。本書の答えをすべて書いておけば、矢印キーの操作ミスが起きません。

コピー

```
# 本書の答え（TypeScript / ESLint / Tailwind / src なし / App Router /
# React Compiler なし / Cache Components なし / エイリアス @/* / npm を使う /
# AGENTS.md あり / 匿名フィードバックなし）をオプションで指定する
npx create-next-app@latest book-management --ts --eslint --tailwind --no-src-dir --app --no-react-compiler
--no-cache-components --import-alias "@/*" --use-npm --agents-md --no-agent-feedback
```

▼ 実行結果（パスは例。数字は時期によって変わります）

コピー

```
Creating a new Next.js app in C:\Users\you\projects\book-management.

Using npm.

Initializing project with template: app-tw


Installing dependencies:
- next
- react
- react-dom

Installing devDependencies:
- @tailwindcss/turbopack
- @types/node
- @types/react
- @types/react-dom
- eslint
- eslint-config-next
- tailwindcss
- typescript

npm warn deprecated eslint@9.39.5: This version is no longer supported. Please see https://eslint.org/version-support for other options.

added 363 packages, and audited 364 packages in 2m

145 packages are looking for funding
  run `npm fund` for details

5 high severity vulnerabilities

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.

Generating route types...
✓ Types generated successfully

Initialized a git repository.

Success! Created book-management at C:\Users\you\projects\book-management
```

▼ コードを1つずつ分解して解説

解説1: `--` で始まるオプションが「質問への答え」になる

コピー

```
--ts --eslint --tailwind --app
```

- `--ts` は ② TypeScript に Yes、`--eslint` は ③ で ESLint、`--tailwind` は ⑤ に Yes、`--app` は ⑦ に Yes と答えたのと同じです。
- 質問と同じ内容を、キー操作ではなく文字で先に伝えているだけです。

解説2: `--no-` で始まるオプションは「No と答える」

コピー

```
--no-src-dir --no-react-compiler --no-cache-components --no-agent-feedback
```

- `--no-` を頭に付けると「その機能は使わない」という意味になります。⑥ `src/` フォルダ、④ React Compiler、⑧ Cache Components、⑪ 匿名フィードバックをすべて No にしています。

- 特に `--no-cache-components` は大切です。付け忘れると推奨設定の Yes が使われてしまいます。

解説3: 残りのオプション

コピー

```
--import-alias "@/*" --use-npm --agents-md
```

- `--import-alias "@/*"` は ⑨ のエイリアスを `@/*` にする指定、`--agents-md` は ⑩ に Yes と答える指定です。
- `--use-npm` は「パッケージのインストールに npm を使う」という指定です（pnpm や yarn という別の道具もありますが、本書は npm で統一します）。

> **用語:** 「オプション（フラグ）」とは、コマンドの後ろに付けて動きを細かく指定する追加の文字列です。`--` で始まるものが多く、`npx create-next-app@latest --help` で一覧を見られます。

> **静的解析（static analysis）とは？** コードを実行せずに、ソースコードを読み解いて問題を見つける手法のこと。ESLint がこの役割を担います。例えば「この変数は宣言されているけど使われていない」「`if` 文の条件が常に true になる」といった問題を、実行前にチェックしてくれます。

> **パスエイリアスとは？（重要）** 通常、別のファイルをインポートするときは相対パス（`../../components/Header` のように、現在のファイルからの相対位置）を使います。深い階層になると `../../../../` のように `../` が増えて読みづらくなります。
>
> パスエイリアスを使うと、`@/components/Header` のようにプロジェクトのルート（`package.json` がある場所）を起点とした絶対インポートで書けるようになります。ファイルを移動してもインポート文を直さなくてよいので、リファクタリング（コードの整理）も楽になります。

1.3 なぜ Tailwind CSS を選ぶのか

CSS のスタイリング手法にはさまざまな選択肢があります。以下の比較表で、それぞれの特徴を整理します。

項目	Tailwind CSS	CSS Modules	styled-components
スタイルの書き場所	JSX のクラス名に直接記述	`.module.css` ファイルに記述	JavaScript ファイル内に記述
学習コスト	クラス名を覚える必要がある（慣れると高速）	通常の CSS と同じ知識で書ける	CSS + JavaScript の知識が必要
ファイル数	CSS ファイル不要（少ない）	コンポーネントごとに CSS ファイルが必要（多い）	CSS ファイル不要（少ない）
名前の衝突	なし（ユーティリティクラスのため）	なし（自動でスコープされる）	なし（自動でスコープされる）
バンドルサイズ	未使用クラスが自動削除される（小さい）	使った分だけ（中程度）	ランタイムで CSS を生成（やや大きい）
レスポンシブ対応	`md:`, `lg:` などのプレフィックスで簡単	メディアクエリを自分で書く	メディアクエリを自分で書く
デザインの一貫性	デフォルトのデザインシステム内蔵	自分で設計する必要がある	自分で設計する必要がある
Server Components 対応	完全対応	完全対応	対応に制限あり（`'use client'` が必要）
本チュートリアルでの採用	**採用**	不採用	不採用

Tailwind CSS を選ぶ最大の理由は「開発速度」です。 CSS ファイルを作成して行き来する必要がなく、JSX を書きながらそのままスタイリングできます。最初はクラス名を覚えるのに少し時間がかかりますが、慣れると非常に高速に UI を構築できるようになります。

1.4 コマンド実行中のターミナル出力

質問に全て答え終わると、ターミナルに次のような表示が流れます。次は 2026年10月に実際に実行したときの表示です（パスは例。パッケージの数や秒数は時期によって変わります）。

コピー

```
Creating a new Next.js app in C:\Users\you\projects\book-management.

Using npm.

Initializing project with template: app-tw


Installing dependencies:
- next
- react
- react-dom

Installing devDependencies:
- @tailwindcss/turbopack
- @types/node
- @types/react
- @types/react-dom
- eslint
- eslint-config-next
- tailwindcss
- typescript

npm warn deprecated eslint@9.39.5: This version is no longer supported. Please see https://eslint.org/version-support for other options.

added 363 packages, and audited 364 packages in 2m

145 packages are looking for funding
  run `npm fund` for details

5 high severity vulnerabilities

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.

Generating route types...
✓ Types generated successfully

Initialized a git repository.

Success! Created book-management at C:\Users\you\projects\book-management
```

最後に `Success! Created book-management at ...` と出ていれば成功です。途中の行の意味は次のとおりです。

表示	意味
`Initializing project with template: app-tw`	「App Router ＋ Tailwind（tw）」用のひな形を使って作り始めた、という意味です。
`Installing dependencies` / `devDependencies`	これからインストールする部品の一覧です（違いは下の囲みで説明します）。
`npm warn deprecated ...`	**warn（警告）** であって、エラーではありません。「この版はサポートが終わっています」というお知らせなので、作業はそのまま続けて大丈夫です。
`added 363 packages ... in 2m`	363個の部品を入れるのに約2分かかった、という意味です。
`5 high severity vulnerabilities`	部品の中に、注意が必要な既知の問題（脆弱性）が見つかったというお知らせです。すぐ下のつまずきポイントを読んでください。
`Generating route types...`	ページの URL（ルート）に合わせた TypeScript の型を自動で作っています。
`Initialized a git repository.`	Git（変更履歴を記録する仕組み）の管理を始めた、という意味です。

> **つまずきポイント:「vulnerabilities」と出たので `npm audit fix --force` を実行してしまう**
>
> 「脆弱性（vulnerabilities）」と出ると不安になり、表示どおり `npm audit fix --force` を実行したくなります。しかし `--force` は「互換性が壊れても構わず書き換える」という強い指示で、学習用プロジェクトでは逆に動かなくなる原因になります。

NG（表示どおりに実行してしまう）：

コピー

```
# 互換性を無視して部品を入れ替える（実行しないこと）
npm audit fix --force
```

OK（まず中身を確認するだけにする）：

コピー

```
# どの部品のどんな問題かを確認するだけ（何も書き換えない）
npm audit
```

▼ **実行結果**（先頭部分）

コピー

```
# npm audit report

braces  *
Severity: high
braces vulnerable to stack-exhaustion denial of service through deeply nested patterns -
https://github.com/advisories/GHSA-vfj7-8cjw-p6xm
fix available via `npm audit fix --force`
Will install eslint-config-next@14.2.35, which is a breaking change
node_modules/braces
```

`Will install eslint-config-next@14.2.35, which is a breaking change` に注目してください。「直すために eslint-config-next を **14.2.35（古い版）に入れ替えます。これは互換性を壊す変更です**」と書いてあります。Next.js 16 用の設定を 14 用に戻してしまうので、`--force` を付けて実行すると ESLint の設定が合わなくなります。今回の問題は開発中だけ使うコードチェック道具（ESLint）の部品の中にあり、作るアプリ本体の動作には関係しません。部品の作者が修正版を出すのを待ち、後で `npm update`（互換性の範囲で新しくする）を実行すれば十分です。

> 「added XXX packages」とは？: `npm install` が `node_modules/` フォルダに何個のパッケージを置いたかを示します。React/Next.js本体に加えて、依存パッケージ（依存関係でついてくる関連ライブラリ）が一緒に入るので300〜400個程度になります（今回は 363個）。これが普通です。

> **`npm install` で何が起きているか:**
>
> 1. `package.json` に書かれた依存パッケージのリストを読み取る
> 2. npm レジストリ（https://www.npmjs.com）から、それぞれのパッケージをダウンロード
> 3. ダウンロードしたパッケージを `node_modules/` フォルダに展開
> 4. 各パッケージがさらに依存しているパッケージも芋づる式にダウンロード（これが「依存関係の解決」）
> 5. インストールされた正確なバージョンを `package-lock.json` に記録
>
> `node_modules/` はとても巨大になりやすいですが、`.gitignore` でGit管理から除外するのが標準的なやり方です（後述）。

この流れは「買い物」にたとえると分かりやすくなります。`package.json` は**買い物メモ**、npm レジストリは**お店**、`node_modules/` は買ってきた物をしまう**冷蔵庫**、`package-lock.json` は何をいくつ買ったかの**レシート**です。

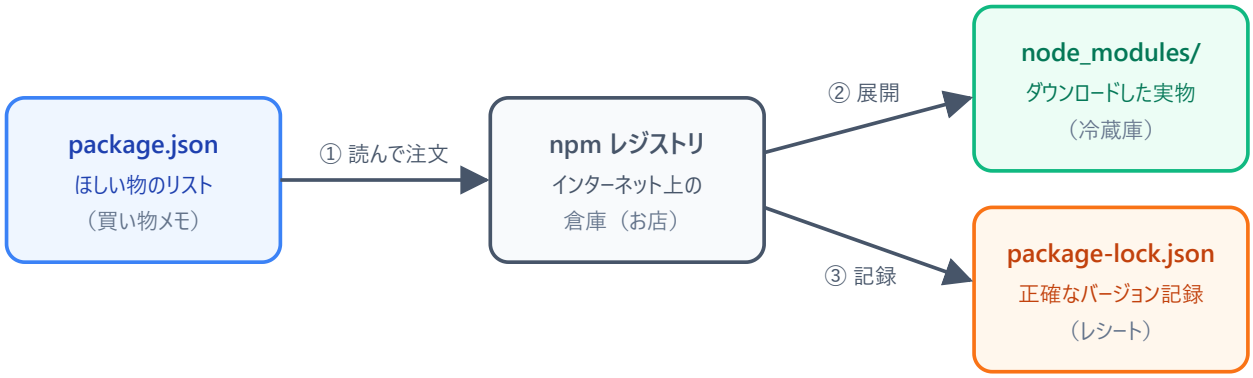


図6-5: npm install の流れ（買い物メモ → お店 → 冷蔵庫とレシート）

Git に入れる（チームで共有する）のはメモとレシート、つまり `package.json` と `package-lock.json` だけです。冷蔵庫の中身（`node_modules/`）は、メモとレシートさえあれば `npm install` でいつでも同じ状態に作り直せるので、共有しません。

> **`dependencies` と `devDependencies` の違い:**
>
> - **`dependencies`**：アプリが本番環境で動くために必要なパッケージ。例: React、Next.js本体。
> - **`devDependencies`**：開発中だけ必要なパッケージ。例: TypeScript（本番ではJavaScriptに変換済み）、ESLint（本番では使わない）。
>
> 本番ビルド時には `devDependencies` を除外できるので、本番のサーバーに置くファイルを軽くできます。

完了したら、表示されたプロジェクトフォルダに `cd` で移動します。

コピー

```
# cd: Change Directory の略。指定したフォルダに移動するコマンド
# book-management: 移動先のフォルダ名（先ほど create-next-app で作成したフォルダ）
cd book-management
```

1.5 生成されるファイルの役割

`create-next-app` を実行すると、以下のファイルとディレクトリが自動生成されます（create-next-app 16.4.1 で実際に作られたもの）。

コピー

```
book-management/
  .git/                 ... Git の履歴を保存する隠しフォルダ（自動生成。触らない）
  .next/                ... next dev / next build が作る作業用フォルダ（自動生成。.gitignoreで除外）
  node_modules/         ... インストールされたパッケージ群（数百フォルダ。.gitignoreで除外する）
  public/               ... 静的ファイル（画像など）。ここに置いたファイルは / 直下のURLでアクセス可能
    file.svg            ... ファイルアイコン（デフォルトサンプル）
    globe.svg           ... 地球アイコン（デフォルトサンプル）
    next.svg            ... Next.js ロゴ（デフォルトサンプル）
    vercel.svg          ... Vercel ロゴ（デフォルトサンプル）
    window.svg          ... ウィンドウアイコン（デフォルトサンプル）
  app/                  ... App Router のルートディレクトリ。フォルダ構成がそのままURLになる
    favicon.ico         ... ブラウザのタブに表示されるアイコン
    globals.css         ... グローバル CSS（Tailwind CSS を読み込む）
    layout.tsx          ... ルートレイアウト（全ページ共通の枠）
    page.tsx            ... トップページ（ / にアクセスしたときの表示）
  .gitignore            ... Git で管理しないファイルの指定
  AGENTS.md             ... AI コーディング支援向けの説明書（質問⑩で Yes にしたので作られた）
  eslint.config.mjs     ... ESLint の設定ファイル
  next-env.d.ts         ... Next.js の TypeScript 型定義（自動生成）
  next.config.ts        ... Next.js の設定ファイル
  package-lock.json     ... パッケージの正確なバージョン記録（自動更新されるので手で編集しない）
  package.json          ... プロジェクトの設定とパッケージ一覧
  README.md             ... プロジェクトの説明
  tsconfig.json         ... TypeScript の設定ファイル
```

ファイルが多くて圧倒されるかもしれませんが、「自分で書き換えるのはどれか」で色分けすると、実はほとんどが**触らなくてよいファイル**だと分かります。



図6-6: create-next-app が作ったファイルと、それぞれの役割（色は「触ってよいか」の目安）

> **ヒント:** 先頭が `.`（ドット）のファイルやフォルダ（`.git/`、`.next/`、`.gitignore`）は、macOS の Finder では**隠しファイル**として表示されません（Windows のエクスプローラーでも `.git/` は隠しフォルダになります）。VS Code のエクスプローラーならほぼすべて表示されます（`.git/` だけは VS Code でも既定で隠されます）。

各ファイルの役割を表にまとめます。

ファイル	役割
`package.json`	プロジェクト名、使用パッケージ、実行スクリプトなどを定義するファイルです。`npm install` はこのファイルを読んでパッケージをインストールします。
`tsconfig.json`	TypeScript コンパイラの設定ファイルです。`@/*` のパスエイリアスもここで定義されています。
`next.config.ts`	Next.js の動作をカスタマイズする設定ファイルです。画像の外部ドメイン許可などを設定できます。今回は Tailwind CSS を読み込むための設定（`turbopack.rules`）が最初から書かれています。
`eslint.config.mjs`	ESLint のルール設定です。`eslint-config-next` の推奨ルール（`core-web-vitals` と `typescript`）が最初から有効になっています。
`AGENTS.md`	AI のコーディング支援に向けた説明書です。「この Next.js は新しい版なので、`node_modules/next/dist/docs/` の説明書を読んでから書くこと」といった注意が書かれています。
`app/layout.tsx`	すべてのページを囲むルートレイアウトです。`<html>` と `<body>` タグを含み、共通のヘッダーやフッターを配置します。
`app/page.tsx`	トップページのコンポーネントです。URL `/` にアクセスしたときに表示されます。
`app/globals.css`	アプリ全体に適用されるグローバル CSS です。先頭の `@import "tailwindcss";` の1行で Tailwind CSS が読み込まれます。

> **注意（古い記事との違い）**：少し前の記事や本では、`tailwind.config.ts`（Tailwind の設定）、`postcss.config.mjs`（PostCSS の設定）、`.eslintrc.json`（ESLint の古い形式の設定）というファイルが登場します。現在の create-next-app が入れる **Tailwind CSS v4** は設定ファイルなしで動き（色などのカスタマイズは CSS の中に書きます）、ESLint の設定も `eslint.config.mjs` という新しい形式に変わったため、これらのファイルは作られません。**無くて正常**なので、自分で作る必要はありません。

`.gitignore` で何を除外するか、なぜか: Git の管理から除外する（コミットしない）べきファイルの種類を `.gitignore` ファイルに書きます。`create-next-app` が自動生成する `.gitignore` には、次のようなものが含まれます。

除外対象	理由
`/node_modules`	パッケージ群はとても大きい上に `package.json` から再インストールできるため。コミットすると無駄に容量を食う。
`/.next/`	Next.js のビルド結果。実行のたびに再生成されるため。
`.env*`	秘密の環境変数（APIキー、パスワードなど）が入っているため。GitHub にアップすると漏洩する。`.env.local` も `.env` も、`.env` で始まるファイルはすべて除外される。
`npm-debug.log*` など	ログファイルは個人のPC固有のものなので共有不要。
`.DS_Store`	macOS が自動生成する隠しファイル。アプリと無関係。

ルール:「自動生成されるもの」「PC固有のもの」「秘密情報を含むもの」はGitに入れない、と覚えておくと良いです。

1.6 主な設定ファイルの中身を見てみる

`package.json` の構造

`create-next-app` が生成する `package.json` は、次のような構造になっています（実際の中身に説明のコメントを足したもの。本物の JSON にはコメントを書けないので、これは読むための版です）。

コピー

```
{
  // プロジェクト名（フォルダ名と同じ）
  "name": "book-management",
  // プロジェクトのバージョン（自由に決められる）
  "version": "0.1.0",
  // true にすると npm publish で公開できなくなる（誤公開防止）
  "private": true,
  // npm run <名前> で実行できるコマンドの集まり
  "scripts": {
    // 開発サーバー起動（Next.js 16 では Turbopack が標準なのでオプション不要）
    "dev": "next dev",
    // 本番用ビルド（最適化された静的ファイルを生成）
    "build": "next build",
    // ビルド済みアプリを本番モードで起動
    "start": "next start",
    // ESLint でコード品質チェックを実行（Next.js 16 では next lint ではなく eslint を直接呼ぶ）
    "lint": "eslint"
  },
  // 本番でも必要なパッケージ
  "dependencies": {
    // Next.js フレームワーク
    "next": "16.4.1",
    // React 本体
    "react": "19.3.0",
    // React を DOM にレンダリングするためのパッケージ
    "react-dom": "19.3.0"
  },
  // 開発中だけ必要なパッケージ
  "devDependencies": {
    // Turbopack で Tailwind CSS を処理するための部品
    "@tailwindcss/turbopack": "^4",
    // Node.js の型定義
    "@types/node": "^20",
    // React の型定義
    "@types/react": "^19",
    // React DOM の型定義
    "@types/react-dom": "^19",
    // ESLint 本体
    "eslint": "^9",
    // Next.js 用 ESLint 設定
    "eslint-config-next": "16.4.1",
    // Tailwind CSS 本体（v4）
    "tailwindcss": "^4",
    // TypeScript コンパイラ
    "typescript": "^5"
  }
}
```

1.6.1 サンプル: 使えるスクリプトを一覧表示する

> **▼ このコードがやること（先に日本語で）：** `package.json` の `"scripts"` に何が書いてあるかを、ファイルを開かずにターミナルで確かめます。`npm run` とだけ打つと、`npm run ○○` の ○○ に入れられる名前の一覧が表示されます。ついでに、入っている Next.js のバージョンも確認します。

コピー

```
# プロジェクトのフォルダ（book-management）の中で実行する
# 使えるスクリプトの一覧を表示する
npm run
# インストールされている Next.js のバージョンを表示する
npx next --version
```

▼ 実行結果

コピー

```
Lifecycle scripts included in book-management@0.1.0:
  start
    next start
available via `npm run`:
  dev
    next dev
  build
    next build
  lint
    eslint
Next.js v16.4.1
```

▼ コードを1つずつ分解して解説

解説1: `npm run` だけで一覧が出る

コピー

```
npm run
```

- `npm run dev` の `dev` の部分を省略すると、「どの名前なら実行できるか」の一覧が出ます。
- `start` だけ別の見出し（Lifecycle scripts）に入っているのは、`start` が npm にとって特別な名前で、`npm start` と `run` なしでも実行できるからです。
- 一覧の下の行（`next dev` など）は、その名前で実際に実行される中身です。`npm run dev` は「`next dev` を実行して」という意味だと分かります。

解説2: `npx next --version` で Next.js の版を確かめる

コピー

```
npx next --version
```

- `npx next` は、このプロジェクトの `node_modules/` に入っている Next.js のコマンドを実行します。`--version` を付けると版だけを表示して終わります。

- 教材や記事と動きが違うときは、まずこの版を確認する習慣をつけると、原因の切り分けが早くなります。

> **用語:**「スクリプト（script）」とは、よく使うコマンドに短い名前を付けて `package.json` に登録したものです。長いコマンドを覚えなくても `npm run 名前` で実行できます。

`tsconfig.json` の主要設定

コピー

```
{
  // TypeScript コンパイラの動作設定
  "compilerOptions": {
    // コンパイル後のJavaScriptのバージョン（ES2017相当）
    "target": "ES2017",
    // 使えるグローバルAPI（DOM、最新のES機能など）
    "lib": ["dom", "dom.iterable", "esnext"],
    // .js ファイルもTypeScriptで扱えるようにする
    "allowJs": true,
    // ライブラリの型チェックをスキップ（高速化）
    "skipLibCheck": true,
    // 厳格な型チェックを有効化（推奨）
    "strict": true,
    // .ts → .js への変換ファイルは出力しない（Next.jsが代行）
    "noEmit": true,
    // CommonJS と ES Modules の混在を許可
    "esModuleInterop": true,
    // モジュールシステムは最新の ES Modules
    "module": "esnext",
    // モジュール解決方式（バンドラー用）
    "moduleResolution": "bundler",
    // .json ファイルを import 可能にする
    "resolveJsonModule": true,
    // 各ファイルを独立したモジュールとして扱う
    "isolatedModules": true,
    // JSX を React 17 以降の新しい方式で変換する（import React が不要）
    "jsx": "react-jsx",
    // 増分コンパイルを有効化（再ビルドが速くなる）
    "incremental": true,
    "plugins": [
      // Next.js 用の TypeScript プラグイン
      { "name": "next" }
    ],
    // パスエイリアスの定義
    "paths": {
      // @/foo はプロジェクト直下の foo を指す（./ は tsconfig.json がある場所）
      "@/*": ["./*"]
    }
  },
  // 型チェック対象（.next/ の中に自動生成される型も含める）
  "include": [
    "next-env.d.ts",
    "**/*.ts",
    "**/*.tsx",
    ".next/types/**/*.ts",
    ".next/dev/types/**/*.ts",
```

```
    "**/*.mts"
  ],
  // 型チェック対象から除外
  "exclude": ["node_modules"]
}
```

`next.config.ts` の中身

コピー

```
// next.config.ts

// Next.js の設定型をインポート（type 付きで型のみ取得）
import type { NextConfig } from "next";

// 設定オブジェクトの型を NextConfig に固定
const nextConfig: NextConfig = {
  /* config options here */
  // Turbopack（開発サーバーとビルドの道具）への追加設定
  turbopack: {
    // 「どのファイルをどう変換するか」の決まり
    rules: {
      // すべての .css ファイルを対象にする
      "*.css": {
        // Tailwind CSS 用の変換部品を通してから使う
        loaders: ["@tailwindcss/turbopack"],
        // 変換後も CSS として扱う
        as: "*.css",
      },
    },
  },
};

// デフォルトエクスポート（Next.js が自動で読み込む）
export default nextConfig;
```

`/* config options here */`（ここに設定を書く）の場所に、画像の外部ドメイン許可（`images.remotePatterns`）などの設定を追加していきます。`turbopack` の部分は「CSS ファイルを読み込むときに Tailwind CSS の変換を通す」という意味で、これがあるから `globals.css` の `@import "tailwindcss";` が働きます。**消さないでください。**

> **注意:** 質問⑧で Cache Components を Yes にしてしまった場合は、この `/* config options here */` の下に `cacheComponents: true,` と `partialPrefetching: true,` の2行が入っています。直し方は 1.7 のつまずきポイントを見てください。

1.7 つまずきポイント: Cache Components を Yes にしてしまった

> **つまずきポイント: Cache Components が有効なまま第7章に進む**
>
> 質問⑧の既定値は Yes なので、Enter を押しただけだと有効になります。この章のページは問題なく表示されますが、第7章で「サーバーで Supabase からデータを取るトップページ」を作ると、`npm run build`（本番用ビルド）が失敗します。

NG（Cache Components が有効な `next.config.ts`）：

コピー

```
// next.config.ts（質問⑧で Yes を選んだ場合）
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  /* config options here */
  // ↓ この2行が Cache Components を有効にしている
  cacheComponents: true,
  partialPrefetching: true,
  turbopack: {
    rules: {
      "*.css": {
        loaders: ["@tailwindcss/turbopack"],
        as: "*.css",
      },
    },
  },
};

export default nextConfig;
```

この状態で、第7章と同じ形のトップページ（サーバー用の Supabase クライアントで books テーブルを読むページ）を `npm run build` すると、次のエラーで止まります。

▼ **実行結果**（実際の出力の抜粋）

コピー

```
- Cache Components enabled
- Partial Prefetching enabled

  Creating an optimized production build ...
✓ Compiled successfully in 9.1s
  Running TypeScript ...
  Finished TypeScript in 2.3s ...
  Collecting page data using 5 workers ...
  Generating static pages using 5 workers (0/4) ...
  Generating static pages using 5 workers (1/4)
Error: Route "/": Next.js encountered uncached or runtime data during prerendering.

`fetch(...)`, `cookies()`, `headers()`, `params`, `searchParams`, or `connection()` accessed outside of
`<Suspense>` prevents the route from being prerendered, blocking the page load and leading to a slower user
experience.

（直し方の候補と詳しい調べ方の案内が英語で14行ほど続くので省略）

Error occurred prerendering page "/". Read more: https://nextjs.org/docs/messages/prerender-error
Export encountered an error on /page: /, exiting the build.
× Next.js build worker exited with code: 1 and signal: null
```

エラーの意味は「`/` のページで、`cookies()` などの毎回変わるデータを `<Suspense>` の外で使っているので、前もってページを作っておけません」です。Cache Components は「ページを前もって作っておく」ことを強く求める仕組みなので、データを毎回取りに行く第7章のページとは相性が悪いのです。

OK（2行を消して Cache Components を使わない）：

コピー

```
// next.config.ts（本書の設定）
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  /* config options here */
  turbopack: {
    rules: {
      "*.css": {
        loaders: ["@tailwindcss/turbopack"],
        as: "*.css",
      },
    },
  },
};

export default nextConfig;
```

`cacheComponents: true,` と `partialPrefetching: true,` の2行を消して保存すれば、本書どおりの設定（質問⑧で No を選んだのと同じ状態）になります。開発サーバーを動かしている場合は、`next.config.ts` の変更後に一度止めて（Ctrl+C）起動し直してください。

---

2. Tailwind CSS の基礎

2.1 ユーティリティファーストとは

従来の CSS では、まずクラス名を考え、そのクラスに対してスタイルを定義していました。

コピー

```
/* 従来の CSS */
/* card という名前のクラスにスタイルを定義 */
.card {
  /* 背景色を白に */
  background-color: white;
  /* 角を 8px の半径で丸める */
  border-radius: 8px;
  /* 内側の余白を 16px に */
  padding: 16px;
  /* 薄い影を下方向に */
  box-shadow: 0 1px 3px rgba(0, 0, 0, 0.1);
}
```

コピー

```
<!-- HTML 側で class 属性に名前を指定して上記スタイルを適用 -->
<div class="card">...</div>
```

Tailwind CSS では、あらかじめ用意された小さなユーティリティクラスを組み合わせてスタイリングします。

コピー

```
<!-- Tailwind CSS: 1つ1つが小さな役割を持つクラスを並べる -->
<!-- bg-white: 背景白 / rounded-lg: 大きめの角丸 / p-4: 内側16px余白 / shadow-sm: 控えめな影 -->
<div class="bg-white rounded-lg p-4 shadow-sm">...</div>
```

それぞれのクラスが1つの CSS プロパティに対応しています。

クラス	対応する CSS
`bg-white`	`background-color: white;`
`rounded-lg`	`border-radius: 0.5rem;`
`p-4`	`padding: 1rem;`
`shadow-sm`	`box-shadow: 0 1px 3px 0 rgb(0 0 0 / 0.1), 0 1px 2px -1px rgb(0 0 0 / 0.1);`

> **ヒント（実際に出力される CSS）**：Tailwind CSS v4 が実際に作る CSS は、`.p-4 { padding: calc(var(--spacing) * 4); }` のように「変数 × 数」の形になっています。`--spacing` は `0.25rem` と決められているので、計算結果は上の表と同じ `1rem` です。表は分かりやすさのため、計算後の値で書いています。

ユーティリティファーストのメリット:

- CSS ファイルを作る必要がない（JSX だけで完結する）
- クラス名を考える時間がゼロになる
- デザインシステム（余白・色・サイズ）が統一される
- 使っていないスタイルが自動で削除される（バンドルサイズが小さい）

ユーティリティファーストのデメリット:

- JSX のクラス名が長くなることがある
- 最初はクラス名を覚えるのに時間がかかる

2.2 よく使うクラス一覧

本チュートリアルで頻繁に使うクラスを以下にまとめます。この表をブックマークしておくと、開発中に素早く参照できます。

余白（Spacing）

クラス	CSS	説明
`p-4`	`padding: 1rem;` (16px)	内側の余白（全方向）
`px-4`	`padding-left: 1rem; padding-right: 1rem;`	内側の余白（左右）
`py-2`	`padding-top: 0.5rem; padding-bottom: 0.5rem;`	内側の余白（上下）
`pt-4`	`padding-top: 1rem;`	内側の余白（上のみ）
`pb-4`	`padding-bottom: 1rem;`	内側の余白（下のみ）
`m-4`	`margin: 1rem;`	外側の余白（全方向）
`mx-auto`	`margin-left: auto; margin-right: auto;`	要素を水平方向に中央揃え
`mt-8`	`margin-top: 2rem;`	外側の余白（上のみ）
`mb-4`	`margin-bottom: 1rem;`	外側の余白（下のみ）
`space-y-4`	子要素間に `margin-top: 1rem;`	子要素の縦方向の間隔
`gap-4`	`gap: 1rem;`	Flex/Grid の子要素間の間隔

数値の法則: Tailwind の数値は `0.25rem` (4px) 単位です。`p-1` = 4px、`p-2` = 8px、`p-4` = 16px、`p-8` = 32px のように、数値を4倍するとピクセル値になります。

色（Colors）

クラス	説明
`text-gray-900`	濃いグレーの文字色（ほぼ黒）
`text-gray-600`	中間のグレーの文字色
`text-gray-400`	薄いグレーの文字色
`text-blue-600`	青い文字色（リンクなど）
`text-red-600`	赤い文字色（エラーメッセージなど）
`text-green-600`	緑の文字色（成功メッセージなど）
`text-white`	白い文字色
`bg-white`	白い背景色
`bg-gray-50`	ごく薄いグレーの背景色
`bg-gray-100`	薄いグレーの背景色
`bg-blue-600`	青い背景色（ボタンなど）
`bg-red-600`	赤い背景色（削除ボタンなど）
`border-gray-200`	薄いグレーのボーダー色
`border-gray-300`	ボーダー色（フォーム入力欄など）

色の濃さ: `50` が最も薄く、`950` が最も濃い色です。`100` 刻みで用意されています。`gray-50`（ほぼ白）、`gray-500`（中間）、`gray-900`（ほぼ黒）のようなイメージです。

タイポグラフィ（Typography）

クラス	CSS	説明
`text-xs`	`font-size: 0.75rem;`	極小テキスト（12px）
`text-sm`	`font-size: 0.875rem;`	小さいテキスト（14px）
`text-base`	`font-size: 1rem;`	通常テキスト（16px）
`text-lg`	`font-size: 1.125rem;`	やや大きいテキスト（18px）
`text-xl`	`font-size: 1.25rem;`	大きいテキスト（20px）
`text-2xl`	`font-size: 1.5rem;`	見出し（24px）
`text-3xl`	`font-size: 1.875rem;`	大見出し（30px）
`font-bold`	`font-weight: 700;`	太字
`font-semibold`	`font-weight: 600;`	やや太字
`font-medium`	`font-weight: 500;`	やや太字（控えめ）
`font-normal`	`font-weight: 400;`	通常の太さ
`text-center`	`text-align: center;`	テキスト中央揃え
`truncate`	`overflow: hidden; text-overflow: ellipsis; white-space: nowrap;`	テキストが長い場合に「...」で省略

レイアウト（Layout / Flexbox / Grid）

クラス	CSS	説明
`flex`	`display: flex;`	Flexbox コンテナにする
`flex-col`	`flex-direction: column;`	子要素を縦に並べる
`items-center`	`align-items: center;`	交差軸方向に中央揃え
`justify-between`	`justify-content: space-between;`	両端に配置
`justify-center`	`justify-content: center;`	主軸方向に中央揃え
`grid`	`display: grid;`	Grid コンテナにする
`grid-cols-1`	`grid-template-columns: repeat(1, 1fr);`	1列のグリッド
`grid-cols-2`	`grid-template-columns: repeat(2, 1fr);`	2列のグリッド
`grid-cols-3`	`grid-template-columns: repeat(3, 1fr);`	3列のグリッド
`w-full`	`width: 100%;`	横幅を親要素いっぱいに
`max-w-7xl`	`max-width: 80rem;`	最大横幅（1280px）
`min-h-screen`	`min-height: 100vh;`	最小高さを画面いっぱいに
`hidden`	`display: none;`	要素を非表示にする

ボーダー・角丸・影（Borders / Radius / Shadow）

クラス	CSS	説明
`border`	`border-width: 1px;`	1px のボーダーを追加
`border-2`	`border-width: 2px;`	2px のボーダーを追加
`rounded`	`border-radius: 0.25rem;`	少し角丸
`rounded-md`	`border-radius: 0.375rem;`	中程度の角丸
`rounded-lg`	`border-radius: 0.5rem;`	大きめの角丸
`rounded-full`	`border-radius: 9999px;`	完全な丸（バッジやアイコンに）
`shadow-sm`	小さなシャドウ	控えめな影
`shadow`	中程度のシャドウ	標準的な影
`shadow-lg`	大きなシャドウ	目立つ影

状態変化（Hover / Focus）

クラス	説明
`hover:bg-blue-700`	マウスを載せたときに背景色を濃い青に変更
`hover:text-blue-600`	マウスを載せたときに文字色を青に変更
`hover:shadow-md`	マウスを載せたときに影を追加
`focus:outline-none`	フォーカス時のデフォルトのアウトラインを消す
`focus:ring-2`	フォーカス時に2pxのリングを表示
`focus:ring-blue-500`	フォーカスリングの色を青に設定
`transition`	CSS トランジションを有効にする（スムーズなアニメーション）
`duration-200`	トランジション時間を200msに設定
`cursor-pointer`	マウスカーソルをポインター（指）にする
`disabled:opacity-50`	無効状態のときに半透明にする
`disabled:cursor-not-allowed`	無効状態のときにカーソルを「禁止」にする

2.3 レスポンシブデザイン

Tailwind CSS では、ブレークポイントのプレフィックスを付けることで、画面サイズに応じたスタイルを簡単に適用できます。

プレフィックス	最小幅	対象デバイス
(なし)	0px	すべて（モバイルファースト）
`sm:`	640px	大きめスマートフォン
`md:`	768px	タブレット
`lg:`	1024px	ノートPC
`xl:`	1280px	デスクトップ
`2xl:`	1536px	大画面

重要: Tailwind はモバイルファースト設計です。プレフィックスなしのクラスがモバイル（最小画面）のスタイルになり、プレフィックス付きのクラスが「その画面サイズ以上」に適用されます。

実例: レスポンシブなグリッドレイアウト

コピー

```
{/* grid: グリッド表示 / grid-cols-1: 1列 / md:grid-cols-2: 768px以上で2列 / lg:grid-cols-3: 1024px以上で3列 / gap-4: 子要素間に16px間隔 */}
<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
  {/* 1つ目のセル */}
  <div>カード1</div>
  {/* 2つ目のセル */}
  <div>カード2</div>
  {/* 3つ目のセル */}
  <div>カード3</div>
</div>
```

この例では次のように表示が変わります。

画面サイズ	適用されるクラス	表示
0px 〜 767px	`grid-cols-1`	1列表示（カードが縦に並ぶ）
768px 〜 1023px	`md:grid-cols-2`	2列表示
1024px 〜	`lg:grid-cols-3`	3列表示

物差しの上に並べると、どの幅でどのクラスが効くのかがひと目で分かります。

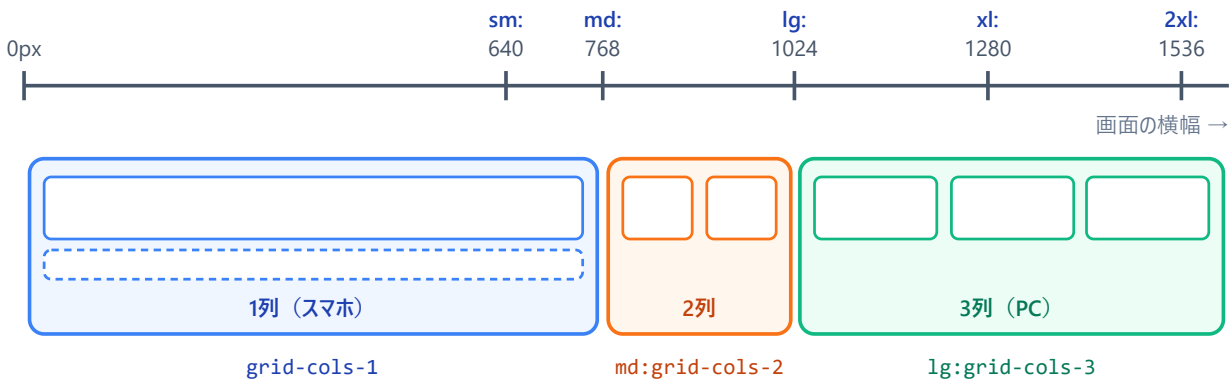


図6-7: ブレークポイントの物差しと、grid-cols-1 md:grid-cols-2 lg:grid-cols-3 の見え方

2.4「このクラスを付けるとこう表示される」具体例

実際にボタンを例にして、クラスを1つずつ追加していくとどう変わるか見てみましょう。

Step 1: テキストだけ（クラスなし）

コピー

```
{/* 何もクラスを付けないボタン。ブラウザのデフォルト見た目になる */}
<button>書籍を登録する</button>
```

ブラウザデフォルトのボタンが表示されます。装飾はほぼなく、小さな灰色の枠線と白い背景のボタンです。

Step 2: 背景色と文字色を追加

コピー

```
{/* bg-blue-600: 青い背景 / text-white: 白い文字 */}
<button className="bg-blue-600 text-white">書籍を登録する</button>
```

青い背景に白い文字のボタンになります。ただし、余白がなくテキストがギリギリです。

Step 3: 余白を追加

コピー

```
{/* px-4: 左右の内側余白16px / py-2: 上下の内側余白8px */}
<button className="bg-blue-600 text-white px-4 py-2">書籍を登録する</button>
```

左右に16px、上下に8pxの余白が入り、見た目が改善されます。

Step 4: 角丸を追加

コピー

```
{/* rounded-md: 中程度の角丸（border-radius: 6px） */}
<button className="bg-blue-600 text-white px-4 py-2 rounded-md">書籍を登録する</button>
```

角が丸くなり、モダンな印象になります。

Step 5: ホバーエフェクトとトランジションを追加

コピー

```
{/* hover:bg-blue-700: マウスホバー時に少し濃い青へ */}
{/* transition: 変化をスムーズに / duration-200: 200ミリ秒かけて変化 */}
<button className="bg-blue-600 text-white px-4 py-2 rounded-md hover:bg-blue-700 transition duration-200">
  書籍を登録する
</button>
```

マウスを載せると背景色が少し濃くなり、200ms かけてスムーズに変化します。

Step 6: フォーカスリングを追加

コピー

```
{/* focus:outline-none: デフォルトの黒い枠を消す */}
{/* focus:ring-2: フォーカス時に2pxのリング */}
{/* focus:ring-blue-500: リング色は青 */}
{/* focus:ring-offset-2: リングとボタンの間に2pxの隙間 */}
<button className="bg-blue-600 text-white px-4 py-2 rounded-md hover:bg-blue-700 transition duration-200
focus:outline-none focus:ring-2 focus:ring-blue-500 focus:ring-offset-2">
  書籍を登録する
</button>
```

Tab キーでフォーカスしたときに、ボタンの周りに青いリングが表示されます。これはアクセシビリティ（キーボード操作対応）のために重要です。

完成形: 本チュートリアルで使うボタンスタイル

コピー

```
{/* font-medium: やや太字（500） */}
{/* disabled:opacity-50: disabled属性付きのときは半透明 */}
{/* disabled:cursor-not-allowed: disabled時はカーソルを「禁止」マークに */}
<button className="bg-blue-600 text-white px-4 py-2 rounded-md font-medium hover:bg-blue-700 transition
duration-200 focus:outline-none focus:ring-2 focus:ring-blue-500 focus:ring-offset-2 disabled:opacity-50
disabled:cursor-not-allowed">
  書籍を登録する
</button>
```

---

3. 必要なパッケージのインストール

プロジェクトのディレクトリに移動してから、必要なパッケージをインストールします。Supabase とやり取りするための部品を**2つ**まとめて入れます。

コピー

```
# プロジェクトフォルダに移動
cd book-management

# npm install: package.json に依存パッケージを追加し、node_modules/ にダウンロード
# @supabase/supabase-js: Supabase 公式の JavaScript/TypeScript クライアントライブラリ
# @supabase/ssr: Next.js のようにサーバーとブラウザの両方で動くアプリ向けの補助ライブラリ
npm install @supabase/supabase-js @supabase/ssr
```

▼ 実行結果

コピー

```
added 10 packages, and audited 374 packages in 18s

146 packages are looking for funding
  run `npm fund` for details

5 high severity vulnerabilities

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
```

`added 10 packages` は「2つ頼んだら、それぞれが必要とする部品も合わせて10個入った」という意味です。`vulnerabilities` の行は 1.4 で説明したものと同じ（ESLint の部品の話）なので、ここでも `npm audit fix --force` は実行しません。

3.1 各パッケージの役割

パッケージ名	役割	説明
`@supabase/supabase-js`	Supabase クライアント	Supabase（データベース、認証、ストレージ）と通信するための公式 JavaScript/TypeScript ライブラリです。`supabase.from("books").select("*")` のような書き方で、PostgreSQL データベースにデータの読み書きができます。
`@supabase/ssr`	Next.js 用の補助	`@supabase/supabase-js` を Next.js で使いやすくする公式ライブラリです。**ブラウザ用**（`createBrowserClient`）と**サーバー用**（`createServerClient`）の2種類のクライアントを作る関数が入っています。第7章以降で使う `@/lib/supabase/client` と `@/lib/supabase/server` は、この2つの関数で作ります（節5）。
`next`（プリインストール済み）	Next.js フレームワーク	React ベースのフルスタックフレームワークです。ルーティング、サーバーサイドレンダリング、API ルートなどの機能を提供します。
`react`（プリインストール済み）	React ライブラリ	UI コンポーネントを構築するためのライブラリです。`useState` や `useEffect` などのフック（Hook）を使って、動的な UI を作成します。
`react-dom`（プリインストール済み）	React DOM	React コンポーネントをブラウザの DOM（画面）にレンダリングするためのライブラリです。
`typescript`（プリインストール済み）	TypeScript	JavaScript に型システムを追加した言語です。開発時の型チェックに使われます。
`tailwindcss` / `@tailwindcss/turbopack`（プリインストール済み）	Tailwind CSS	ユーティリティファーストの CSS フレームワーク（v4）と、それを Turbopack で処理するための部品です。`create-next-app` で Tailwind CSS を選択した場合、自動的にインストールされます。
`eslint`（プリインストール済み）	ESLint	コードの品質チェックツールです。バグの原因になりやすいパターンを検出します。

> **追加で入れるのが Supabase の2つだけで十分な理由:** 本チュートリアルの書籍管理アプリでは、Supabase との通信がメインの外部依存です。UI は Tailwind CSS（プリインストール済み）で構築し、フォーム処理は React の標準機能で実現します。追加のUIライブラリやフォームライブラリは使用しないため、シンプルな構成を保てます。

3.2 パッケージがインストールされたことの確認

`package.json` を開き、`dependencies` に `@supabase/supabase-js` と `@supabase/ssr` が追加されていることを確認します。

コピー

```
{
  "dependencies": {
    // 今インストールした Next.js 用の補助ライブラリ（^は後述）
    "@supabase/ssr": "^0.12.7",
    // 今インストールした Supabase クライアント
    "@supabase/supabase-js": "^2.118.0",
    // Next.js本体
    "next": "16.4.1",
    // React本体
    "react": "19.3.0",
    // React DOM
    "react-dom": "19.3.0"
  }
}
```

（数字は 2026年10月に実行したときのものです。あなたの環境ではもっと新しい数字になっていても問題ありません。）

3.2.1 サンプル: 入った部品の版をコマンドで確かめる

> **▼ このコードがやること（先に日本語で）：** `package.json` には「この範囲の版ならOK」という希望が書かれているだけで、実際に `node_modules/` に入った版とは少し違うことがあります。`npm ls`（list の略）で、**実際に入っている版**と、部品同士の親子関係を表示します。

コピー

```
# 指定した部品が、実際にどの版で入っているかを木の形で表示する
npm ls @supabase/supabase-js @supabase/ssr
```

▼ 実行結果（1行目のパスは例）

コピー

```
book-management@0.1.0 C:\Users\you\projects\book-management
├─┬ @supabase/ssr@0.12.7
│ └── @supabase/supabase-js@2.118.0 deduped
└── @supabase/supabase-js@2.118.0
```

▼ コードを1つずつ分解して解説

解説1: 木の形の読み方

コピー

```
├┬ @supabase/ssr@0.12.7
│└─ @supabase/supabase-js@2.118.0 deduped
```

- 1段目の `@supabase/ssr@0.12.7` は、あなたが直接入れた部品と、その版（0.12.7）です。
- その下に1段下がって `@supabase/supabase-js` が出ているのは、「`@supabase/ssr` も内部で `@supabase/supabase-js` を使っている」という親子関係を表しています。

解説2: `deduped`（重複をまとめた）の意味

コピー

```
│└─ @supabase/supabase-js@2.118.0 deduped
└─ @supabase/supabase-js@2.118.0
```

- `deduped` は「同じ部品を2つ入れず、1つを共有している」という意味です。あなたが直接入れた `@supabase/supabase-js@2.118.0` を、`@supabase/ssr` も一緒に使っています。
- 何も表示されない、または `(empty)` と出る場合は、インストールに失敗しているか、別のフォルダで実行しています。`cd book-management` で正しいフォルダに入ってからもう一度試してください。

> **用語:** 「依存関係（dependency）」とは、ある部品が動くために必要な別の部品との関係のことです。`npm ls` はこの関係を木（ツリー）の形で見せてくれます。

> **`^` の意味:** `^2.49.1` は「メジャーバージョン 2 の範囲で最新版を許容する」という意味です。`2.49.1` から `2.99.99` まではインストール可能ですが、`3.0.0` はインストールされません。これにより、破壊的変更を避けつつ最新のバグ修正やパフォーマンス改善を受けられます。

> **セマンティックバージョニング（SemVer）の補足:** バージョン番号は `メジャー.マイナー.パッチ` の形（例: `2.49.1`）になっています。
>
> - **メジャー**（2）: 互換性のない変更（古い書き方が動かなくなる可能性あり）
> - **マイナー**（49）: 機能追加（後方互換あり）
> - **パッチ**（1）: バグ修正
>
> `^` は「マイナーとパッチの自動アップデートはOK、メジャーは固定」という意味のルールです。
>
> ただし、`@supabase/ssr` の `^0.12.7` のように**メジャーが 0 の場合**は特別で、「0.12.x の範囲（パッチだけ）」しか自動で上がりません。0 から始まる版は「まだ仕様が固まっていない段階」とされ、マイナーが上がるだけでも互換性が変わることがあるためです。

4. プロジェクト構成の作成

4.1 ディレクトリ構成

書籍管理アプリでは、以下のディレクトリ構成を採用します。



テキスト形式でも示します（`package.json` などの設定ファイルは省略しています）。

コピー

```
book-management/
├── app/                          # App Router のルート。フォルダ名 = URL になる
│   ├── page.tsx                  # / にアクセスしたときのページ（トップ：書籍一覧）
│   ├── layout.tsx                # 全ページ共通の枠（HTMLタグ、ヘッダーなど）
│   ├── globals.css               # アプリ全体のグローバルCSS（Tailwindディレクティブ含む）
│   └── books/                    # /books 配下のページをまとめるフォルダ
│       ├── new/                  # /books/new ページ用のフォルダ
│       │   └── page.tsx          # 書籍登録ページ
│       └── [id]/                 # [id] は動的セグメント。/books/任意のID にマッチ
│           ├── page.tsx          # 書籍詳細ページ（/books/123 など）
│           └── edit/             # /books/[id]/edit ページ用のフォルダ
│               └── page.tsx      # 書籍編集ページ
├── components/                   # 再利用可能なUI部品を置くフォルダ
│   ├── Header.tsx                # 共通ヘッダー（全ページ上部のナビ）
│   ├── BookCard.tsx              # 書籍カード（一覧の各アイテム）
│   ├── BookForm.tsx              # 書籍登録・編集フォーム
│   ├── BookList.tsx              # 書籍一覧表示（BookCardを並べる）
│   ├── DeleteButton.tsx          # 削除ボタン（確認ダイアログ付き）
│   ├── StatusBadge.tsx           # ステータスバッジ（読書中など）
│   ├── RatingStars.tsx           # 評価（星）表示
│   └── LoadingSpinner.tsx        # ローディング表示（くるくる回るやつ）
├── lib/                          # アプリ全体で使うユーティリティを置くフォルダ
│   └── supabase/                 # Supabase に接続するクライアントを作るファイルをまとめるフォルダ
│       ├── client.ts             # ブラウザで動く部品（Client Component）用
│       └── server.ts             # サーバーで動く部品（Server Component）用
└── types/                        # TypeScript 型定義を置くフォルダ
    └── book.ts                   # Book / BookInsert / BookUpdate などの型
```

4.2 各ファイルの役割

ページファイル（`app/` ディレクトリ）

ファイル	URL パス	役割
`app/page.tsx`	`/`	トップページ。データベースから書籍の一覧を取得し、BookList コンポーネントで表示します。
`app/layout.tsx`	全ページ共通	ルートレイアウト。`<html>` と `<body>` タグ、ヘッダーなどの全ページ共通要素を定義します。
`app/globals.css`	全ページ共通	Tailwind CSS のディレクティブとカスタムスタイルを定義します。
`app/books/new/page.tsx`	`/books/new`	書籍登録ページ。BookForm コンポーネントを使い、新しい書籍の情報を入力・保存します。
`app/books/[id]/page.tsx`	`/books/123`	書籍詳細ページ。URL の `[id]` 部分に書籍の ID が入り、その書籍の詳細情報を表示します。
`app/books/[id]/edit/page.tsx`	`/books/123/edit`	書籍編集ページ。既存の書籍情報を読み込み、BookForm コンポーネントで編集・更新します。

`[id]` とは？（動的ルーティング） フォルダ名を角括弧 `[]` で囲むと「動的ルート」になります。`/books/1` 、`/books/2` 、`/books/abc` のように、`[id]` の部分に任意の値が入ります。ページコンポーネントの `params` プロパティからその値を取得できます。

コンポーネントファイル（`components/` ディレクトリ）

ファイル	役割
`Header.tsx`	アプリ上部に表示されるナビゲーションバー。アプリ名と「書籍を登録する」リンクを含みます。全ページで共通して表示されます。
`BookCard.tsx`	書籍一覧の中の1冊分の表示カード。タイトル、著者、ステータスバッジ、評価（星）を表示します。クリックすると詳細ページに遷移します。
`BookForm.tsx`	書籍の登録・編集に使うフォーム。タイトル、著者、ステータス、評価、メモの入力欄を持ちます。新規登録と編集の両方で共通して使用します。
`BookList.tsx`	BookCard を並べて一覧表示するコンポーネント。レスポンシブなグリッドレイアウトで配置します。書籍が0件の場合は「書籍が登録されていません」というメッセージを表示します。
`DeleteButton.tsx`	書籍を削除するボタン。クリック時に確認ダイアログを表示し、承諾した場合のみ削除を実行します。
`StatusBadge.tsx`	読書ステータス（「読みたい」「読書中」「読了」）を色付きバッジで表示するコンポーネント。ステータスに応じて色が変わります。
`RatingStars.tsx`	5段階の星評価を表示するコンポーネント。黄色い星と灰色の星で評価を視覚化します。
`LoadingSpinner.tsx`	データ読み込み中に表示するスピナー（回転アニメーション）。API からのレスポンスを待っている間、ユーザーに読み込み中であることを伝えます。

ユーティリティファイル（`lib/` ディレクトリ）

ファイル	役割
`supabase/client.ts`	ブラウザ用の Supabase クライアントを作る `createClient()` 関数をエクスポートします。フォーム送信や削除ボタンなど、`"use client"` を付けた部品から `import { createClient } from "@/lib/supabase/client";` で使います。
`supabase/server.ts`	サーバー用の Supabase クライアントを作る `createClient()` 関数をエクスポートします。トップページ（`app/page.tsx`）のような Server Component から `import { createClient } from "@/lib/supabase/server";` で使います。どちらも環境変数から Supabase の URL と API キーを読み取ります。

型定義ファイル（`types/` ディレクトリ）

ファイル	役割
`book.ts`	書籍データの TypeScript 型定義。`Book` 型（データベースから取得する完全なデータ）、`BookInsert` 型（新規登録時に送るデータ）、`BookUpdate` 型（更新時に送るデータ）を定義します。

4.2.1 ファイル同士の「import 関係」を地図にする

表で役割を見ただけでは、「どのファイルがどのファイルを使うのか」がつかみにくいものです。会社にたとえると、`app/` のページは**お客さんの窓口**、`components/` は**窓口で使う道具**、`lib/supabase/` は**倉庫（データベース）への電話**、`types/book.ts` は**全員が見る伝票の書式**です。窓口は道具と電話を使い、道具も電話も伝票の書式に従います。

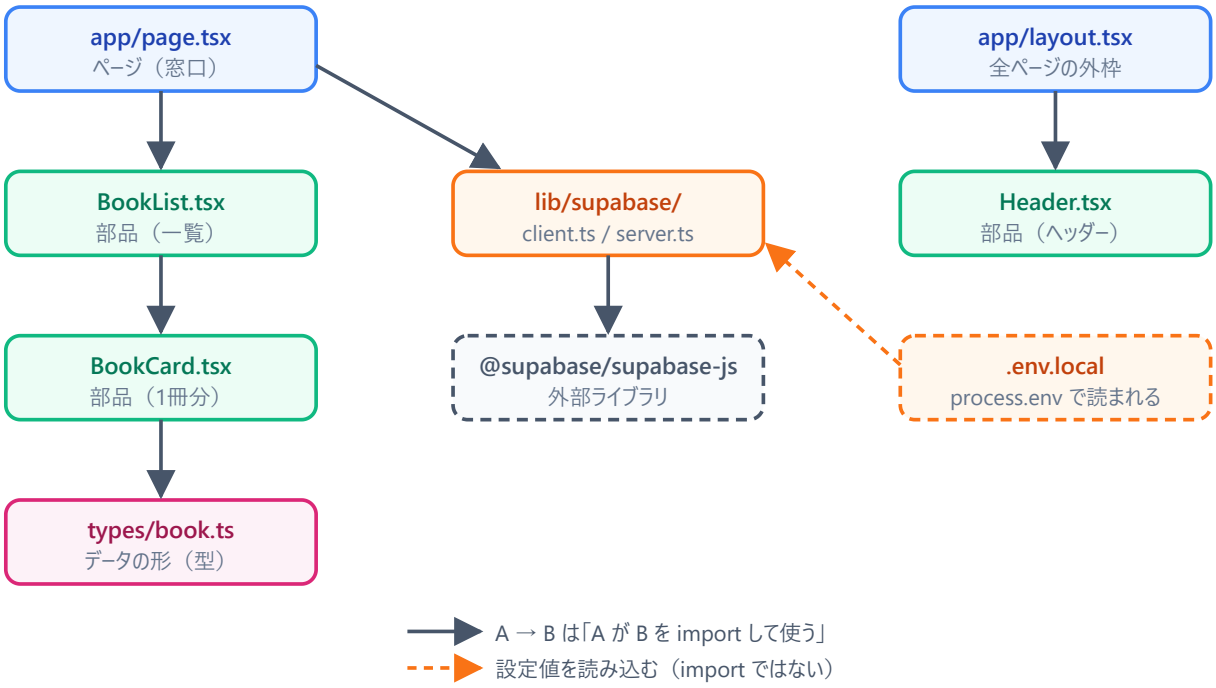


図6-8: ファイル同士の import 関係（矢印は上から下への一方通行）

この図から、次の2つのルールが読み取れます。

- **矢印は上から下への一方通行**: ページ → 部品 → 型、の順に「使う」関係が流れます。逆向き（例: `types/book.ts` が `app/page.tsx` を import する）は作りません。逆向きの矢印ができると、どこを直すとどこが壊れるのか追いにくくなるからです。
- **データベースにつながる入口は `lib/supabase/` フォルダの1か所だけ**: ページや部品は、自分で接続を作らず、必ずこのフォルダの `createClient` を import して使います（サーバーで動くページは `server.ts`、ブラウザで動く部品は `client.ts`）。接続設定（`.env.local`）を読むのもここだけなので、接続まわりで問題が起きたら「まず `lib/supabase/` と `.env.local` を見る」と決めておけます。

4.3 ディレクトリとファイルの作成

以下のコマンドで必要なディレクトリとファイルを一括作成します。

> **▼ このコードがやること（先に日本語で）**：これから書いていくコードを置くための「フォルダ（ディレクトリ）」をまとめて作ります。`mkdir` はフォルダを作るコマンドで、`-p` を付けると途中の親フォルダもまとめて作ってくれます。`[id]` のように角括弧が入った特殊なフォルダ名はそのままだとシェルに誤解されるため、`\` で記号を打ち消して（エスケープして）います。うまくいかなければ、コメントにある通り VS Code から手作業で作ってもOKです。

コピー

```
# ディレクトリの作成
# mkdir: Make Directory の略。フォルダを作成するコマンド
# -p: 親フォルダが存在しなければ一緒に作る（既に存在してもエラーにならない）
mkdir -p app/books/new
# [id] は角括弧の意味を持つ特殊文字なので、シェルで誤解されないようにバックスラッシュでエスケープ
mkdir -p app/books/\[id\]/edit
mkdir -p components
mkdir -p lib/supabase
mkdir -p types
```

> **注意:** `[id]` は角括弧を含むため、シェルによってはエスケープが必要です。うまくいかない場合は、VS Code のエクスプローラーからフォルダを手動で作成してください。

> **▼ このコードがやること（先に日本語で）**：さきほど作ったフォルダの中に、これから編集していく「空っぽのファイル」を一気に用意します。`touch` は中身が空のファイルを作るコマンドで、中身は後の節で書き込んでいきます。
> Windows の PowerShell には `touch` がないので、その場合は下の「4.3.1」の PowerShell 版を使ってください。

コピー

```
# ファイルの作成（中身は後ほど記述）
# touch: 空のファイルを作成するコマンド（既に存在する場合は最終更新時刻だけ変える）
# 共通ヘッダー
touch components/Header.tsx
# 書籍カード
touch components/BookCard.tsx
# 書籍フォーム
touch components/BookForm.tsx
# 書籍一覧
touch components/BookList.tsx
# 削除ボタン
touch components/DeleteButton.tsx
# ステータスバッジ
touch components/StatusBadge.tsx
# 評価表示
touch components/RatingStars.tsx
# ローディング
touch components/LoadingSpinner.tsx
# Supabase クライアント（ブラウザ用）
touch lib/supabase/client.ts
# Supabase クライアント（サーバー用）
touch lib/supabase/server.ts
# Book型定義
touch types/book.ts
```

4.3.1 サンプル: Windows の PowerShell で同じフォルダとファイルを作る

▼ **このコードがやること（先に日本語で）**：上の `mkdir -p` と `touch` は macOS / Linux（と Git Bash）向けの書き方です。Windows の PowerShell では、フォルダもファイルも `New-Item`（新しい項目を作る）というコマンドで作ります。カンマ `,` で区切ると、いくつもまとめて作れます。最後に、作ったものを一覧表示して確かめます。どれも `book-management` フォルダの中で実行します。

コピー

```
# フォルダをまとめて作る（-Force: 途中のフォルダも作り、既にあってもエラーにしない）
# [id] を含むパスは "..." で囲めば、そのままの名前で作られる
New-Item -ItemType Directory -Force -Path "app/books/new", "app/books/[id]/edit", "components",
"lib/supabase", "types" | Out-Null

# 空のファイルをまとめて作る（| Out-Null は、作るたびに出る長い表示を消すため）
New-Item -ItemType File -Path "components/Header.tsx", "components/BookCard.tsx",
"components/BookForm.tsx", "components/BookList.tsx", "components/DeleteButton.tsx",
"components/StatusBadge.tsx", "components/RatingStars.tsx", "components/LoadingSpinner.tsx",
"lib/supabase/client.ts", "lib/supabase/server.ts", "types/book.ts" | Out-Null

# 4つのフォルダの中身を、下の階層まで「いまの場所から見たパス」で一覧表示する
Get-ChildItem -Recurse app, components, lib, types | Resolve-Path -Relative
```

▼ 実行結果

コピー

```
.\app\books
.\app\favicon.ico
.\app\globals.css
.\app\layout.tsx
.\app\page.tsx
.\app\books\new
.\app\books\[id]
.\app\books\[id]\edit
.\components\BookCard.tsx
.\components\BookForm.tsx
.\components\BookList.tsx
.\components\DeleteButton.tsx
.\components\Header.tsx
.\components\LoadingSpinner.tsx
.\components\RatingStars.tsx
.\components\StatusBadge.tsx
.\lib\supabase
.\lib\supabase\client.ts
.\lib\supabase\server.ts
.\types\book.ts
```

（`app\` の中の4つは create-next-app が作ったファイルです。本章で作ったものが分かるよう、説明用に `app\` だけを用意した小さなフォルダで実行した結果を載せています。）

▼ コードを1つずつ分解して解説

解説1: `New-Item -ItemType Directory` でフォルダを作る

コピー

```
New-Item -ItemType Directory -Force -Path "app/books/new", "app/books/[id]/edit"
```

- `-ItemType Directory` は「フォルダを作る」という指定です。`-Path` の後ろに作りたい場所を書きます。
- `-Force` を付けると、途中のフォルダ（`books` など）が無くても一緒に作り、すでにあってもエラーにしません。bash の `mkdir -p` と同じ働きです。
- `[id]` を含む名前も、`"..."` で囲めばそのまま作れます。

解説2: ファイルを作るときは `-Force` を付けない

コピー

```
New-Item -ItemType File -Path "types/book.ts" | Out-Null
```

- `-ItemType File` は「空のファイルを作る」という指定です。

- ここで `-Force` を付けないのはわざとです。**ファイルに `-Force` を付けると、すでにあるファイルの中身が空っぽに上書きされます。**書いたコードが消える事故を防ぐため、ファイルには付けません。
- 付けずに2回実行すると、2回目は次のエラーになります（1行目を抜粋）。ファイルは無事なので心配いりません。

コピー

```
New-Item : The file 'C:\Users\you\projects\book-management\types\book.ts' already exists.
```

解説3: 作ったものを一覧で確かめる

コピー

```
Get-ChildItem -Recurse app, components, lib, types | Resolve-Path -Relative
```

- `Get-ChildItem` はフォルダの中身を表示するコマンド（`ls` や `dir` と同じ）です。`-Recurse` を付けると、下の階層まで全部たどります。
- `| Resolve-Path -Relative` は、表示を「いまの場所から見たパス」（`.\app\page.tsx` のような形）に変える指定です。先頭の `.\` は「いまいる場所」という意味です。
- 区切りが `\`（Windows の書き方）になりますが、`/` で書いた場所と同じ意味です。

> **用語:** 「PowerShell（パワーシェル）」とは、Windows に最初から入っているコマンドを打つための道具です。VS Code のターミナルも、Windows では既定で PowerShell が開きます。

4.4 `@/` はどこを指す？（パスエイリアスの解決）

この章のコードでは、`import Header from "@/components/Header";` のように、先頭が `@/` のパスがたくさん出てきます。1.2 の質問⑨で「`@/*` をそのまま使う」と答えたので、`@/` は **プロジェクト直下（`package.json` がある場所）の近道**として働きます。住所にたとえると、`@/` は「本社ビル」という略称です。「本社ビルの3階（components）の Header さん」と書けば、どの支店から手紙を出しても同じ人に届きます。一方、相対パス（`../`）は「ここから2つ戻って右」という道案内なので、出発地点が変わるたびに書き直しが必要です。

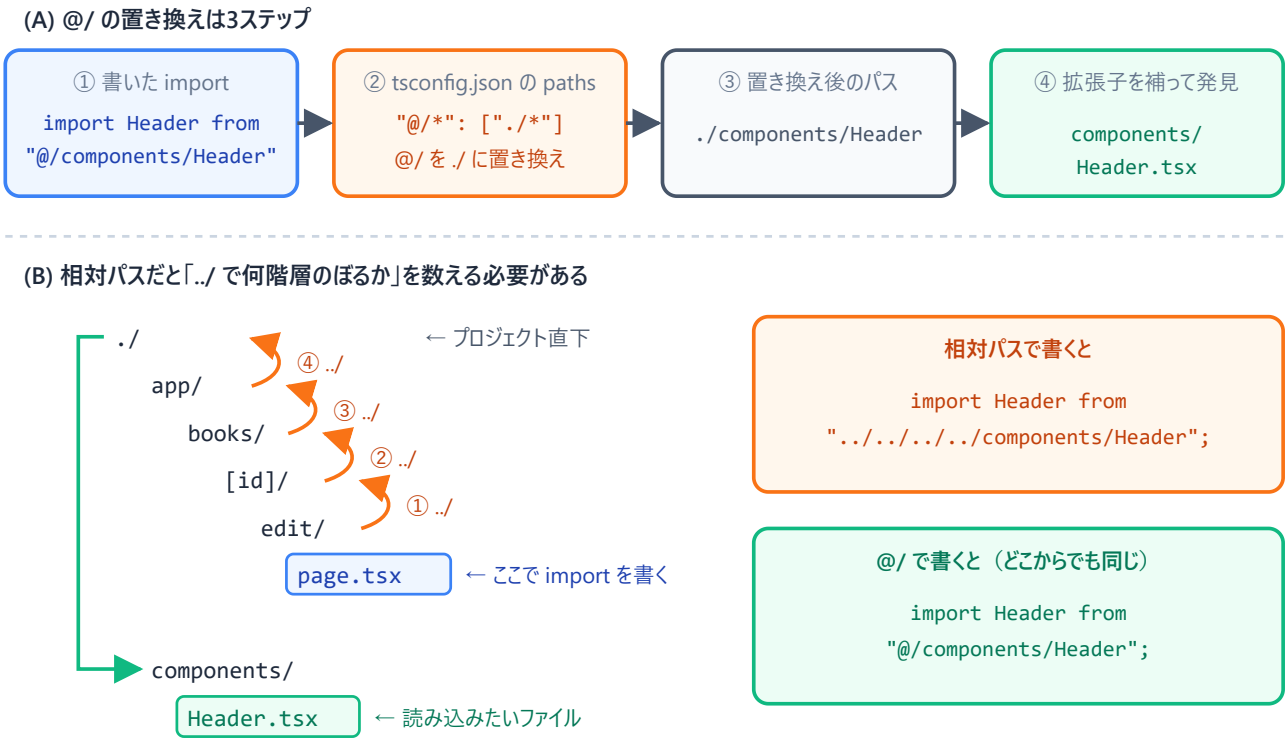


図6-9: パスエイリアス @/ が実際のファイルにたどり着くまで（A）と、相対パスとの比較（B）

(A) のとおり、`@/` の置き換えは `tsconfig.json` の `"paths"` に書かれたルール（`"@/*": ["./*"]`）だけで決まります。`./` は「`tsconfig.json` があるフォルダ」、つまりプロジェクト直下のことです。Next.js も VS Code もこのルールを読んで、同じファイルにたどり着きます。拡張子（`.tsx` や `.ts`）は書かなくても自動で補われます。

4.4.1 つまずきポイント: 古い記事をまねて `@/src/...` と書いてしまう

> **つまずきポイント: `@/src/components/Header` と書いてしまう**
>
> インターネット上の記事には、`src/` フォルダを使う構成（1.2 の質問⑥で Yes を選んだ構成）で書かれたものがたくさんあります。そうした記事をまねて、`@/` のあとに `src/` を付けてしまうミスはとても多いです。本書の構成には `src/` フォルダが無いので、ファイルが見つかりません。

NG（存在しない `src/` を付けている）：

コピー

```
// app/layout.tsx（一部）
// @/ はプロジェクト直下のことなので、./src/components/Header を探しに行ってしまう
import Header from "@/src/components/Header";
```

▼ 実行結果（`npm run build` の出力から抜粋）

コピー

```
> Build error occurred
Error: Turbopack build failed with 1 error:
./app/layout.tsx:17:1
Error: Module not found: Can't resolve '@/src/components/Header'
  15 | // default import: Header コンポーネントを取り込む
  16 | // @/components/Header はプロジェクト直下の components/Header（パスエイリアス @/ = プロジ...
> 17 | import Header from "@/src/components/Header";
     | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  18 |
  19 | /**
  20 |  * フォント設定

Import map: aliased to relative './src/components/Header' inside of [project]/


https://nextjs.org/docs/messages/module-not-found
```

`Module not found: Can't resolve '...'` は「そのファイルが見つかりません」という意味です。最後のほうの `aliased to relative './src/components/Header'` に、`@/` を `./` に置き換えた結果、**プロジェクト直下の `src/components/Header` を探しに行った**ことがはっきり出ています（16行目の `...` は、長い行を Next.js が省略して表示したものです）。

OK（`@/` のあとは、プロジェクト直下からの場所を書く）：

コピー

```
// app/layout.tsx（一部）
// @/ = プロジェクト直下なので、components から書き始める
import Header from "@/components/Header";
```

> **ヒント（ほかの「見つからない」原因）**：同じ `Module not found` は、次のようなときにも出ます。
>
> - **ファイル名の大文字・小文字の違い**：`@/components/header`（小文字の h）。Windows や macOS では動いてしまうことがありますが、本番（第10章の Vercel など Linux のサーバー）ではエラーになります。ファイル名と import の文字は一字一句そろえましょう。
> - **フォルダを違う場所に作った**：`app/components/Header.tsx` のように `app/` の中に `components/` を作ると、`@/components/Header`（＝プロジェクト直下の `components/Header`）では見つかりません。VS Code のエクスプローラーで、`components` が `app` と同じ高さ（`package.json` と同じ場所）にあるか確認してください。

5. Supabase クライアントの設定

5.1 環境変数の取得

Supabase クライアントの設定には、以下の2つの情報が必要です。

1. **Project URL** -- あなたのプロジェクトの API エンドポイント（`https://xxxxxxxxxxxxxxx.supabase.co` の形）
2. **Publishable key（パブリッシャブル・キー）** -- ブラウザに置いてもよい公開用の API キー（`sb_publishable_` で始まる文字列）

これらは Supabase のダッシュボードから取得できます。

取得手順:

1. Supabase のダッシュボード にアクセスします
2. 書籍管理アプリ用のプロジェクト（第5章で作ったもの）を選択します
3. 画面上部の「**Connect**」ボタンをクリックします（左下の「**Project Settings**」（歯車アイコン）→「**API Keys**」からも見られます）
4. 表示された画面から、次の2つをコピーします: - **Project URL**: `https://xxxxxxxxxxxxxxx.supabase.co` の形式 - **Publishable key**: `sb_publishable_` で始まる文字列

> **注意（ダッシュボードの表記）**: Supabase の画面は改良が続いていて、ボタンの位置や名前が変わることがあります。見つからないときは、プロジェクトの画面で「API Keys」や「Connect」という文字を探してください。

> **注意（古い anon key について）**: 以前の Supabase では、Publishable key の代わりに **anon key**（`eyJhbGciOi...` で始まる長い文字列）を使っていました。anon key は同じ「API Keys」の画面にまだあり、今も同じように使えますが、Supabase は **2026年末までに廃止する予定**です。これから作るプロジェクトでは Publishable key を使ってください。どちらを使っても、この後のコードは同じです。

> **注意（Secret key は絶対に使わない）**: 同じ画面に **Secret key**（`sb_secret_` で始まる）も表示されます。これは RLS（第5章で設定した行レベルのセキュリティ）を無視してすべてのデータを読み書きできる「合鍵」です。ブラウザに届く `NEXT_PUBLIC_` 付きの環境変数には**絶対に入れない**でください。本書のアプリでは Secret key は使いません。

5.2 .env.local の作成

プロジェクトのルートディレクトリ（`book-management/` の直下。`package.json` と同じ場所）に `.env.local` ファイルを作成します。

▼ **このコードがやること（先に日本語で）**: Supabase（データベース）に接続するための「URL」と「鍵（キー）」を、コードの外側の設定ファイルに書き出します。こうした外部から渡す設定値を「環境変数」と呼び、`キー=値` の形で1行ずつ書きます。`NEXT_PUBLIC_` を頭に付けた変数だけがブラウザ側からも読めるようになる点がポイントです。`xxxxxxxx` の部分は、必ず自分の Supabase プロジェクトの値に置き換えてください。

コピー

```
# book-management/.env.local
# このファイルは ENV ファイルと呼ばれ、「キー=値」の形で環境変数を1行ずつ書く
# NEXT_PUBLIC_ プレフィックスを付けると Next.js がブラウザ側にも公開してくれる

# Supabase プロジェクトのURL（ダッシュボードからコピー）
NEXT_PUBLIC_SUPABASE_URL=https://xxxxxxxxxxxxxxx.supabase.co

# Supabase の Publishable key（公開用のキー）。名前は第7章以降のコードに合わせて ANON_KEY にする
# 古い anon key（eyJ... で始まる長い文字列）を使う場合も、同じこの行に貼る
NEXT_PUBLIC_SUPABASE_ANON_KEY=sb_publishable_xxxxxxxxxxxxxxxxxxxxxxx
```

なぜ変数名が `ANON_KEY` のままなのか: 中身は Publishable key ですが、変数名は第7章以降のコード（`process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY`）に合わせて `NEXT_PUBLIC_SUPABASE_ANON_KEY` にしています。変数名は「コードとファイルで同じ名前なら何でもよい」ラベルにすぎないので、昔からの名前を使い続けても問題ありません。大切なのは、**`.env.local` の名前とコードの名前が一字一句そろっていること**です（下の 5.2.2 で、そろっていないときの症状を確かめます）。

`NEXT_PUBLIC_` プレフィックスの重要性: Next.js では、環境変数に `NEXT_PUBLIC_` プレフィックスを付けると、ブラウザ側の JavaScript からもアクセスできるようになります。プレフィックスがない環境変数はサーバー側でのみ使用可能です。

Supabase の Publishable key（や旧 anon key）はブラウザから直接 API を呼び出すために使うため、`NEXT_PUBLIC_` が必要です。このキーは Row Level Security（RLS）で守られる前提の公開用キーなので、ブラウザに届いても安全です（第5章で RLS を設定しました）。

`.env.local` に書いた値が、どうやってコードに届くのかを図にすると次のようになります。

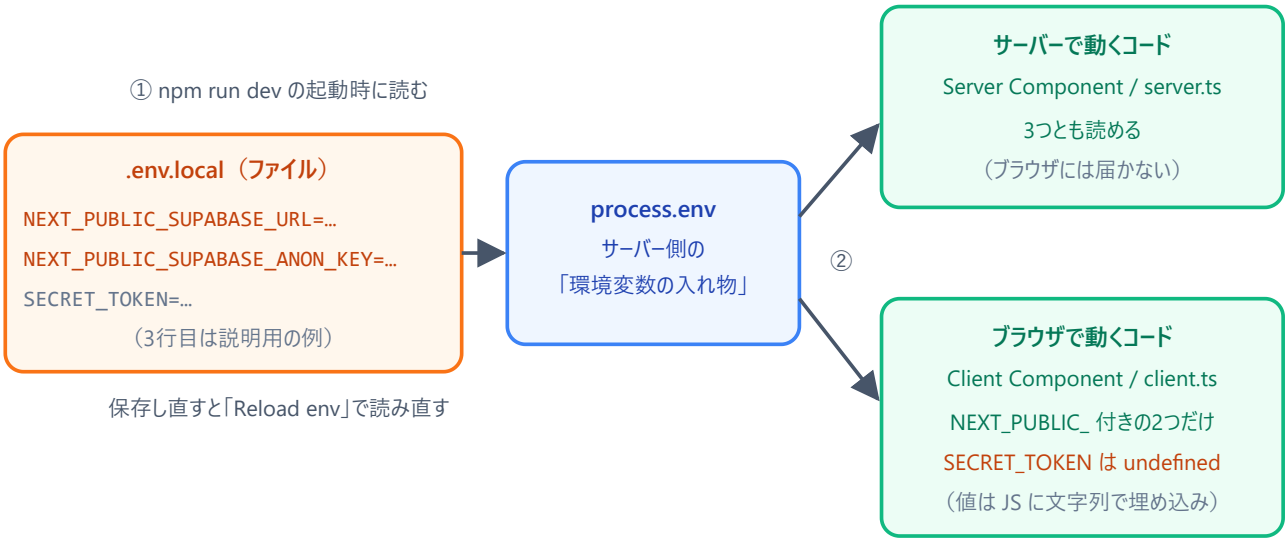


図6-10: .env.local の値がコードに届くまで（NEXT_PUBLIC_ 付きだけがブラウザまで届く）

ポイントは②です。ブラウザで動くコードには `process.env` という入れ物がそもそもありません。そこで Next.js は、ブラウザ用の JavaScript を作るときに `process.env.NEXT_PUBLIC_SUPABASE_URL` という部分を見つけて、**値そのもの（`"https://....supabase.co"` という文字列）に書き換えて埋め込みます**。実際に開発サーバーが作ったブラウザ用のファイルを調べると、`NEXT_PUBLIC_` 付きの値は文字列として埋め込まれ、付いていない変数は `process.env.SECRET_TOKEN` のまま（ブラウザでは中身がないので `undefined`）になっていました。

ENV ファイルの種類と読み込み順（Next.js）：Next.js では複数の `.env` 系ファイルを使い分けられます。同じ変数が複数のファイルにある場合、上のものが優先されます。

ファイル	いつ読み込まれるか	Gitに入れる？
`.env.local`	全環境（開発・本番ともに）。ただしテスト環境（`NODE_ENV=test`）では読まれない	**入れない**（.gitignoreで除外）
`.env.development`	開発時（`npm run dev`）のみ	既定では除外される（秘密を含まないなら共有してもよい）
`.env.production`	本番時（`npm run build` / `npm run start`）のみ	既定では除外される（秘密を含まないなら共有してもよい）
`.env`	すべての環境（デフォルト値）	既定では除外される（秘密を含まないなら共有してもよい）

読み込み優先順位（高 → 低）：`.env.local` > `.env.development` / `.env.production` > `.env`

覚え方:「`.local` が付くファイルは個人専用＝Gitに入れない＝秘密情報を入れてOK」。`.env.local` だけは絶対にコミットしない、と覚えれば事故を防げます。

> **`.env.local` がGitにアップロードされない理由:** `create-next-app` が自動生成した `.gitignore` ファイルには `.env*` が含まれています（ `.env` で始まるファイルはすべて除外、という意味）。そのため、`.env.local` は Git にコミットされず、API キーが公開リポジトリに漏洩する心配はありません。

必ず `xxxxxxxxxxxxxxxx` の部分を、自分のプロジェクトの値に置き換えてください。

5.2.1 サンプル: `.env.local` が読めているかを確かめるスクリプト

> **▼ このコードがやること（先に日本語で）:** アプリを起動する前に、`.env.local` に必要な2つの値が正しい名前で書かれているかを確かめる小さなスクリプトです。Node.js の `--env-file` オプションで `.env.local` を読み込ませてから実行し、値があれば `OK`、無ければ `NG` と表示します。秘密を画面に出しすぎないよう、値は先頭20文字だけ見せます。

コピー

```
// scripts/check-env.ts
// .env.local に必要な2つの値が書かれているかを、アプリを起動せずに確かめるスクリプト

// 確かめたい環境変数の名前を配列にまとめる
const names = ["NEXT_PUBLIC_SUPABASE_URL", "NEXT_PUBLIC_SUPABASE_ANON_KEY"];

// 1つずつ取り出して、値があるかどうかを表示する
for (const name of names) {
  // process.env[name] で、その名前の環境変数の値を読む（無ければ undefined）
  const value = process.env[name];
  if (value) {
    // 秘密を画面に出しすぎないよう、先頭20文字だけ見せる
    console.log(`OK  ${name} = ${value.slice(0, 20)}...`);
  } else {
    console.log(`NG  ${name} が見つかりません`);
  }
}
```

`book-management/` の中に `scripts` フォルダを作ってこのファイルを保存し、プロジェクトのフォルダで次のように実行します。

コピー

```
# --env-file=.env.local: このファイルの中身を環境変数として読み込んでから実行する
node --env-file=.env.local scripts/check-env.ts
```

▼ 実行結果（説明用のダミーの値で実行したもの）

コピー

```
OK  NEXT_PUBLIC_SUPABASE_URL = https://abcdefghijkl...
OK  NEXT_PUBLIC_SUPABASE_ANON_KEY = sb_publishable_dummy...
```

ファイル名を打ち間違えると（`.env.locl`）、スクリプトが動く前に Node.js が止まります。

コピー

```
# わざとファイル名を打ち間違える
node --env-file=.env.locl scripts/check-env.ts
```

▼ 実行結果

コピー

```
C:\Program Files\nodejs\node.exe: .env.locl: not found
```

▼ コードを1つずつ分解して解説

解説1: 名前の一覧を配列にして、順番に調べる

コピー

```
const names = ["NEXT_PUBLIC_SUPABASE_URL", "NEXT_PUBLIC_SUPABASE_ANON_KEY"];

for (const name of names) {
  const value = process.env[name];
```

- 調べたい名前を配列（`[ ]` で囲んだ並び）にまとめ、`for (const name of names)` で1つずつ取り出しています。
- `process.env[name]` は、`process.env.NEXT_PUBLIC_SUPABASE_URL` と書くのと同じ意味です。名前を変数で指定したいときは `[ ]` を使います。

解説2: 値があれば OK、無ければ NG

コピー

```
if (value) {
  console.log(`OK  ${name} = ${value.slice(0, 20)}...`);
} else {
  console.log(`NG  ${name} が見つかりません`);
}
```

- 値が無いとき `process.env[name]` は `undefined` になり、`if (value)` は偽になります。

- `value.slice(0, 20)` は「先頭から20文字だけ切り出す」という意味です。キーを丸ごと画面に出さないための工夫です。

解説3: `--env-file` は Node.js の機能

コピー

```
node --env-file=.env.local scripts/check-env.ts
```

- `--env-file=ファイル名` を付けると、Node.js がそのファイルを読んで `process.env` に入れてから実行してくれます。Next.js が `.env.local` を読む仕組みとは別ですが、「書いた名前と値がちゃんと読めるか」を確かめるには十分です。
- `.ts` ファイルをそのまま実行できるのは、第2章で学んだとおり Node.js が型の部分を消して動かしてくれるからです（Node.js 24 以降なら追加の設定なしで動きます）。

> **用語:** 「スクリプト」とは、手作業の確認や準備を自動でやってくれる小さなプログラムのことです。アプリ本体とは別に `scripts/` のようなフォルダにまとめておくのが一般的です。

5.2.2 つまずきポイント: 変数名がコードとそろっていない

> **つまずきポイント: Supabase の画面に出ている変数名をそのまま使ってしまう**
>
> Supabase の「Connect」画面には、Next.js 用の設定例として `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY` という名前が表示されることがあります。これをそのまま貼ると、本書のコードが読む `NEXT_PUBLIC_SUPABASE_ANON_KEY` と名前が違うため、キーが「無い」ことになります。

NG（変数名がコードと違う）：

コピー

```
# book-management/.env.local
NEXT_PUBLIC_SUPABASE_URL=https://xxxxxxxxxxxxxxx.supabase.co
# 名前が PUBLISHABLE_KEY になっている（コードは ANON_KEY を読む）
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=sb_publishable_xxxxxxxxxxxxxxxxxxxxxxx
```

この状態で 5.2.1 のスクリプトを実行すると、2つ目が NG になります。

▼ 実行結果

コピー

```
OK  NEXT_PUBLIC_SUPABASE_URL = https://abcdefghijkl...
NG  NEXT_PUBLIC_SUPABASE_ANON_KEY が見つかりません
```

OK（値はそのまま、名前だけ本書のコードに合わせる）：

コピー

```
# book-management/.env.local
NEXT_PUBLIC_SUPABASE_URL=https://xxxxxxxxxxxxxxx.supabase.co
# 名前を ANON_KEY にする（値は Publishable key のまま）
NEXT_PUBLIC_SUPABASE_ANON_KEY=sb_publishable_xxxxxxxxxxxxxxxxxxxxxxx
```

5.2.3 つまずきポイント: `.env.local` の名前や場所が違う

> **つまずきポイント: `.env.local.txt` になっている／ `app/` の中に置いている**
>
> Windows のメモ帳で保存すると、`.env.local.txt` のように `.txt` が勝手に付くことがあります（エクスプローラーの設定によっては `.txt` が見えません）。また、`app/` などのフォルダの中に置いてしまうと Next.js は読みません。`.env.local` は **`package.json` と同じ場所**に、**拡張子なしの名前**で置きます。

NG（名前や場所が違う）：

コピー

```
book-management/.env.local.txt      ← .txt が付いている
book-management/app/.env.local      ← app/ の中にある
```

OK（`package.json` と同じ場所に、正しい名前で置く）：

コピー

```
book-management/.env.local
book-management/package.json
```

見分け方は簡単です。`npm run dev` の起動時の表示に `- Environments: .env.local` の行があるかを見てください。この行が無ければ、Next.js は `.env.local` を見つけられていません。次は `.env.local.txt` しか無い状態で起動し、Supabase を使うページ（第7章で作るトップページと同じ形のもの）を開いたときの実際の表示です。

▼ 実行結果

コピー

```
⚠ Port 3000 is in use by process 50292, using available port 3001 instead.
▲ Next.js 16.4.1 (Turbopack)
- Local:         http://localhost:3001
- Network:       http://192.168.x.x:3001
✓ Ready in 2.9s
✓ Running next.config.ts took 59ms

○ Compiling / ...
× Error: 環境変数 NEXT_PUBLIC_SUPABASE_URL / NEXT_PUBLIC_SUPABASE_ANON_KEY が設定されていません。.env.local ファイルを確認してください。
    at createClient (lib\supabase\server.ts:18:11)
    at Page (app\page.tsx:4:38)
  16 |   // どちらかが空なら、原因と直し方が分かるメッセージでわざと止める
  17 |   if (!supabaseUrl || !supabaseKey) {
> 18 |     throw new Error(
     |           ^
  19 |       "環境変数 NEXT_PUBLIC_SUPABASE_URL / NEXT_PUBLIC_SUPABASE_ANON_KEY が設定されていません。" +
  20 |         ".env.local ファイルを確認してください。"
  21 |     ); {
  digest: '310640217'
}
 GET / 500 in 580ms (next.js: 259ms, application-code: 321ms)
```

（このときは別のアプリが 3000 番を使っていたので、1行目のとおり自動で 3001 番で起動しています。9.4 の表も見てください。） `- Environments: .env.local` の行が無く、次の 5.3 で作る `server.ts` のエラーチェックが「設定されていません」と知らせています。`GET / 500` の 500 は「サーバー側でエラーが起きた」という意味です。

5.2.4 つまずきポイント: `.env.local` を直したのに反映されない

> **つまずきポイント: 直したつもりが、開発サーバーに伝わっていない**
>
> Next.js 16 の開発サーバーは、`.env.local` を**保存し直すと自動で読み直して**くれます。そのときターミナルに `Reload env: .env.local` と表示されます。

上の 5.2.3 の状態のまま、エディタで `.env.local` を正しい名前で保存したときの実際の表示です。

▼ 実行結果

コピー

```
  Reload env: .env.local
✓ Compiled in 26ms
 GET / 200 in 7.5s (next.js: 61ms, application-code: 7.4s)
```

`GET / 200`（200 は「成功」）に変わりました。ただし、**ファイル名を変更しただけ**（`.env.local.txt` → `.env.local`）のときは、この自動読み直しが起きないことがありました。筆者の実験でも、名前を変えただけでは `500` のままで、もう一度エディタで保存したところで `Reload env` が出て直りました。

NG（`Reload env` が出ていないのに、ブラウザだけ再読み込みし続ける）：

コピー

```
（ターミナルに Reload env: .env.local が出ていない）
```

OK（確実な直し方: 開発サーバーを起動し直す）：

コピー

```
# ターミナルで Ctrl+C を押して開発サーバーを止めてから、もう一度起動する
npm run dev
```

起動し直したら、`- Environments: .env.local` の行が出ていることを確認してください。「迷ったら再起動」と覚えておけば、`.env.local` まわりの「直したのに直らない」はほぼ防げます。

5.3 lib/supabase/client.ts と server.ts の作成

データベースとやり取りするための「窓口（Supabase クライアント）」を作るファイルを2つ用意します。窓口が2つあるのは、Next.js のコードには**サーバーで動くもの**と**ブラウザで動くもの**の2種類があるからです。お店にたとえると、`server.ts` は**店の奥の事務所の電話**、`client.ts` は**店頭のお客さん用の電話**です。どちらも同じ倉庫（Supabase）につながりますが、置き場所と使い方が少し違います。

> **注意（第5章で作った `lib/supabase.ts` について）**：第5章では、接続の動作確認のために `lib/supabase.ts` という1ファイル版のクライアントを作りました。第7章以降は、この章で作る2つのファイル（`lib/supabase/server.ts` と `lib/supabase/client.ts`）を使います。1ファイル版は役目を終えたので削除して構いません（`lib/supabase.ts` と `lib/supabase/` フォルダが並んでいると、どちらを使うのか紛らわしくなるためです）。

ファイル	使う場所	呼び出し方	第7章以降の例
`lib/supabase/server.ts`	Server Component（`"use client"` が無いファイル）	`const supabase = await createClient();`	トップページ `app/page.tsx` で書籍一覧を読む
`lib/supabase/client.ts`	Client Component（`"use client"` があるファイル）	`const supabase = createClient();`	登録フォームや削除ボタンで書き込む

5.3.1 lib/supabase/client.ts（ブラウザ用）

`lib/supabase/client.ts` ファイルに以下のコードを記述します。

> **▼ このコードがやること（先に日本語で）**：ブラウザで動く部品（Client Component）から使う、Supabase への窓口を作る関数 `createClient` を用意します。さきほど `.env.local` に書いた URL とキーを読み込み、もし設定が抜けていたら、原因がすぐ分かるエラーをわざと出してから止まります。設定がそろっていれば、`@supabase/ssr` の `createBrowserClient` に渡して窓口を作り、返します。

コピー

```
// lib/supabase/client.ts
// ブラウザで動く部品（Client Component）から使う Supabase クライアントを作るファイル

// @supabase/ssr から「ブラウザ用のクライアントを作る関数」を名前付きインポートする
import { createBrowserClient } from "@supabase/ssr";

// Client Component から呼び出す関数。呼ぶたびに Supabase への窓口を作って返す
export function createClient() {
  // .env.local に書いた接続先 URL を読み出す
  const supabaseUrl = process.env.NEXT_PUBLIC_SUPABASE_URL;
  // .env.local に書いた Publishable key（旧 anon key でも可）を読み出す
  const supabaseKey = process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY;

  // どちらかが空なら、原因と直し方が分かるメッセージでわざと止める
  if (!supabaseUrl || !supabaseKey) {
    throw new Error(
      "環境変数 NEXT_PUBLIC_SUPABASE_URL / NEXT_PUBLIC_SUPABASE_ANON_KEY が設定されていません。" +
        ".env.local ファイルを確認してください。"
    );
  }

  // URL とキーを渡して、ブラウザ用のクライアントを作って返す
  return createBrowserClient(supabaseUrl, supabaseKey);
}
```

5.3.2 lib/supabase/server.ts（サーバー用）

`lib/supabase/server.ts` ファイルに以下のコードを記述します。

> **▼ このコードがやること（先に日本語で）**：サーバーで動く部品（Server Component）から使う窓口を作る関数 `createClient` を用意します。前半の「環境変数を読んで、無ければ止める」部分は `client.ts` と同じです。違うのは後半で、サーバーでは「いま届いたリクエストの Cookie（クッキー：ブラウザが持っている小さなメモ）」を Supabase に読み書きさせる方法も一緒に渡します。Cookie を受け取るのに時間がかかる場合があるので、関数に `async` を付けています。

コピー

```
// lib/supabase/server.ts
// サーバーで動く部品（Server Component）から使う Supabase クライアントを作るファイル

// @supabase/ssr から「サーバー用のクライアントを作る関数」を名前付きインポートする
import { createServerClient } from "@supabase/ssr";
// Next.js が用意している「いま届いたリクエストの Cookie を読み書きする関数」
import { cookies } from "next/headers";

// Server Component から呼び出す関数。cookies() を待つので async にする
export async function createClient() {
  // .env.local に書いた接続先 URL を読み出す
  const supabaseUrl = process.env.NEXT_PUBLIC_SUPABASE_URL;
  // .env.local に書いた Publishable key（旧 anon key でも可）を読み出す
  const supabaseKey = process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY;

  // どちらかが空なら、原因と直し方が分かるメッセージでわざと止める
  if (!supabaseUrl || !supabaseKey) {
    throw new Error(
      "環境変数 NEXT_PUBLIC_SUPABASE_URL / NEXT_PUBLIC_SUPABASE_ANON_KEY が設定されていません。" +
        ".env.local ファイルを確認してください。"
    );
  }

  // いま処理しているリクエストの Cookie 置き場を受け取る
  const cookieStore = await cookies();

  // URL・キー・Cookie の読み書き方法を渡して、サーバー用のクライアントを作って返す
  return createServerClient(supabaseUrl, supabaseKey, {
    cookies: {
      // Supabase が Cookie を読みたいときに呼ばれる
      getAll() {
        return cookieStore.getAll();
      },
      // Supabase が Cookie を書きたいときに呼ばれる
      setAll(cookiesToSet) {
        try {
          // 受け取った Cookie を1つずつ書き込む
          cookiesToSet.forEach(({ name, value, options }) =>
            cookieStore.set(name, value, options)
          );
        } catch {
          // Server Component の表示中は Cookie を書けないので、ここに来ても無視してよい
          // （このアプリはログイン機能を使わないため、実害はない）
        }
      },
    },
  });
}
```

この2つのファイルは、Next.js 16.4.1・`@supabase/ssr` 0.12.7 で `npm run build`（型チェックを含む本番用ビルド）が通ることを確認しています。

コードの解説:

部分	解説
`import { createBrowserClient } from "@supabase/ssr";`	Supabase の Next.js 向けライブラリから、ブラウザ用の窓口を作る関数を取り出します。中括弧 `{}` で囲むのが「named import（名前付きインポート）」で、ライブラリが公開している特定の関数だけを取り出します。
`import { createServerClient } from "@supabase/ssr";`	同じライブラリから、サーバー用の窓口を作る関数を取り出します。
`import { cookies } from "next/headers";`	Next.js の、リクエストの Cookie を扱う関数です。サーバーでしか使えません（ブラウザ用の `client.ts` では使いません）。
`process.env.NEXT_PUBLIC_SUPABASE_URL`	`.env.local` に定義した環境変数を読み取ります。`process.env` は環境変数が入っている入れ物です。
`if (!supabaseUrl || !supabaseKey)`	どちらかが未設定（`undefined` や空文字）の場合にエラーを投げます。`.env.local` の作成忘れや変数名の打ち間違いを早く見つけられます。
`export function createClient()` / `export async function createClient()`	窓口を作る関数を外に公開します。2つのファイルで**同じ名前**にしているので、使う側は import 元（`client` か `server` か）を変えるだけで済みます。
`createServerClient(url, key, { cookies: { getAll, setAll } })`	サーバー用の窓口を作ります。3つ目の引数で「Cookie の読み方（`getAll`）と書き方（`setAll`）」を教えています。

import 文の種類（重要）: JavaScript/TypeScript には3種類のインポート方法があります。

- **default import**: `import React from "react";` のように中括弧なし。各ファイルにつき1つだけ `export default` できる主要なものを取り出す。
- **named import**: `import { useState } from "react";` のように中括弧あり。複数の名前を指定して取り出せる（例: `import { useState, useEffect } from "react";`）。
- **type import**: `import type { Book } from "@/types/book";` のように `type` キーワード付き。型のみをインポートし、ビルド時に取り除かれる（JavaScriptには型がないため）。

なぜ「1つの supabase を共有」ではなく「呼ぶたびに作る関数」なのか: 古い記事では、`export const supabase = createClient(...)` のように**1つだけ作って全ファイルで使い回す**書き方（シングルトンパターン）がよく出てきます。ブラウザだけで動くアプリならそれでも動きますが、Next.js のサーバーは**たくさんの人のリクエストを同時に**処理します。サーバー用の窓口は「いま処理しているリクエストの Cookie」を使うので、1つを全員で共有すると、Aさんの Cookie で Bさんの処理をしてしまう恐れがあります。そこで、**使う場所で `createClient()` を呼んで、その場で窓口を作る**書き方にしています。

Server Component と Client Component で Supabase を使うときの違い（重要）：

Next.js の App Router では、コンポーネントが「Server Component」（サーバー側で実行）か「Client Component」（ブラウザ側で実行）かを区別します。それぞれで Supabase を使う際の注意点が異なります。

観点	Server Component	Client Component
ファイル先頭の宣言	（宣言なし。デフォルト）	`"use client"` を書く
実行場所	サーバー（あなたのPC or 本番サーバー）	ユーザーのブラウザ
使う窓口	`@/lib/supabase/server` の `createClient`（`await` が必要）	`@/lib/supabase/client` の `createClient`
使える React フック	使えない（useState など不可）	使える
データ取得	`await supabase.from(...)` を直接書ける	ボタンを押したときの処理（イベント）などの中で呼ぶ
環境変数	サーバー専用変数（`NEXT_PUBLIC_` なし）も読める	`NEXT_PUBLIC_` 付きの変数のみ
このチュートリアル	一覧ページ・詳細ページなど、データを**読んで表示**する画面	登録フォームや削除ボタンなど、データを**書き込む**操作

第7章では、トップページ（`app/page.tsx`）を Server Component にして `server.ts` の窓口で書籍一覧を読み、登録フォーム（`app/books/new/page.tsx`）を Client Component にして `client.ts` の窓口で書き込みます。

▼ コードを1つずつ分解して解説

上のコードを、初心者がつまずきやすい部分ごとに**1つずつ**ていねいに見ていきましょう。

解説1: 窓口を作る関数を「名前付きインポート」する

コピー

```
import { createBrowserClient } from "@supabase/ssr";
```

- `import ... from "..."` は、別のファイルやライブラリから機能を**取り込む**ための構文です。
- `{ createBrowserClient }` のように波括弧 `{ }` で囲むのが「名前付きインポート」で、そのライブラリが公開している**特定の名前の機能だけ**をピンポイントで取り出します。

- ここでは `@supabase/ssr`（節3でインストールした Supabase の Next.js 向けライブラリ）から、`createBrowserClient`（ブラウザ用の窓口を作る関数）だけを取り出しています。`server.ts` では同じように `createServerClient` を取り出します。

> **用語:**「ライブラリ」とは、誰かが作って公開してくれた便利な機能のかたまりのことです。`npm install` で自分のプロジェクトに取り込み、`import` で必要な部分だけ呼び出して使います。

解説2: 環境変数から接続情報を読み取る

コピー

```
// .env.local に書いた接続先 URL を読み出す
const supabaseUrl = process.env.NEXT_PUBLIC_SUPABASE_URL;
// .env.local に書いた Publishable key（旧 anon key でも可）を読み出す
const supabaseKey = process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY;
```

- `process.env`（プロセス・エンブ）は、**環境変数の入れ物**です。さきほど `.env.local` に書いた値が、ここに入ってきます。
- `process.env.NEXT_PUBLIC_SUPABASE_URL` と書くと、`.env.local` の `NEXT_PUBLIC_SUPABASE_URL=...` の値（URL）を取り出せます。
- `process.env.〇〇` は**省略せずにこの形のまま**書くのが大切です。ブラウザ用のファイルでは、Next.js がこの形の部分を探して値に書き換えるので（図の②）、`const env = process.env;` のように一度まとめてから取り出すと、書き換えが働かず `undefined` になってしまいます。
- URL とキーをコードに直接書かず、外部の設定ファイル経由で渡すことで、**秘密情報の漏洩を防ぎ**、環境ごとに値を切り替えられます。

> **用語:**「環境変数」とは、コードの外側からプログラムへ渡す設定値のことです。API キーのような秘密情報や、開発用と本番用で変えたい値を、コード本体と分けて管理するために使います。

解説3: 設定が抜けていたらわざとエラーを出す

コピー

```
if (!supabaseUrl || !supabaseKey) {
  throw new Error(
    "環境変数 NEXT_PUBLIC_SUPABASE_URL / NEXT_PUBLIC_SUPABASE_ANON_KEY が設定されていません。" +
      ".env.local ファイルを確認してください。"
  );
}
```

- `!supabaseUrl` の `!`（エクスクラメーション）は「否定」の記号で、「`supabaseUrl` が空っぽ（`undefined` や空文字）なら `true` 」という意味になります。`||` は「または」です。

- つまりこの `if` は「URL かキーのどちらかがちゃんと取れなかったとき」だけ中に入ります。
- `throw new Error(...)` は、エラーを**わざと発生させて処理を止める**書き方です。原因（環境変数が未設定）と対処法（`.env.local` を確認）をメッセージに書いておくと、5.2.3 で見たように、エラー画面を見ただけで原因が分かります。
- `+` は文字列をつなげる演算子で、長いメッセージを2行に分けて書いています。
- このチェックを通り抜けた後は、TypeScript も「`supabaseUrl` と `supabaseKey` は空ではない文字列だ」と分かるので、次の行で安心して渡せます。

> **用語:** 「例外（throw / Error）」とは、処理を続けられない問題が起きたときに発生させる中断の合図です。早い段階でわかりやすく失敗させることを「フェイルファスト（fail fast）」と呼び、原因究明を楽にします。

解説4: サーバー用は Cookie の読み書き方法も渡す

コピー

```
const cookieStore = await cookies();

return createServerClient(supabaseUrl, supabaseKey, {
  cookies: {
    getAll() {
      return cookieStore.getAll();
    },
    setAll(cookiesToSet) {
      try {
        cookiesToSet.forEach(({ name, value, options }) =>
          cookieStore.set(name, value, options)
        );
      } catch {
        // Server Component の表示中は Cookie を書けないので、ここに来ても無視してよい
      }
    },
  },
});
```

- `await cookies()` で、いま処理しているリクエストの Cookie 置き場を受け取ります。`await` を使うので、関数全体に `async` が付いています（だから使う側も `await createClient()` と書きます）。
- `getAll()` は「Cookie を全部見せて」、`setAll()` は「この Cookie を書いておいて」と Supabase から頼まれたときの答え方です。
- `try { ... } catch { ... }` は「やってみて、失敗したら何もしない」という書き方です。Server Component の表示中は Cookie を書き込めない決まりがあり、そのときに出る失敗を無視しています。ログイン機能を使わない本書のアプリでは、ここで困ることはありません。
- ブラウザ用の `createBrowserClient` は、ブラウザが自分で Cookie を扱えるので、この3つ目の引数がいりません。

> **用語:**「Cookie（クッキー）」とは、Webサイトがブラウザに預けておく小さなメモです。ログイン状態を覚えておく、などに使われます。

解説5: 2つのファイルで同じ名前 `createClient` にしている理由

コピー

```
// サーバーで動くページでの使い方（第7章のトップページなど）
import { createClient } from "@/lib/supabase/server";

// ブラウザで動く部品での使い方（第7章の登録フォームなど）
import { createClient } from "@/lib/supabase/client";
```

- どちらも関数名は `createClient` です。違うのは **import 元のパス**（`server` か `client` か）だけです。
- 使う側は「このファイルはサーバー？ブラウザ？」を考えて、import 元を選ぶだけで済みます。
- 逆を選ぶとエラーになります。たとえば `"use client"` の部品で `server` の方を import すると、ブラウザでは使えない `cookies()` を呼ぼうとして失敗します。迷ったら、ファイルの先頭に `"use client"` があるかどうかで判断してください。

> **用語:**「export」は、ファイルの中で作ったものを**外部に公開する**指定です。逆に他ファイルでそれを受け取るのが「import」です。この2つでファイル間の機能の受け渡しを行います。

---

6. 型定義ファイルの作成

6.1 types/book.ts の作成

`types/book.ts` ファイルに以下のコードを記述します。この型定義は、アプリ全体で書籍データの構造を統一するために使います。項目（列）は、第5章で作った `books` テーブルと同じ並び・同じ名前にしています。

> **▼ このコードがやること（先に日本語で）:**「書籍データはどんな形をしているか」を TypeScript の型として書き出し、アプリ全体で構造を統一します。型とは「このデータには title と author が必須」といった設計図のことで、間違ったデータを使うとコードを動かす前に気づけます。場面ごとに必要な項目が違うため、取得用の `Book`・登録用の `BookInsert`・更新用の `BookUpdate` の3つに分けている点が要チェックです。後半ではステータスの日本語ラベルや色も定数としてまとめています。

コピー

```
// types/book.ts
// 書籍データに関する型と定数をまとめて定義するファイル

/**
 * 読書ステータスの型定義
 *
 * - "reading"      : 読書中（今読んでいる途中）
 * - "completed"    : 読了（読み終わった）
 * - "want_to_read" : 読みたい（これから読む）
 */
// type: TypeScript の型エイリアス（型に名前を付ける構文）
// "reading" | "completed" | "want_to_read": ユニオン型。この3つの文字列のいずれかしか取れない
// export: 他のファイルからこの型をインポートできるようにする
export type BookStatus = "reading" | "completed" | "want_to_read";

/**
 * 書籍データの型定義（データベースから取得した完全なデータ）
 *
 * Supabase の books テーブル（第5章で作成）の1行に対応します。
 * すべての列が含まれており、データベースから取得した
 * 書籍データはこの型に一致します。
 */
// type Book = { ... }: オブジェクトの形を表す型を Book という名前で定義
export type Book = {
  /** 書籍ID（UUID形式、Supabase が自動生成） */
  // UUID は "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx" 形式の文字列
  id: string;

  /** 書籍のタイトル（必須） */
  // 例: "リーダブルコード"
  title: string;

  /** 著者名（必須） */
  // 例: "Dustin Boswell"
  author: string;

  /** 出版社（任意、未入力の場合は null） */
  // string | null: 文字列か null のどちらか（ユニオン型）
  publisher: string | null;

  /** 出版日（任意。"2024-01-15" のような日付の文字列、未入力なら null） */
  // データベースの date 型は、JavaScript では文字列として届く
  published_date: string | null;

  /** 評価（1〜5の整数、未評価の場合は null） */
  // number | null: 数値か null のどちらか
  rating: number | null;

  /** 読書ステータス（デフォルト: "want_to_read"） */
  // 上で定義した BookStatus 型（3つの文字列のどれか）
  status: BookStatus;

  /** メモ・感想（任意、未入力の場合は null） */
  // 文字列または null
```

```
  memo: string | null;

  /** 作成日時（ISO 8601 形式の文字列、Supabase が自動生成） */
  // 例: "2026-10-15T10:30:00.000Z"
  created_at: string;

  /** 更新日時（ISO 8601 形式の文字列、Supabase が自動生成） */
  // 行が更新されるたびに Supabase のトリガーが書き換える
  updated_at: string;
};

/**
 * 書籍の新規登録時に送るデータの型定義
 *
 * id, created_at, updated_at はデータベースが自動生成するため、
 * 送信データには含めません。
 * title と author は必須、それ以外はオプショナル（省略可能）です。
 */
export type BookInsert = {
  /** 書籍のタイトル（必須） */
  // 必須項目（? が付いていない）
  title: string;

  /** 著者名（必須） */
  // 必須項目。空文字や null にせず、必ず名前を入れて送る
  author: string;

  /** 出版社（省略時は null） */
  // ?: オプショナル。省略可能
  publisher?: string | null;

  /** 出版日（省略時は null） */
  // 省略してもよいし、明示的に null を送ってもよい
  published_date?: string | null;

  /** 評価（1〜5、省略時は null） */
  // 同上
  rating?: number | null;

  /** 読書ステータス（省略時は "want_to_read" がデフォルト） */
  // 省略時の値はDB側のデフォルトで補う
  status?: BookStatus;

  /** メモ・感想（省略時は null） */
  // 同上
  memo?: string | null;
};

/**
 * 書籍の更新時に送るデータの型定義
 *
 * 更新したい列だけを送ればよいため、
 * すべての項目がオプショナルです。
 * Partial<BookInsert> と同じ意味ですが、明示的に定義しています。
 */
export type BookUpdate = {
```

```
  /** 書籍のタイトル */
  // すべて ? 付き（更新時は変更したい項目のみ送る）
  title?: string;

  /** 著者名 */
  // 同上（送る場合は空にしない）
  author?: string;

  /** 出版社（null で消去） */
  // 同上
  publisher?: string | null;

  /** 出版日（null で消去） */
  // 同上
  published_date?: string | null;

  /** 評価（1〜5、null で評価をクリア） */
  // null を明示的に送ると評価をリセットできる
  rating?: number | null;

  /** 読書ステータス */
  // 同上
  status?: BookStatus;

  /** メモ・感想（null でメモをクリア） */
  // 同上
  memo?: string | null;
};

/**
 * ステータスの表示ラベル定義
 *
 * コンポーネントでステータスを日本語表示するときに使います。
 * 例: statusLabels["reading"] → "読書中"
 */
// Record<BookStatus, string>: TypeScript の組み込み型。
// 「キーが BookStatus 型、値が string 型のオブジェクト」を意味する
// 3つのキー（reading, completed, want_to_read）すべてが必須になる
export const statusLabels: Record<BookStatus, string> = {
  // "reading" の表示用ラベル
  reading: "読書中",
  // "completed" の表示用ラベル
  completed: "読了",
  // "want_to_read" の表示用ラベル
  want_to_read: "読みたい",
};

/**
 * ステータスに対応する色のクラス定義
 *
 * StatusBadge コンポーネントでバッジの色を切り替えるときに使います。
 * Tailwind CSS のクラス名を直接指定しています（第7章の StatusBadge と同じ色）。
 */
// Record<BookStatus, { bg: string; text: string }>:
// キーが BookStatus、値が { bg: string; text: string } 型のオブジェクト
// 各ステータスごとに「背景色クラス」と「文字色クラス」を持つ
```

```
export const statusColors: Record<
  BookStatus,
  { bg: string; text: string }
> = {
  // 読書中: 青系（進行中の印象）
  reading: {
    // 背景は薄い青
    bg: "bg-blue-100",
    // 文字は濃い青
    text: "text-blue-800",
  },
  // 読了: 緑系（完了の印象）
  completed: {
    // 背景は薄い緑
    bg: "bg-green-100",
    // 文字は濃い緑
    text: "text-green-800",
  },
  // 読みたい: 黄系（これから、の印象）
  want_to_read: {
    // 背景は薄い黄
    bg: "bg-yellow-100",
    // 文字は濃い黄
    text: "text-yellow-800",
  },
};
```

コードの解説:

型 / 定数	用途
`BookStatus`	読書ステータスを3つのリテラル型のユニオンで定義します。`"reading" | "completed" | "want_to_read"` 以外の値を代入しようとすると、TypeScript がコンパイルエラーを出します。第5章のテーブルの CHECK 制約（この3つ以外は保存できない決まり）と同じ3つです。
`Book`	データベースの `books` テーブルの1行に対応する型です。`supabase.from("books").select("*")` の返り値はこの型の配列になります。
`BookInsert`	新規登録時に必要なデータの型です。`title` と `author` が必須、それ以外はオプショナル（`?` 付き）です。`id` や `created_at` はデータベースが自動で付与するため含みません。
`BookUpdate`	更新時に送るデータの型です。すべての項目がオプショナルで、変更したい項目だけを送ります。
`statusLabels`	ステータスの文字列を日本語ラベルに変換するためのマッピングオブジェクトです。`Record<BookStatus, string>` は「キーが `BookStatus` 型、値が `string` 型のオブジェクト」を意味します。
`statusColors`	ステータスに対応する Tailwind CSS のクラス名をマッピングしたオブジェクトです。StatusBadge コンポーネントでバッジの背景色と文字色を動的に切り替えるために使います。

> **注意（著者は必須）**：`author` は `title` と同じく**必須**です。第5章のテーブルでも `author text NOT NULL`（空を許さない）になっています。登録フォームで著者が空のまま送ると、データベースに保存できません。「著者が分からない本」を登録したいときも、空にせず「不明」などの文字を入れる決まりにします。

> **第7章とのつながり:** 第7章では、説明の流れに合わせて `BookCard.tsx` の中に `Book` 型、`StatusBadge.tsx` の中に `BookStatus` 型を書きながら学びます。中身はこのファイルの型と同じ形なので、どちらから import しても同じように使えます。

Supabase の自動生成型（Database 型）について: 大規模プロジェクトでは Supabase CLI で `npx supabase gen types typescript` を実行し、データベースの定義からTypeScriptの型を自動生成することがあります（第5章でも紹介しました）。生成された型は次のような形をしています（抜粋）。

コピー

```
export type Database = {
  public: {
    Tables: {
      books: {
        // SELECTで返ってくる行の型
        Row: { id: string; title: string; author: string };
        // INSERT時に渡す型
        Insert: { title: string; author: string };
        // UPDATE時に渡す型
        Update: { title?: string; author?: string };
      };
    };
  };
};
```

本チュートリアルでは学習のしやすさのため、手書きで `Book`, `BookInsert`, `BookUpdate` を定義しています。仕組みは同じです。第5章で自動生成版の `types/book.ts` を作った場合は、この章のコードで上書きして構いません（型の名前は同じです）。

なぜ3つの型を分けるのか？

データベースとやり取りする場面によって、必要な項目が異なります。

- **取得（SELECT）**: すべての項目が返ってくる → `Book` 型
- **登録（INSERT）**: `id` や `created_at` は不要 → `BookInsert` 型
- **更新（UPDATE）**: 変更したい項目だけ送る → `BookUpdate` 型

型を分けることで、各場面で正確なデータ構造を強制でき、バグを防げます。

3つの型の違いを1枚の表にすると、次のようになります。伝票にたとえると、`Book` は「倉庫から届いた控え（全項目が埋まっている）」、`BookInsert` は「新規注文の用紙（番号や日付は倉庫側が書く）」、`BookUpdate` は「変更届（変えたい欄だけ書けばよい）」です。



図6-11: Book / BookInsert / BookUpdate に入る項目の比較

▼ コードを1つずつ分解して解説

上の型定義を、初心者がつまずきやすい部分ごとに**1つずつ**ていねいに見ていきましょう。

解説1: 決まった3つの文字列しか入らない「ユニオン型」

コピー

```
export type BookStatus = "reading" | "completed" | "want_to_read";
```

- `type 名前 = ...` は、よく使うデータの形に**自分で名前を付ける**構文（型エイリアス）です。
- `"reading" | "completed" | "want_to_read"` の `|`（縦棒）は「または」を意味し、**この3つの文字列のどれか**しか入れられない、という制約になります。
- たとえば `"finished"` のような違う文字列を入れようとすると、コードを動かす前に TypeScript がエラーで教えてくれます。タイプミスを防げるのが利点です（6.3 で実際に試します）。
- 先頭の `export` で、他のファイルからこの `BookStatus` を使えるようにしています。

> **用語:** 「ユニオン型」とは、複数の候補のうちの**いずれか**を表す型です。決まった選択肢しか取らない値（ステータスや種別など）を安全に扱うのに向いています。

解説2: 取得用の完全なデータ型 `Book`

コピー

```
export type Book = {
  id: string;
  title: string;
  author: string;
  publisher: string | null;
  published_date: string | null;
  rating: number | null;
  status: BookStatus;
  memo: string | null;
  created_at: string;
  updated_at: string;
};
```

（コメントを省いた抜粋です。）

- `type Book = { ... }` は、**オブジェクト（複数の項目を持つデータ）の形**を定義しています。データベースの books テーブルの1行ぶんに対応します。
- 各行は `項目名: 型` の形で、「`title` は文字列」「`rating` は数値」のように、項目ごとに入る値の種類を決めています。
- `status: BookStatus` のように、解説1で作った型を**部品として再利用**できます。
- `number | null` は「数値、または null（値なし）」という意味で、未評価のときに `null` を許すための書き方です。`publisher`（出版社）・`published_date`（出版日）・`memo`（メモ）も、入力しなくてよい項目なので `| null` が付いています。
- `published_date` はデータベースでは日付（`date` 型）ですが、JavaScript には `"2024-01-15"` のような**文字列**で届くので `string` にしています。

> **用語:** 「null」とは「値が存在しない」ことを表す特別な値です。`number | null` のように書くと、「数値が入るか、まだ何も入っていないか」の両方を表現できます。

解説3: 場面で型を分ける（Insert と Update）

コピー

```
export type BookInsert = {
  title: string;
  author: string;
  publisher?: string | null;
  published_date?: string | null;
  rating?: number | null;
  status?: BookStatus;
  memo?: string | null;
};
```

（コメントを省いた抜粋です。）

- `BookInsert` は**新規登録のときに送るデータ**の型です。`id` や `created_at` はデータベースが自動で付けるので、ここには含めません。
- 項目名の後ろの `?`（クエスチョン）は「**省略してもよい**（オプショナル）」という印です。`status?` は、書かなくてもエラーにならず、DB 側のデフォルト値（`"want_to_read"`）が使われます。
- `title` と `author` には `?` が付いていません。この2つは**必ず送る**項目です。
- 一方 `BookUpdate` は更新用で、**全項目に `?` が付いています**。変更したい項目だけを送れば済むようにするためです。
- このように場面ごとに必要な項目を分けることで、「登録時に id を送ってしまう」「著者を送り忘れる」といった間違いを型で防げます。

> **用語:** 「オプショナル（`?`）」とは、そのプロパティが**あってもなくてもよい**という指定です。必須項目には付けず、省略を許したい項目にだけ付けます。

解説4: ステータスごとの値をまとめる `Record` 型

コピー

```
export const statusLabels: Record<BookStatus, string> = {
  reading: "読書中",
  completed: "読了",
  want_to_read: "読みたい",
};
```

（コメントを省いた抜粋です。）

- これは型ではなく**実際の値（定数）**です。`"reading"` のような内部用の文字列を、`"読書中"` という画面表示用の日本語に変換するための対応表です。

- `Record<BookStatus, string>` は「**キーが `BookStatus`、値が文字列**のオブジェクト」という型で、3つのステータス全部を漏れなく書くことを強制してくれます。
- 使うときは `statusLabels["reading"]` のように書くと `"読書中"` が取り出せます。
- 同じ仕組みの `statusColors` では、値が文字列ではなく `{ bg, text }`（背景色クラスと文字色クラス）のオブジェクトになっています。色は第7章の StatusBadge と同じ（読書中は青、読了は緑、読みたいは黄）です。

> **用語:**「Record」は TypeScript の組み込み型で、キーの型 `K` と値の型 `V` を指定して**対応表（マップ）の形**を作ります。選択肢ごとの設定値をまとめるのに便利です。

6.2 サンプル: ステータスと日本語ラベルの対応を動かしてみる

> **▼ このコードがやること（先に日本語で）:** 6.1 の `BookStatus` 型と `statusLabels` の対応表だけを取り出した小さなプログラムです。3つのステータスを順番に取り出し、それぞれの日本語ラベルを表示します。Next.js を起動しなくても、Node.js だけで動きを確かめられます。

コピー

```
// scripts/status-demo.ts
// 読書ステータスの型と、日本語ラベルの対応表を小さく試すスクリプト

// 読書ステータスとして使ってよい3つの文字列だけを並べた型
type BookStatus = "reading" | "completed" | "want_to_read";

// ステータス → 日本語ラベルの対応表（3つすべて書かないと型エラーになる）
const statusLabels: Record<BookStatus, string> = {
  reading: "読書中",
  completed: "読了",
  want_to_read: "読みたい",
};

// 3つのステータスを順番に取り出してラベルを表示する
const all: BookStatus[] = ["reading", "completed", "want_to_read"];
for (const status of all) {
  console.log(`${status} → ${statusLabels[status]}`);
}
```

コピー

```
# scripts フォルダに保存して、Node.js でそのまま実行する
node scripts/status-demo.ts
```

▼ 実行結果

コピー

```
reading → 読書中
completed → 読了
want_to_read → 読みたい
```

▼ コードを1つずつ分解して解説

解説1: `BookStatus[]` は「BookStatus が並んだ配列」

コピー

```
const all: BookStatus[] = ["reading", "completed", "want_to_read"];
```

- 型の後ろに `[]` を付けると「その型の値がいくつも並んだ配列」という意味になります。
- 配列の中にも型のチェックが働くので、ここに `"finished"` を入れると型エラーになります。

解説2: `statusLabels[status]` で対応表を引く

コピー

```
for (const status of all) {
  console.log(`${status} → ${statusLabels[status]}`);
}
```

- `for (const status of all)` で、配列から1つずつ取り出して `status` に入れます。
- `statusLabels[status]` は、対応表から `status` というキーの値を取り出します。`status` が `"reading"` なら `"読書中"` です。
- `` `...${...}...` `` はテンプレートリテラルで、文字列の中に値を埋め込んでいます。

> **用語:**「対応表（マップ）」とは、キーを渡すと値が返ってくるデータの形です。辞書で単語（キー）から意味（値）を引くのと同じです。

6.3 つまずきポイント: 古いステータス名や打ち間違いを入れてしまう

> **つまずきポイント: `node` で動いたから型も正しい、と思い込む**
>
> 古い記事やメモを見て、存在しないステータス（たとえば `"finished"`）を書いてしまうことがあります。第2章で学んだとおり、`node ファイル名.ts` は**型を消して動かすだけ**なので、型の間違いがあってもエラーにならず、おかしな結果のまま動いてしまいます。型の間違いを見つけるのは `tsc`（TypeScript のチェック役）や VS Code の赤線です。

NG（候補に無い `"finished"` を入れている）：

コピー

```
// scripts/typo-demo.ts
// わざと型に合わない値を入れて、node と tsc の反応の違いを見るスクリプト

// 読書ステータスとして使ってよい3つの文字列
type BookStatus = "reading" | "completed" | "want_to_read";

// ステータス → 日本語ラベルの対応表
const statusLabels: Record<BookStatus, string> = {
  reading: "読書中",
  completed: "読了",
  want_to_read: "読みたい",
};

// わざと間違えた値（"finished" は BookStatus の候補に無い）
const myStatus: BookStatus = "finished";
console.log("ラベル:", statusLabels[myStatus]);
```

コピー

```
# まず node で実行する（型は見ない）
node scripts/typo-demo.ts
# 次に tsc で型だけをチェックする（--noEmit: JavaScript のファイルは作らない）
npx tsc --noEmit scripts/typo-demo.ts
```

▼ 実行結果

コピー

```
ラベル: undefined
scripts/typo-demo.ts(15,7): error TS2322: Type '"finished"' is not assignable to type 'BookStatus'.
```

1行目は `node` の結果です。エラーにはならず、ラベルが `undefined`（値なし）になっています。画面に「undefined」と出てしまう、気づきにくいバグです。2行目は `tsc` の結果で、「15行目7文字目: `"finished"` という型は `BookStatus` に入れられません」と、原因の場所まで教えてくれています。

OK（候補にある値を使う）：

コピー

```
// 読了は "completed"
const myStatus: BookStatus = "completed";
```

> **注意:** この `typo-demo.ts` は**試したら必ず削除**してください。`tsconfig.json` は `scripts/` の中の `.ts` も型チェックの対象にするので、間違いを残したままだと `npm run build`（型チェック付きの本番用ビルド）が止まります。

---

7. ルートレイアウトの設定

7.1 app/layout.tsx の作成

`app/layout.tsx` はすべてのページを囲む「外枠」のようなものです。HTML の `<html>` と `<body>` タグを含み、全ページ共通のヘッダーや CSS をここで設定します。

以下のコードで `app/layout.tsx` を書き換えます（`create-next-app` が生成したデフォルトの内容を完全に置き換えます）。

> **▼ このコードがやること（先に日本語で）:** すべてのページを包む「共通の外枠」を作ります。`<html>` や `<body>` タグ、日本語フォント、ブラウザのタブに出るタイトル、そして全ページ上部のヘッダーをここでまとめて設定します。各ページの中身は `{children}`（チルドレン：差し込み口）の場所にはめ込まれる、という仕組みがポイントです。フォントやレイアウトの細かい指定はコメントと解説表で1つずつ確認できます。

コピー

```
// app/layout.tsx
// 全ページに適用される「ルートレイアウト」。<html> と <body> はここでしか書かない

// type import: Metadata 型だけをインポート（実体はビルド後のコードに残らない）
import type { Metadata } from "next";

// named import: next/font/google から Noto_Sans_JP 関数だけを取り出す
// next/font は Next.js が用意した、フォント最適化のための機能
import { Noto_Sans_JP } from "next/font/google";

// CSS のサイドエフェクトインポート（左辺なし）
// このファイルを読み込んだだけで CSS が適用される
import "./globals.css";

// default import: Header コンポーネントを取り込む
// @/components/Header はプロジェクト直下の components/Header（パスエイリアス @/ = プロジェクト直下）
import Header from "@/components/Header";

/**
 * フォント設定
 *
 * Google Fonts から Noto Sans JP（日本語対応フォント）を読み込みます。
 * next/font/google を使うことで、フォントファイルがビルド時に
 * 自動的に最適化・セルフホスティングされます。
 * （外部の Google Fonts サーバーにリクエストが飛ばないため高速）
 */
const notoSansJP = Noto_Sans_JP({
  // 読み込むサブセット（文字種）。日本語はデフォルトで含まれる
  subsets: ["latin"],
  // 使用するフォントの太さ（通常 / やや太字 / 太字）
  weight: ["400", "500", "700"],
  // フォント読み込み中は代替フォントで表示し、完了後に切り替える
  display: "swap",
  // 重要なフォントなので、HTMLの <link rel="preload"> で先読みする
  preload: true,
});

/**
 * メタデータ設定
 *
 * ブラウザのタブに表示されるタイトルや、
 * 検索エンジンに表示される説明文を設定します。
 */
// export const metadata: Next.js が自動でこの変数を読み取って <head> に反映する
// 型は Metadata（先ほど type import したもの）
export const metadata: Metadata = {
  // <title> タグの中身（ブラウザのタブ表示）
  title: "書籍管理アプリ",
  // <meta name="description"> の中身（検索結果に出る説明文）
  description:
    "読んだ本、読んでいる本、これから読む本を管理するWebアプリケーション",
};

/**
 * ルートレイアウトコンポーネント
 *
 * すべてのページはこのレイアウトの中にレンダリングされます。
 * { children } には各ページのコンテンツが入ります。
 *
 * 構造:
 *   <html>
 *     <body>
 *       <Header />          ← 全ページ共通のヘッダー
 *       <main>{children}</main> ← 各ページの内容
 *     </body>
 *   </html>
 */
// export default function: ファイルの主要な関数として1つだけ default エクスポート
// RootLayout: コンポーネントの名前。Next.js が自動でこの関数を呼ぶ
// { children }: 分割代入で props から children を取り出している
// Readonly<{ children: React.ReactNode }>: 引数の型。children は読み取り専用、React の任意のノード
export default function RootLayout({
  children,
```

```
}: Readonly<{
  children: React.ReactNode;
}>) {
  // JSX を return する。1つの親要素（<html>）で全体を包む必要がある
  return (
    // lang="ja": HTML の言語属性を日本語に。スクリーンリーダーや検索エンジン用
    <html lang="ja">
      {/* body の className: notoSansJP のクラス名 + Tailwind の背景色と最小高さ */}
      {/* テンプレートリテラル `${...}` で文字列を結合 */}
      <body className={`${notoSansJP.className} bg-gray-50 min-h-screen`}>
        {/* 共通ヘッダーを最上部に配置 */}
        <Header />
        {/* main: ページのメインコンテンツ領域 */}
        {/* max-w-7xl: 最大幅1280px / mx-auto: 左右マージン自動で中央寄せ */}
        {/* px-4 sm:px-6 lg:px-8: 画面サイズに応じて左右パディングを変える（モバイル16px → タブレット24px → PC32px） */}
        {/* py-8: 上下パディング32px */}
        <main className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
          {/* children: 各ページの中身がここに差し込まれる */}
          {children}
        </main>
      </body>
    </html>
  );
}
```

コードの解説:

部分	解説
`import type { Metadata } from "next";`	Next.js のメタデータ型をインポートします。`type` キーワードは「型のみインポート」を意味し、ランタイムには影響しません（ビルド時に消える）。
`import { Noto_Sans_JP } from "next/font/google";`	Google Fonts から Noto Sans JP フォントをインポートします。`next/font` を使うと、フォントがビルド時にダウンロードされ、パフォーマンスが最適化されます。
`import "./globals.css";`	Tailwind CSS のグローバルスタイルを読み込みます。この1行がないと Tailwind のクラスが機能しません。`./` は同じフォルダにあるファイルを指す相対パス。
`import Header from "@/components/Header";`	共通ヘッダーコンポーネントをインポートします。`@/` はプロジェクト直下を指すエイリアスです。これにより `../components/Header` のように相対パスを書かなくて済みます。
`subsets: ["latin"]`	ラテン文字のサブセットを読み込みます。日本語文字はデフォルトで含まれます。
`weight: ["400", "500", "700"]`	使用するフォントウェイト（太さ）を指定します。400 = 通常、500 = やや太字、700 = 太字。指定しない太さは使えなくなる代わりに、ダウンロードサイズが減ります。
`display: "swap"`	フォントの読み込み方式を指定します。`"swap"` はフォント読み込み完了前にシステムフォントで表示し、読み込み後に切り替えます（FOIT: Flash Of Invisible Text を防ぐ）。
`export const metadata`	ページのメタデータを定義します。`title` がブラウザのタブに表示され、`description` が検索エンジンの説明文に使われます。Next.js がこの export を自動的に検出して `<head>` タグに反映します。
`lang="ja"`	HTML の言語を日本語に設定します。スクリーンリーダーや検索エンジンが正しく言語を認識できます。
`bg-gray-50`	`<body>` の背景色をごく薄いグレーにします。真っ白よりも目に優しく、カードなどの白い要素が際立ちます。
`min-h-screen`	`<body>` の最小高さを画面いっぱいにします。コンテンツが少ないページでも背景色が画面全体に適用されます。
`max-w-7xl mx-auto`	`<main>` の最大幅を 1280px に制限し、左右の margin を auto にして中央寄せにします。大画面でもコンテンツが横に広がりすぎるのを防ぎます。
`px-4 sm:px-6 lg:px-8`	画面サイズに応じて左右の padding を変えます。モバイルでは 16px、タブレットでは 24px、PCでは 32px です。
`{children}`	各ページのコンテンツがここに挿入されます。`/` にアクセスすれば `app/page.tsx` の内容が、`/books/new` にアクセスすれば `app/books/new/page.tsx` の内容が入ります。

▼ コードを1つずつ分解して解説

上のレイアウトを、初心者がつまずきやすい部分ごとに**1つずつ**ていねいに見ていきましょう。

解説1: 3種類のインポートを使い分ける

コピー

```
import type { Metadata } from "next";
import { Noto_Sans_JP } from "next/font/google";
import "./globals.css";
import Header from "@/components/Header";
```

- 1行目の `import type` は「**型だけ**を取り込む」インポートで、ビルド後のコードには残りません（型は実行時には不要なため）。
- 2行目の `{ Noto_Sans_JP }` は波括弧付きの「名前付きインポート」で、日本語フォントを読み込む関数を取り出しています。
- 3行目の `import "./globals.css";` は**左側に名前がない**特殊な形で、「このファイルを読み込むだけ」を意味します。これで Tailwind のスタイルが全ページに効きます。
- 4行目の `import Header` は波括弧なしの「デフォルトインポート」。`@/` はプロジェクト直下の近道（パスエイリアス）なので、`@/components/Header` は `components/Header` を指します。

> **用語:** 「パスエイリアス（ `@/` ）」とは、長い相対パス（ `../../components/...` ）を短く書くための別名です。プロジェクト直下を起点にできるので、ファイルを移動してもインポート文を直さずに済みます。

解説2: フォントを最適化して読み込む

コピー

```
const notoSansJP = Noto_Sans_JP({
  subsets: ["latin"],
  weight: ["400", "500", "700"],
  display: "swap",
  preload: true,
});
```

- `Noto_Sans_JP({ ... })` のように、フォント関数に**設定オブジェクト**を渡して呼び出します。戻り値を `notoSansJP` に保存し、あとで `<body>` に適用します。
- `weight: ["400", "500", "700"]` は使う太さ（通常・やや太字・太字）の指定で、使う分だけ読み込むことでファイルサイズを抑えます。
- `display: "swap"` は、フォント読み込み中はいったん代替フォントで表示し、読み込み完了後に切り替える設定です。文字が一瞬消える現象を防ぎます。
- `next/font` を使うとフォントがビルド時にダウンロードされて自前配信されるため、表示が速く安定します。

> **用語:**「フォントの最適化」とは、必要な文字種・太さだけをまとめてダウンロードし、表示の遅延やレイアウトのガタつきを減らす仕組みのことです。

解説3: メタデータでタブのタイトルなどを設定

コピー

```
export const metadata: Metadata = {
  title: "書籍管理アプリ",
  description:
    "読んだ本、読んでいる本、これから読む本を管理するWebアプリケーション",
};
```

- `metadata` という名前で `export` した値は、**Next.js が自動で見つけて** HTML の `<head>` に反映してくれます（自分で `<title>` を書く必要がありません）。
- `title` はブラウザのタブに表示される文字、`description` は検索結果に出る説明文になります。
- `: Metadata` は「この値は `Metadata` という型に従う」という指定で、解説1で `import type` したものです。項目名を間違えると型チェックで気づけます。

> **用語:**「メタデータ」とは、ページ自体の内容ではなく、ページに**付随する情報**（タイトル・説明・OGP など）のことです。検索エンジンや SNS が参照します。

解説4: children に各ページが差し込まれる仕組み

コピー

```
export default function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode;
}>) {
  return (
    <html lang="ja">
      <body className={`${notoSansJP.className} bg-gray-50 min-h-screen`}>
        <Header />
        <main className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
          {children}
        </main>
      </body>
    </html>
  );
}
```

- `RootLayout` は全ページを包む外枠コンポーネントです。引数の `{ children }` には、**表示中の各ページの中身**が自動的に入ってきます。
- `Readonly<{ children: React.ReactNode }>` は引数の型で、`React.ReactNode` は「React が表示できるあらゆる中身（要素・文字列など）」を表します。
- `className={` ${notoSansJP.className} ... `}` の `` `${...}` `` はテンプレートリテラルで、フォントのクラス名と Tailwind のクラスを**つなげて1つの文字列**にしています。
- `<Header />` を常に上に置き、その下の `<main>` 内の `{children}` 部分だけがページごとに入れ替わります。これで全ページ共通の枠を1か所で管理できます。

> **用語:** 「children（チルドレン）」とは、コンポーネントの中に**差し込まれる中身**を受け取る特別な props です。レイアウトのように「枠だけ用意して中身は後から入れる」設計でよく使います。

この「枠」と「中身」の関係を図にすると次のようになります。額縁（layout.tsx）は1つだけで、飾る絵（page.tsx）だけが URL によって掛け替えられる、とイメージしてください。

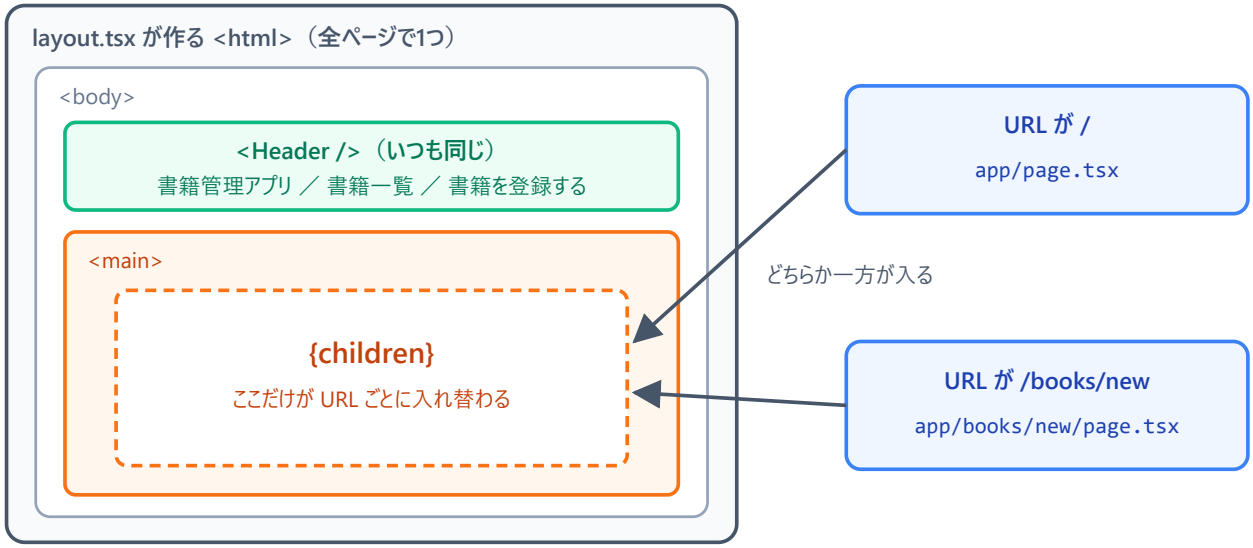


図6-12: layout.tsx（額縁）と page.tsx（絵）の関係

7.2 globals.css の確認

`app/globals.css` を開くと、`create-next-app` が作った次のような内容が入っています。

コピー

```
@import "tailwindcss";

:root {
  --background: #ffffff;
  --foreground: #171717;
}

@theme inline {
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  --font-sans: var(--font-geist-sans);
  --font-mono: var(--font-geist-mono);
}

@media (prefers-color-scheme: dark) {
  :root {
    --background: #0a0a0a;
    --foreground: #ededed;
  }
}

body {
  background: var(--background);
  color: var(--foreground);
  font-family: Arial, Helvetica, sans-serif;
}
```

先頭の `@import "tailwindcss";` 以外は、最初のサンプルページ用の設定（パソコンがダークモードのときに背景を黒くする、など）です。本書のアプリでは色やフォントを Tailwind のクラスと `layout.tsx` で指定するので、**ファイルの中身を次の1行（とコメント）だけに書き換えてください。**

コピー

```
/* app/globals.css */

/* Tailwind CSS v4 を読み込む。この1行で、flex や bg-white などのクラスがすべて使えるようになる */
@import "tailwindcss";
```

> **なぜ残りを消すのか**: 残しておくと、パソコンがダークモードのときだけ文字色が白っぽくなり、白いカードの上の文字が読みにくくなることがあります。「自分のパソコンでは普通なのに、友達のパソコンでは見た目が違う」という原因の分かりにくい違いを防ぐため、最初に消しておきます（ダークモード対応は第9章で扱います）。

> **用語**:「 `@import` 」は、CSS の中で別の CSS を読み込む命令です。`@import "tailwindcss";` は「Tailwind CSS の部品一式を読み込む」という意味で、Tailwind CSS v4 ではこの1行だけで準備が整います。

7.2.1 つまずきポイント: 古い書き方（`@tailwind base;`）を書いてしまう

> **つまずきポイント: Tailwind CSS v3 時代の3行を書く**
>
> 古い記事では、`globals.css` に `@tailwind base;`、`@tailwind components;`、`@tailwind utilities;` の3行を書くよう説明されています。これは **Tailwind CSS v3** の書き方です。現在の create-next-app が入れる **v4** でこの3行を書くと、エラーは出ないのに**色や文字の大きさのクラスが効かなくなります**。

NG（v3 の書き方）：

コピー

```
/* app/globals.css（v4 では NG） */
@tailwind base;
@tailwind components;
@tailwind utilities;
```

OK（v4 の書き方）：

コピー

```
/* app/globals.css */
@import "tailwindcss";
```

本書の環境で、それぞれの書き方で `npm run build` して、できあがった CSS ファイルを調べた結果は次のとおりです。

`globals.css` の書き方	できた CSS の大きさ	`.flex`	`.bg-white` / `.bg-blue-50` / `.text-3xl`
NG: `@tailwind base;` などの3行	1,236 バイト	ある	**無い**
OK: `@import "tailwindcss";`	12,332 バイト	ある	ある

NG の書き方では、`flex`（横並び）のような一部のクラスだけが作られ、色（`bg-white` など）や文字サイズ（`text-3xl` など）のクラスが1つも作られませんでした。v4 では色や大きさの「見本帳（テーマ）」を `@import "tailwindcss";` で読み込む仕組みに変わったため、古い3行ではその見本帳が読み込まれないからです。「レイアウトは崩れていないのに、色も文字の大きさも付かない」ときは、まず `globals.css` の1行目を確認してください。

> **Tailwind CSS v3 の書き方について（参考）**：v3 では、`@tailwind base;`（リセット CSS と基本スタイル）、`@tailwind components;`（コンポーネント用クラス）、`@tailwind utilities;`（`flex` や `pt-4` などのユーティリティクラス）の3行と、`tailwind.config.ts`（設定ファイル）を組み合わせて使っていました。古い記事や、v3 で作られた既存のプロジェクトを読むときのために、「昔はこう書いていた」と知っておけば十分です。

8. 共通ヘッダーコンポーネント

8.1 components/Header.tsx の作成

`components/Header.tsx` ファイルに以下のコードを記述します。

> **▼ このコードがやること（先に日本語で）**：全ページの上部に出る「ナビゲーションバー（共通ヘッダー）」を作ります。アプリ名のロゴと、「書籍一覧」「書籍を登録する」へのリンクを並べ、いま開いているページのリンクだけ色を変えて目立たせます。`usePathname()` という機能で現在のURLを取得するため、先頭に `"use client"` と書いてブラウザ側で動くコンポーネントにしている点がポイントです。スタイル（色や余白）の意味は各行のコメントを参照してください。

コピー

```
// components/Header.tsx
// 全ページ上部に表示する共通ヘッダー（ナビゲーションバー）

// "use client": このファイルを Client Component として宣言する特別な文字列
// 必ずファイルの一番上（importよりも前）に書く
// これがあるとブラウザで実行され、React フックや状態管理が使える
"use client";

// default import: Next.js の Link コンポーネント
// HTML の <a> よりも高速にページ遷移できる（SPA的な切り替え）
import Link from "next/link";

// named import: usePathname フック（現在のURLパスを取得）
// 注意: import元は "next/navigation"（App Router用）。Pages Router の "next/router" とは別物
import { usePathname } from "next/navigation";

/**
 * 共通ヘッダーコンポーネント
 *
 * すべてのページの上部に表示されるナビゲーションバーです。
 * - アプリ名（クリックでトップページに遷移）
 * - 「書籍を登録する」ボタン
 * を含みます。
 *
 * "use client" を使用する理由:
 * usePathname() フックを使用するため、Client Component にする必要があります。
 * Server Component では React のフック（useState, useEffect, usePathname 等）は使えません。
 */
// export default: このファイルを import するときの主要な対象
// function Header(): 関数コンポーネントの宣言。引数なし、JSX を返す
export default function Header() {
  // 現在のURLパスを取得（アクティブなナビゲーションリンクのスタイル変更に使用）
  // usePathname() の戻り値は "/" や "/books/new" のような文字列
  const pathname = usePathname();

  // JSX を return（ヘッダー全体の構造）
  return (
    // <header>: HTML5 のセマンティックタグ。ページ上部のヘッダーを示す
    // bg-white: 白い背景 / shadow-sm: 控えめな影 / border-b: 下方向に1pxのボーダー / border-gray-200: ボーダー色
    <header className="bg-white shadow-sm border-b border-gray-200">
      {/* 内側の div: ヘッダーの中身を中央寄せ＆最大幅制限する */}
      {/* max-w-7xl: 最大幅1280px / mx-auto: 左右マージンautoで中央寄せ */}
      {/* px-4 sm:px-6 lg:px-8: 画面サイズに応じて左右パディング */}
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
        {/* flex 行: ロゴと右側ナビを横並びに、両端配置、高さ64px */}
        {/* flex: フレックスボックス / items-center: 垂直中央揃え */}
        {/* justify-between: 左右に均等配置（ロゴが左、ナビが右） / h-16: 高さ64px */}
        <div className="flex items-center justify-between h-16">
          {/* アプリ名（ロゴ） */}
          {/* Link: ページリロードなしで遷移する Next.js のコンポーネント */}
          {/* href: 遷移先のパス */}
          <Link
            href="/"
            // text-xl: フォントサイズ20px / font-bold: 太字 / text-gray-900: 濃いグレー
            // hover:text-blue-600: ホバー時に青 / transition duration-200: 200msで滑らかに変化
            className="text-xl font-bold text-gray-900 hover:text-blue-600 transition duration-200"
          >
            書籍管理アプリ
          </Link>

          {/* ナビゲーション */}
          {/* nav: HTML5 のナビゲーション用セマンティックタグ */}
          {/* space-x-4: 子要素間に16pxの水平方向マージン */}
          <nav className="flex items-center space-x-4">
```

```
            {/* 書籍一覧リンク */}
            <Link
              href="/"
              // テンプレートリテラル ${} で動的にクラスを切り替える
              // pathname === "/" が true なら青系（現在地ハイライト）、false ならグレー系
              className={`px-3 py-2 rounded-md text-sm font-medium transition duration-200 ${
                pathname === "/"
                  // 現在地：薄青背景＋青文字
                  ? "bg-blue-50 text-blue-700"
                  // 通常：グレー文字、ホバーで反応
                  : "text-gray-600 hover:text-gray-900 hover:bg-gray-50"
              }`}
            >
              書籍一覧
            </Link>

            {/* 書籍登録ボタン */}
            <Link
              href="/books/new"
              // px-4: 左右パディング16px / py-2: 上下パディング8px / rounded-md: 角丸
              // text-sm: 文字サイズ14px / font-medium: やや太字 / transition: 滑らか変化
              className={`px-4 py-2 rounded-md text-sm font-medium transition duration-200 ${
                pathname === "/books/new"
                  // 現在地：濃い青背景＋白文字
                  ? "bg-blue-700 text-white"
                  // 通常：青背景、ホバーで濃く
                  : "bg-blue-600 text-white hover:bg-blue-700"
              }`}
            >
              書籍を登録する
            </Link>
          </nav>
        </div>
      </div>
    </header>
  );
}
```

コードの解説:

部分	解説
`"use client";`	このファイルを Client Component として宣言します。Next.js の App Router では、デフォルトですべてのコンポーネントが Server Component（サーバーで実行）です。しかし `usePathname()` などの React フックを使うには Client Component にする必要があります。ファイルの先頭にこの宣言を書くだけで切り替えられます。
`import Link from "next/link";`	Next.js のリンクコンポーネントです。HTML の `<a>` タグの代わりに使います。ページ遷移時にページ全体をリロードせず、必要な部分だけを更新するため高速です。
`import { usePathname } from "next/navigation";`	現在のURLパスを取得するフックです。例えば `/books/new` にいるときは `"/books/new"` が返ります。
`pathname === "/"`	現在のページがトップページかどうかを判定し、条件に応じてスタイルを切り替えます。現在いるページのリンクには背景色を付けてハイライト表示します。
`bg-white shadow-sm border-b border-gray-200`	ヘッダーの背景を白にし、下に薄いシャドウとボーダーを付けてコンテンツ部分との区切りを明確にします。
`max-w-7xl mx-auto`	ヘッダー内のコンテンツの最大幅を 1280px にし、中央寄せにします。`layout.tsx` の `<main>` と同じ幅にすることで整った見た目になります。
`flex items-center justify-between h-16`	Flexbox で子要素を横並びにし、垂直方向に中央揃え、左右に均等配置、高さ64px を設定します。
`space-x-4`	ナビゲーションリンク間に 16px の水平方向の間隔を設けます。

▼ コードを1つずつ分解して解説

上のヘッダーを、初心者がつまずきやすい部分ごとに**1つずつ**ていねいに見ていきましょう。

解説1: なぜ先頭に "use client" が必要か

コピー

```
"use client";
```

- この1行は、ファイルを「**Client Component（ブラウザ側で動く部品）**」として宣言する特別な合図です。必ず import より前、ファイルの一番上に書きます。
- Next.js の App Router では、何も書かないとコンポーネントは**サーバー側で動く**設定になります。サーバー側では `useState` などの React の機能や、URL を取る `usePathname()` が使えません。
- このヘッダーは現在地に応じて見た目を変えるため `usePathname()` を使います。そのために `"use client"` を付けてブラウザ側で動くようにしています。

> **用語:** 「Client Component」とは、ユーザーのブラウザ上で実行されるコンポーネントです。クリックや入力などの操作、React フックを使いたい部品はこちらにします。

解説2: Link と usePathname を取り込む

コピー

```
import Link from "next/link";
import { usePathname } from "next/navigation";
```

- `Link` は、ページを**まるごと再読み込みせずに**素早く切り替えるための Next.js 専用のリンク部品です。HTML の `<a>` の代わりに使います。
- `usePathname` は「**いま開いている URL のパス**」（例: `"/"` や `"/books/new"`）を取得するフックです。
- import 元に注意が必要で、`usePathname` は App Router 用の `"next/navigation"` から取り込みます（旧方式の `"next/router"` とは別物です）。

> **用語:** 「フック（Hook）」とは、`use` で始まる React の特別な関数です。状態や現在の URL など、コンポーネントに機能を「引っかけて」追加するために使います。

解説3: 現在のパスを取得する

コピー

```
const pathname = usePathname();
```

- `usePathname()` を呼ぶと、今表示しているページの URL パスが文字列で返ってきます。それを `pathname` に保存します。
- この値を後で `pathname === "/"` のように比較して、「いま見ているページのリンクだけ色を変える」ハイライト表示に使います。
- ページを移動すると `pathname` の値も自動で変わり、ヘッダーの見た目も更新されます。

> **用語:**「パス（pathname）」とは、URL のうちドメイン名より後ろの部分（ `/books/new` など）のことです。どのページにいるかを判別する手がかりになります。

解説4: 現在地によってクラスを切り替える

コピー

```
<Link
  href="/"
  className={`px-3 py-2 rounded-md text-sm font-medium transition duration-200 ${
    pathname === "/"
      ? "bg-blue-50 text-blue-700"
      : "text-gray-600 hover:text-gray-900 hover:bg-gray-50"
  }`}
>
  書籍一覧
</Link>
```

- `className={` ...${ ... } `}` のテンプレートリテラルで、**共通のクラス**と**条件で変わるクラス**を1つの文字列に合体させています。
- `条件 ? Aの値 : Bの値` は三項演算子（条件分岐の短い書き方）で、「`pathname === "/"` が真なら A、偽なら B」を選びます。
- いまトップページにいれば薄い青背景＋青文字（`bg-blue-50 text-blue-700`）でハイライトし、そうでなければグレー文字でホバー時に反応するスタイルになります。
- `href="/"` で遷移先を指定し、クリックすると `Link` が高速にトップページへ切り替えます。

> **用語:**「三項演算子（ `条件 ? A : B` ）」とは、`if/else` を1行で書ける短縮形です。値を2択で切り替えたいとき、JSX の中などで手軽に使えます。

8.2 つまずきポイント: `"use client"` を書き忘れる

> **つまずきポイント: `usePathname` を使うのに `"use client"` が無い**
>
> `"use client"` は1行だけの地味な文字列なので、書き忘れたり、コピーのときに抜けたりしがちです。また、`import` 文より下に書いてしまうと、宣言として認められません。

NG（ `"use client"` が無い）:

コピー

```
// components/Header.tsx（先頭部分）
// "use client" が無いので、このファイルは Server Component として扱われる
import Link from "next/link";
import { usePathname } from "next/navigation";
```

この状態で `npm run build` すると、次のエラーで止まります（開発サーバーでも、ブラウザに同じ内容のエラー画面が出ます）。

▼ **実行結果**（`npm run build` の出力から抜粋）

コピー

```
> Build error occurred
Error: Turbopack build failed with 1 error:
./components/Header.tsx:15:10
Error: You're importing a module that depends on `usePathname` into a React Server Component module. This
API is only available in Client Components. To fix, mark the file (or its parent) with the `"use client"`
directive.
    Learn more: https://nextjs.org/docs/app/api-reference/directives/use-client
  13 | // named import: usePathname フック（現在のURLパスを取得）
  14 | // 注意: import元は "next/navigation"（App Router用）。Pages Router の "next/router" とは別物
> 15 | import { usePathname } from "next/navigation";
     |          ^^^^^^^^^^^
  16 |
  17 | /**
  18 |  * 共通ヘッダーコンポーネント

Ecmascript file had an error

Import trace:
  Server Component:
    ./components/Header.tsx
    ./app/layout.tsx
```

英文は「`usePathname` に頼るモジュールを、React Server Component に import しています。この機能は Client Component でしか使えません。直すには、ファイル（またはその親）に `"use client"` を付けてください」という意味です。最後の `Import trace`（読み込みの道すじ）からは、`layout.tsx`（Server Component）が `Header.tsx` を読み込み、その `Header.tsx` も Server Component のまま `usePathname` を使っている、という流れが読み取れます。

OK（ファイルの一番上、import より前に書く）：

コピー

```
// components/Header.tsx（先頭部分）
// "use client" は必ずファイルの一番上（import より前）に書く
"use client";

import Link from "next/link";
import { usePathname } from "next/navigation";
```

> **ヒント:** エラーの1行目に出るファイル名と行番号（ `./components/Header.tsx:15:10` は「15行目の10文字目」）を見れば、どこを直せばよいかが分かります。エラーは怖がらずに、まず「どのファイルの何行目か」だけでも読む習慣をつけましょう。

---

9. 開発サーバーの起動と動作確認

9.1 仮のトップページを作成

動作確認のため、`app/page.tsx` を以下の内容に書き換えます（ `create-next-app` のデフォルト内容を置き換えます）。

> **▼ このコードがやること（先に日本語で）:** トップページ（URL が `/` のページ）に、動作確認用の仮の画面を表示します。見出しと説明文、白いカード、3色のステータスバッジを並べて、Tailwind CSS のスタイルがちゃんと効いているかを目で確かめられるようにしています。中身はまだ仮で、本物の書籍一覧は次の章で作る、という位置づけです。クラス名（ `bg-white` など）の意味は各行のコメントにあります。

コピー

```
// app/page.tsx
// URL "/" にアクセスしたときに表示されるトップページ

// export default function: ファイルのメインコンポーネントとしてエクスポート
// HomePage: コンポーネント名（任意の名前でOK。Next.jsはdefault exportを自動的にページとして扱う）
export default function HomePage() {
  // JSX を return（このページの中身）
  return (
    // 全体を div で囲む。複数の要素を return するには親要素が必要
    <div>
      {/* h1: 大見出し / text-3xl: 30px / font-bold: 太字 / text-gray-900: 濃いグレー / mb-4: 下マージン16px */}
      <h1 className="text-3xl font-bold text-gray-900 mb-4">
        書籍一覧
      </h1>
      {/* p: 段落 / text-gray-600: 中間グレー / mb-8: 下マージン32px */}
      <p className="text-gray-600 mb-8">
        登録された書籍がここに表示されます。
      </p>

      {/* 動作確認用: Tailwind CSS のスタイルが正しく適用されるかチェック */}
      {/* bg-white: 白背景 / rounded-lg: 角丸大 / shadow-sm: 控えめな影 */}
      {/* border border-gray-200: 1pxの薄いグレーボーダー / p-6: 内側パディング24px */}
      <div className="bg-white rounded-lg shadow-sm border border-gray-200 p-6">
        {/* h2: 中見出し / text-lg: 18px / font-semibold: やや太字（600） */}
        <h2 className="text-lg font-semibold text-gray-800 mb-2">
          セットアップ完了
        </h2>
        {/* text-sm: 14px の小さめテキスト */}
        <p className="text-gray-600 text-sm">
          Next.js + Tailwind CSS + Supabase の開発環境が正しくセットアップされました。
          次の章で、このページに実際の書籍一覧を表示します。
        </p>
        {/* バッジを横並びにする領域 */}
        {/* mt-4: 上マージン16px / flex: 横並び / space-x-2: 子要素間に8px間隔 */}
        <div className="mt-4 flex space-x-2">
          {/* 読みたいバッジ：黄系 */}
          {/* inline-block: インラインブロック表示 / bg-yellow-100: 薄い黄の背景 / text-yellow-800: 濃い黄の文字 */}
          {/* text-xs: 12px / px-2.5 py-1: パディング / rounded-full: 完全な丸（カプセル形） / font-medium: 中太字 */}
          <span className="inline-block bg-yellow-100 text-yellow-800 text-xs px-2.5 py-1 rounded-full font-medium">
            読みたい
          </span>
          {/* 読書中バッジ：青系 */}
          <span className="inline-block bg-blue-100 text-blue-800 text-xs px-2.5 py-1 rounded-full font-medium">
            読書中
          </span>
          {/* 読了バッジ：緑系 */}
          <span className="inline-block bg-green-100 text-green-800 text-xs px-2.5 py-1 rounded-full font-medium">
            読了
          </span>
        </div>
      </div>
    </div>
  );
}
```

▼ コードを1つずつ分解して解説

上の仮トップページを、初心者がつまずきやすい部分ごとに**1つずつ**ていねいに見ていきましょう。

解説1: ページは default export の関数で作る

コピー

```
export default function HomePage() {
  return (
    <div>
      ...
    </div>
  );
}
```

- `app/page.tsx` の中で `export default` した関数が、その URL（ここでは `/`）に表示される**ページの本体**になります。Next.js が自動でこの関数を探して呼び出します。
- 関数名（`HomePage`）は自由に決められます。重要なのは名前ではなく「`export default` であること」です。
- `return ( ... )` の中に画面の中身（JSX）を書きます。複数の要素を返すときは、必ず1つの親要素（ここでは外側の `<div>`）で包む必要があります。

> **用語:** 「default export」とは、1ファイルにつき1つだけ指定できる「主役の書き出し」です。Next.js のページやコンポーネントは、これでファイルの代表を決めます。

解説2: Tailwind のクラスで見出しと説明を装飾

コピー

```
<h1 className="text-3xl font-bold text-gray-900 mb-4">
  書籍一覧
</h1>
<p className="text-gray-600 mb-8">
  登録された書籍がここに表示されます。
</p>
```

- JSX では HTML の `class` 属性のかわりに `className` を使います（`class` は JavaScript の予約語のため）。
- `className` に Tailwind のクラスを**スペース区切りで並べる**と、それぞれが1つのスタイルになります。`text-3xl`（文字サイズ30px）、`font-bold`（太字）、`text-gray-900`（濃いグレー）など。
- `mb-4` / `mb-8` は下方向の余白（margin-bottom）で、数字が大きいほど余白も広がります（4=16px、8=32px）。

> **用語:** 「className」とは、JSX で CSS クラスを指定するための属性名です。HTML の `class` と同じ役割ですが、React では `className` と書く決まりになっています。

解説3: カードとステータスバッジを並べる

コピー

```
<div className="bg-white rounded-lg shadow-sm border border-gray-200 p-6">
  <h2 className="text-lg font-semibold text-gray-800 mb-2">
    セットアップ完了
  </h2>
  ...
  <div className="mt-4 flex space-x-2">
    <span className="inline-block bg-yellow-100 text-yellow-800 text-xs px-2.5 py-1 rounded-full font-medium">
      読みたい
    </span>
    ...
  </div>
</div>
```

- 外側の `<div>` が**白いカード**です。`bg-white`（白背景）、`rounded-lg`（角丸）、`shadow-sm`（薄い影）、`border`（枠線）、`p-6`（内側の余白24px）を組み合わせて作っています。
- バッジを囲む `<div>` の `flex space-x-2` で、3つのバッジを**横並び**にし、間に8pxの隙間を空けています。
- 各バッジ（`<span>`）の `rounded-full` でカプセル型の丸みを付け、`bg-yellow-100` / `bg-blue-100` / `bg-green-100` のように背景色を変えてステータスを色分けしています（6.1 の `statusColors` と同じ色です）。
- これは動作確認用の仮表示で、Tailwind が正しく効いているか（色・角丸・影が出るか）を目で確かめる役割です。

> **用語:**「Flexbox（`flex`）」とは、子要素を横や縦に整列させるための CSS のレイアウト方式です。`space-x-2` のような間隔指定と組み合わせて、並びを手軽に整えられます。

9.2 開発サーバーの起動

ターミナルで、`book-management` **フォルダの中にいることを確かめてから**以下のコマンドを実行します。

コピー

```
# npm run dev: package.json の "scripts" にある "dev" を実行
# 中身は「next dev」で、Next.js の開発サーバーが起動する（Turbopack が標準で使われる）
# ファイルを保存するたびに自動でブラウザが更新される（ホットリロード）
# 停止するには Ctrl+C を押す
npm run dev
```

以下のような出力が表示されます（2026年10月に実際に実行したときの表示。`Network` の行の数字はパソコンごとに違います）。

コピー

```
> book-management@0.1.0 dev
> next dev

▲ Next.js 16.4.1 (Turbopack)
- Local:         http://localhost:3000
- Network:       http://192.168.x.x:3000
- Environments: .env.local
✓ Ready in 7.0s
✓ Running next.config.ts took 80ms

○ Compiling / ...
 GET / 200 in 398ms (next.js: 201ms, application-code: 197ms)
```

最後の2行は、ブラウザで `http://localhost:3000` を開いたときに追加で出る行です。各行の意味を画面イメージに書き込むと、次のようになります。

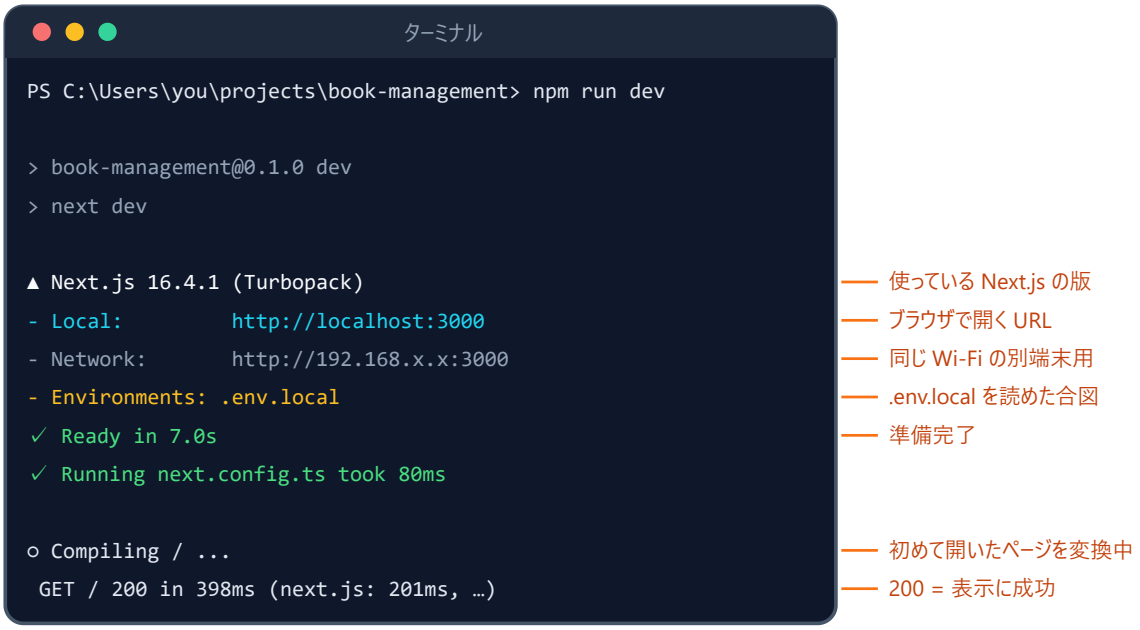


図6-13: npm run dev を実行したときのターミナルの画面イメージ（右は各行の意味）

> **ヒント:** 開発サーバーは、**ターミナルを開いている間ずっと動き続ける**プログラムです。このターミナルでは他のコマンドを打てなくなるので、別のコマンド（ `npm install` など）を実行したいときは、VS Code のターミナルの「＋」で**新しいターミナルを開いて**そちらで実行してください。止めるときは、開発サーバーが動いているターミナルで `Ctrl+C` を押します。

9.3 ブラウザでの確認

ブラウザで `http://localhost:3000` を開くと、以下のように表示されます。



図6-14: http://localhost:3000 を開いたときの画面イメージ（実際の表示を元に作図）

ヘッダー部分:

- 画面最上部に白い背景のヘッダーバーが表示されます
- 左側に「書籍管理アプリ」というテキストが太字で表示されます
- 右側に「書籍一覧」というテキストリンクと「書籍を登録する」という青いボタンが表示されます
- 「書籍一覧」リンクは現在のページなので、薄い青の背景でハイライトされています

メインコンテンツ部分:

- ヘッダーの下に薄いグレー（`bg-gray-50`）の背景があります
- 「書籍一覧」という大きな見出し（30px, 太字）が表示されます
- その下に「登録された書籍がここに表示されます。」という説明文があります
- 白い角丸のカード内に「セットアップ完了」というメッセージが表示されます
- カードの下部に3つのステータスバッジ（「読みたい」は黄、「読書中」は青、「読了」は緑）が横並びで表示されます

確認ポイント:

確認項目	期待される状態
ヘッダーが表示される	白い背景のバーが画面上部にある
Tailwind CSS が適用されている	テキストに色やサイズが適用され、カードに角丸やシャドウがある
フォント（Noto Sans JP）が適用されている	日本語テキストが美しいフォントで表示される
レスポンシブ対応	ブラウザの幅を狭めても、レイアウトが崩れない
ナビゲーションリンクが動作する	「書籍を登録する」ボタンをクリックすると `/books/new` に遷移する（まだページを作成していないため 404 が表示されますが、URL が変わることを確認）

9.4 よくあるトラブルと対処法

症状	原因	対処法
`Module not found: Can't resolve '@/components/Header'`	ファイルが存在しない、またはパスが間違っている	`components/Header.tsx` が正しい場所に作成されているか確認してください。ファイル名の大文字小文字も一致している必要があります。`@/src/...` と書いていないかも確認します（4.4.1）。
スタイルが全く適用されない	`globals.css` が正しく読み込まれていない	`app/layout.tsx` で `import "./globals.css";` が記述されているか、`globals.css` の1行目が `@import "tailwindcss";` か確認してください（7.2.1）。
`環境変数 NEXT_PUBLIC_SUPABASE_URL / NEXT_PUBLIC_SUPABASE_ANON_KEY が設定されていません`	`.env.local` が未作成、名前・場所が違う、変数名が違う	`.env.local` がプロジェクトのルートディレクトリ（`package.json` と同じ階層）にあり、起動時に `- Environments: .env.local` が出ているか確認してください。保存し直すと `Reload env` で読み直されますが、迷ったら `Ctrl+C` で止めて `npm run dev` で起動し直すのが確実です（5.2.2〜5.2.4）。
`⚠ Port 3000 is in use ... using available port 3001 instead.`	3000 番を別のアプリが使っている	Next.js 16 は空いている番号（3001 など）に**自動で切り替えて**起動します。表示された `Local:` の URL（`http://localhost:3001` など）をブラウザで開いてください。
`× Another next dev server is already running.`	同じフォルダで開発サーバーを2つ起動しようとした	すでに動いている方を使います（9.4.1）。
`npm error enoent Could not read package.json`	プロジェクトのフォルダの外で実行している	`cd book-management` でフォルダに入ってから実行します（9.4.2）。
日本語フォントが適用されていない	フォントの読み込みに失敗している	ネットワーク接続を確認してください。初回の起動・ビルド時に Google Fonts からフォントがダウンロードされます。

9.4.1 つまずきポイント: 開発サーバーを2つ起動してしまう

> **つまずきポイント:「動いていないのかな？」と、別のターミナルでもう一度 `npm run dev` する**
>
> 開発サーバーは、起動したターミナルの中でずっと動き続けます。別のターミナルを開いて、同じフォルダでもう一度 `npm run dev` を実行すると、Next.js 16 はそれを見つけて止まります。

NG（同じフォルダで2つ目を起動する）：

コピー

```
# 1つ目がまだ動いているのに、別のターミナルでもう一度起動する
npm run dev
```

▼ **実行結果**（パスとプロセス番号は例）

コピー

```
> book-management@0.1.0 dev
> next dev

⚠ Port 3000 is in use by process 9076, using available port 3001 instead.
▲ Next.js 16.4.1 (Turbopack)
- Local:         http://localhost:3001
- Network:       http://192.168.x.x:3001
- Environments: .env.local
✓ Ready in 832ms
✓ Running next.config.ts took 42ms
× Another next dev server is already running.

- Local:        http://localhost:3000
- PID:          9076
- Dir:          C:\Users\you\projects\book-management
- Log:          .next\dev\logs\next-development.log

You can access the existing server at http://localhost:3000,
or run taskkill /PID 9076 /F to stop it and start a new one.
```

まず「3000 番は使われているので 3001 番にします」と言いかけますが、すぐに「**同じフォルダの開発サーバーがもう動いています**（Another next dev server is already running）」と気づいて止まります。最後の2行は「すでに動いているサーバーに `http://localhost:3000` でアクセスできます。止めて新しく起動したいなら `taskkill /PID 9076 /F` を実行してください」という意味です（`taskkill` は Windows でプログラムを止めるコマンド、PID はプログラムの番号です）。

OK（すでに動いている方を使う）：

コピー

```
1. VS Code のターミナルの一覧から、npm run dev が動いているターミナルを探す
2. ブラウザで http://localhost:3000 を開く（新しく起動しなくてよい）
3. 起動し直したいときは、そのターミナルで Ctrl+C を押してから npm run dev
```

> **ヒント:** どのターミナルで動いているか分からなくなったときは、VS Code のターミナル右上のゴミ箱アイコンで**すべてのターミナルを閉じる**と、中で動いていた開発サーバーも止まります。そのあと新しいターミナルで `npm run dev` すれば、確実に1つだけになります。

9.4.2 つまずきポイント: プロジェクトのフォルダの外で実行してしまう

> **つまずきポイント: `cd book-management` を忘れる**
>
> VS Code で親のフォルダ（ `projects` など）を開いていると、ターミナルも親のフォルダから始まります。そのまま `npm run dev` すると、npm は `package.json` を見つけられません。

NG（ `book-management` の1つ外で実行する）：

コピー

```
# いま C:\Users\you\projects にいる（book-management の外）
npm run dev
```

▼ 実行結果（パスは例）

コピー

```
npm error code ENOENT
npm error syscall open
npm error path C:\Users\you\projects\package.json
npm error errno -4058
npm error enoent Could not read package.json: Error: ENOENT: no such file or directory, open 'C:\Users\you\projects\package.json'
npm error enoent This is related to npm not being able to find a file.
npm error enoent
npm error A complete log of this run can be found in: C:\Users\you\AppData\Local\npm-cache\_logs\2026-10-15T11_50_40_324Z-debug-0.log
```

`ENOENT` は「Error NO ENTry（そんなファイルはありません）」の略です。`path` の行を見ると、`projects\package.json` を探しに行って見つからなかったことが分かります。

OK（プロジェクトのフォルダに入ってから実行する）：

コピー

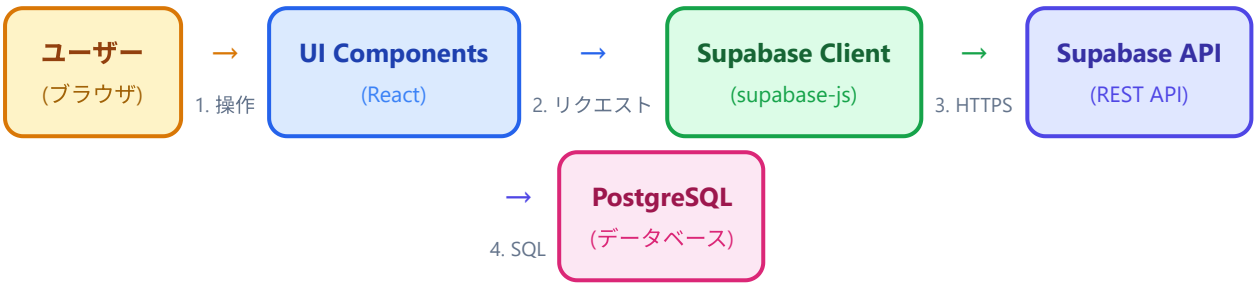
```
# book-management フォルダに入る
cd book-management
# package.json がある場所で実行する
npm run dev
```

> **ヒント:** いまどのフォルダにいるかは、PowerShell なら行の先頭（ `PS C:\Users\you\projects\book-management>` ）に表示されています。macOS / Linux では `pwd` コマンドで確認できます。`npm` のコマンドは、いつも **`package.json` がある場所**で実行する、と覚えておきましょう。

10. データフローの全体像

書籍管理アプリのデータの流れを図で確認しましょう。ユーザーの操作がどのようにデータベースに伝わり、結果がどのように画面に戻ってくるのかを理解することは、アプリ全体の設計を把握する上で非常に重要です。

10.1 全体アーキテクチャ



戻りの流れ: 5. クエリ結果 ← 6. JSON レスポンス ← 7. 型付きデータ (Book[]) ← 8. 画面更新 (再レンダリング)

10.2 各ステップの詳細解説

ステップ	場所	処理内容
1. ユーザーの操作	ブラウザ	ユーザーがボタンをクリックしたり、フォームに入力したりします。例:「書籍を登録する」ボタンを押す。
2. コンポーネントからの呼び出し	UI Components	React コンポーネント内で Supabase クライアントの関数を呼び出します。例: `supabase.from("books").insert(newBook)`
3. HTTP リクエスト	Supabase Client	`@supabase/supabase-js` ライブラリが、JavaScript のコードを HTTPS リクエストに変換して Supabase のサーバーに送信します。URL や Publishable key は `.env.local` の環境変数から読み取られます。
4. SQL クエリの実行	Supabase API	Supabase の REST API（PostgREST）がリクエストを受け取り、対応する SQL クエリに変換して PostgreSQL データベースに実行します。RLS（行レベルセキュリティ）によるアクセス制御もこのタイミングで適用されます。
5. クエリ結果	PostgreSQL	データベースがクエリを実行し、結果を返します。
6. JSON レスポンス	Supabase API	クエリ結果を JSON 形式に変換し、クライアントにレスポンスとして返します。
7. 型付きデータ	Supabase Client	JSON レスポンスを受け取り、TypeScript の型（`Book[]` など）に変換します。
8. 画面更新	UI Components → ブラウザ	受け取ったデータで React の state を更新し、画面が再レンダリングされます。ユーザーは最新のデータを画面上で確認できます。

10.3 具体例: 書籍一覧の取得

実際のコードでデータがどう流れるか、具体例で見てみましょう（第7章では、これと同じ処理をトップページ `app/page.tsx` の中に書きます）。

> **▼ このコードがやること（先に日本語で）**：データベースから書籍の一覧を取り出す処理を、具体的なコードで追いかけます。さきほど作ったサーバー用の Supabase クライアントを使い、「books テーブルから全件を、新しい順に取得する」という指示を出しています。`await` （待つ）は通信が終わるまで結果を待つための合図で、戻り値を `data` （成功データ）と `error` （エラー）に分けて受け取り、必ずエラーの有無を確認するのが安全な書き方です。この1つの呼び出しの裏で、前の図の通信ステップが自動で実行されています。

コピー

```
// 書籍一覧を取得する関数（第7章のトップページで使う処理の骨組み）

// 1. サーバー用の Supabase クライアントを作る関数をインポート
import { createClient } from "@/lib/supabase/server";
// Book 型もインポート（型情報のみ。ランタイムには影響なし）
import type { Book } from "@/types/book";

// async: 中で await を使う関数に付ける印。戻り値は「あとで Book の配列が届く約束（Promise）」
export async function getBooks(): Promise<Book[]> {
  // 2. サーバー用の窓口を作る（中で cookies() を待つので await が必要）
  const supabase = await createClient();

  // 3. データベースから書籍一覧を取得
  // 戻り値は { data, error } のオブジェクトなので、分割代入で取り出す
  const { data, error } = await supabase
    // books テーブルを指定（SQL の FROM books に相当）
    .from("books")
    // すべての列を取得（SQL の SELECT * に相当）
    .select("*")
    // 作成日の降順（新しい順）に並び替え（SQL の ORDER BY に相当）
    .order("created_at", { ascending: false });

  // 4. エラーハンドリング
  // error が null でなければ true（エラーが発生した）
  if (error) {
    // サーバーのターミナルにエラーの内容を出す
    console.error("書籍の取得に失敗しました:", error.message);
    // 失敗したときは空の配列を返して、画面が真っ白にならないようにする
    return [];
  }

  // 5. data は書籍の配列。万一 null のときに備えて ?? で空配列にする
  const books: Book[] = data ?? [];
  return books;
}
```

（このコードは、本章で作ったファイルと一緒に `npx tsc --noEmit` で型チェックを通ることを確認しています。第7章では、この中身をトップページの関数の中に直接書きます。）

この1つの `supabase.from("books").select("*")` の呼び出しの裏側で、上の図の 2 → 3 → 4 → 5 → 6 → 7 のステップがすべて自動的に実行されています。Supabase クライアントがこれらの複雑な処理を抽象化してくれるため、開発者は SQL を直接書くことなく、シンプルな JavaScript/TypeScript のコードでデータベースを操作できます。

▼ コードを1つずつ分解して解説

この取得処理を、初心者がつまずきやすい部分ごとに**1つずつ**ていねいに見ていきましょう。

解説1: サーバー用クライアントと型をインポートする

コピー

```
import { createClient } from "@/lib/supabase/server";
import type { Book } from "@/types/book";
```

- 1行目で、5.3 で作った `lib/supabase/server.ts` から、窓口を作る関数 `createClient` を取り込んでいます。このコードはサーバーで動くので、`client` ではなく `server` の方を使います。
- 2行目で `Book` 型を取り込み、取得したデータが「書籍の形」をしていることを TypeScript に伝えられるようにします。型だけなので `import type` にしています。
- どちらも `@/` （= プロジェクト直下）から始まるパスエイリアスを使い、相対パスより読みやすく書いています。

> **用語:** 「型のインポート」とは、`Book` のような型定義を別ファイルから取り込むことです。データの形を共有することで、プロジェクト全体で同じ構造を保てます。

解説2: 窓口を作り、await で取得結果を待つ

コピー

```
const supabase = await createClient();

const { data, error } = await supabase
  .from("books")
  .select("*")
  .order("created_at", { ascending: false });
```

- 1行目の `await createClient()` で、サーバー用の窓口を作ります。中で Cookie を受け取るのを待つので `await` が必要です（付け忘れると、窓口ではなく「あとで届く約束（Promise）」が入ってしまい、次の `.from` でエラーになります）。
- `await` （アウェイト）は、**通信が終わるまで結果を待つ**ための合図です。データベースとのやり取りには時間がかかるため、答えが返ってくるまでここで待機します。
- `.from("books")` で対象テーブルを、`.select("*")` で全列を、`.order("created_at", { ascending: false })` で「作成日の新しい順」を指定しています。SQL の `FROM` / `SELECT *` / `ORDER BY` に対応します。

- 戻り値は `{ data, error }` という形のオブジェクトなので、`const { data, error } = ...`（分割代入）で**成功データとエラーを同時に取り出して**います。

> **用語:**「await / 非同期処理」とは、時間のかかる処理（通信など）の完了を待つ仕組みです。待っている間ほかの処理を止めず、結果が来たら続きを実行します。

解説3: エラーを必ず確認してから先へ進む

コピー

```
if (error) {
  console.error("書籍の取得に失敗しました:", error.message);
  return [];
}

const books: Book[] = data ?? [];
return books;
```

- `if (error)` は「エラーが入っていれば（通信が失敗していれば）」という意味です。Supabase はエラーを**例外で投げず `error` に入れて返す**ため、自分でこのチェックをするのが安全な書き方です。
- 失敗時は `console.error(...)` で原因をサーバーのターミナルに出し、空の配列 `[]` を返します。失敗したのに先へ進んでしまう事故を防げます。
- 問題がなければ、`data` を `Book[]`（書籍の配列）として受け取ります。`?? []` は「左側が `null` なら右側の空配列を使う」という書き方です。

> **用語:**「エラーハンドリング」とは、失敗が起きたときの対処をあらかじめ書いておくことです。通信は必ず成功するとは限らないため、`error` の確認はセットで行います。

まとめ

この章で実施した内容を振り返ります。

項目	内容
Next.js プロジェクトの作成	`create-next-app` の質問で「カスタマイズ」を選び、TypeScript + ESLint + Tailwind CSS + App Router、`src/` なし、Cache Components なしのプロジェクトを作成しました。
Tailwind CSS の基礎	ユーティリティファーストの概念と、よく使うクラスを学びました。
パッケージのインストール	`@supabase/supabase-js` と `@supabase/ssr` をインストールしました。
ディレクトリ構成の設計	プロジェクト直下の `app/`（ページ）、`components/`（UI部品）、`lib/`（ユーティリティ）、`types/`（型定義）の4つのディレクトリで整理する構成にしました。
Supabase クライアントの設定	`.env.local` に URL と Publishable key を設定し、`lib/supabase/server.ts`（サーバー用）と `lib/supabase/client.ts`（ブラウザ用）の2つでクライアントを作れるようにしました。
型定義の作成	`Book`, `BookInsert`, `BookUpdate` の3つの型と、ステータス（読書中・読了・読みたい）関連の定数を定義しました。
ルートレイアウトの設定	日本語フォント、メタデータ、全ページ共通のレイアウト構造を設定し、`globals.css` を `@import "tailwindcss";` の1行にしました。
ヘッダーコンポーネントの作成	ナビゲーション付きの共通ヘッダーを作成しました。
動作確認	開発サーバーを起動し、ブラウザで表示を確認しました。

作成したファイル一覧:

コピー

```
book-management/
├── .env.local              ✅ 作成済み（Git には入らない）
├── app/
│   ├── layout.tsx          ✅ 作成済み
│   ├── page.tsx            ✅ 作成済み（仮の内容）
│   └── globals.css         ✅ 書き換え済み
├── components/
│   └── Header.tsx          ✅ 作成済み（ほかは空のファイル）
├── lib/
│   └── supabase/
│       ├── client.ts       ✅ 作成済み
│       └── server.ts       ✅ 作成済み
└── types/
    └── book.ts             ✅ 作成済み
```

この章のチェックリスト:

- [ ] `npm run dev` の表示に `- Environments: .env.local` が出ている
- [ ] `http://localhost:3000`（または表示された URL）で、ヘッダーと「セットアップ完了」のカードが見える
- [ ] `next.config.ts` に `cacheComponents: true` の行が無い
- [ ] `app/`・`components/`・`lib/`・`types/` が `package.json` と同じ場所にある
- [ ] `scripts/typo-demo.ts` を作った場合は削除した

次の章では、書籍一覧の表示機能を実装します。`BookCard`、`BookList`、`StatusBadge`、`RatingStars`、`LoadingSpinner` の各コンポーネントを作成し、Supabase からデータを取得してトップページに表示します。

練習問題（解答・解説つき）

学んだことを確かめましょう。まず自分で考えてから、後ろの「解答と解説」を読んで答え合わせをしてください。

問題1: create-next-app の答え

`npx create-next-app@latest book-management` を実行したら、最初に「Would you like to use the recommended Next.js defaults?」と聞かれました。本書の手順では何を選び、その後の質問のうち**既定値から変える必要がある**のはどれでしょうか。また、その理由を1つ答えてください。

問題2: `@/` の行き先

`tsconfig.json` の `"paths"` が `"@/*": ["./*"]` のとき、`app/books/[id]/edit/page.tsx` の中で次のように書きました。この import は、どのファイルを読み込もうとしますか。また、同じファイルを相対パスで書くとどうなりますか。

コピー

```
import Header from "@/components/Header";
```

問題3: エラーメッセージを読む

`npm run build` をしたら、次のエラーが出ました。どこをどう直せばよいでしょうか。

コピー

```
./app/layout.tsx:17:1
Error: Module not found: Can't resolve '@/src/components/Header'
```

問題4: どちらの窓口を使う？

次の2つのファイルでは、`lib/supabase/server.ts` と `lib/supabase/client.ts` のどちらの `createClient` を import すべきでしょうか。

- (a) 先頭に `"use client"` が書かれた、登録フォームの部品 `components/BookForm.tsx`
- (b) `"use client"` が無く、`export default async function` で書かれたトップページ `app/page.tsx`

問題5: 環境変数のトラブル

`.env.local` に次のように書いて `npm run dev` したところ、Supabase を使うページで「環境変数 NEXT_PUBLIC_SUPABASE_URL / NEXT_PUBLIC_SUPABASE_ANON_KEY が設定されていません」と表示されました。原因と直し方を答えてください。

コピー

```
NEXT_PUBLIC_SUPABASE_URL=https://xxxxxxxxxxxxxxx.supabase.co
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=sb_publishable_xxxxxxxxxxxxxxxxxxxxxxx
```

問題6: `NEXT_PUBLIC_` の意味

`.env.local` に `SECRET_TOKEN=abc123`（`NEXT_PUBLIC_` なし）と書きました。この値を、(a) Server Component、(b) Client Component の中で `process.env.SECRET_TOKEN` として読むと、それぞれどうなりますか。

問題7: Tailwind CSS が効かない

画面の横並び（`flex`）は効いているのに、文字の色や大きさ（`text-gray-600`、`text-3xl`）が全く付きません。`app/globals.css` は次のようになっていました。原因と直し方を答えてください。

コピー

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

解答と解説

問題1の解答

解答: 最初の質問では「**No, customize settings**」（カスタマイズ）を選びます。その後の質問で既定値から変えるのは **⑧ Cache Components を No にする**ことです（⑪ 匿名フィードバックも本書では No にしていますが、こちらはどちらでも構いません）。

解説: 推奨設定には Cache Components の有効化が含まれています。Cache Components が有効だと、第7章のトップページのように「サーバーで毎回データを取りに行くページ」が `npm run build` で `Next.js encountered uncached or runtime data during prerendering.` というエラーになります（1.7）。`src/` フォルダの質問⑥は既定値の No のままで、本書の構成（`app/` などをプロジェクト直下に置く）と一致します。

問題2の解答

解答: プロジェクト直下の `components/Header.tsx` を読み込みます。相対パスで書くと `import Header from "../../../../components/Header";` です。

解説: `@/` は `./`（`tsconfig.json` があるプロジェクト直下）に置き換えられ、`./components/Header` になります。拡張子 `.tsx` は自動で補われます（4.4 の図）。相対パスは「ファイルのあるフォルダ（`edit/`）」から数えるので、`edit/` → `[id]/` → `books/` → `app/` → プロジェクト直下、と4回 `../` でのぼってから `components/Header` に進みます。`@/` を使えば、どの深さのファイルからでも同じ文字列で書けます。

問題3の解答

解答: `app/layout.tsx` の17行目の import を `import Header from "@/components/Header";` に直します。

解説: `./app/layout.tsx:17:1` は「`app/layout.tsx` の17行目の1文字目」という場所の情報です。本書の構成には `src/` フォルダが無いので、`@/src/components/Header`（= `./src/components/Header`）は存在しません（4.4.1）。`src/` を使う構成の記事をまねたときによく起きるミスです。

問題4の解答

解答: (a) は `@/lib/supabase/client`、(b) は `@/lib/supabase/server` です。

解説: `"use client"` があるファイルはブラウザで動く Client Component なので、ブラウザ用の `client.ts` を使います。`"use client"` が無いファイルはサーバーで動く Server Component なので、Cookie を扱える `server.ts` を使い、`const supabase = await createClient();` のように `await` を付けて呼びます（5.3 の解説5）。逆にすると、ブラウザでは使えない `cookies()` を呼ぼうとしてエラーになります。

問題5の解答

解答: 2行目の変数名が、コードが読む名前（`NEXT_PUBLIC_SUPABASE_ANON_KEY`）と違うのが原因です。`NEXT_PUBLIC_SUPABASE_ANON_KEY=sb_publishable_...` に直して保存し、ターミナルに `Reload env: .env.local` が出るのを確認します（出なければ `Ctrl+C` で止めて `npm run dev` で起動し直します）。

解説: `lib/supabase/server.ts` と `client.ts` は `process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY` を読むので、名前が1文字でも違うと値は `undefined` になり、5.3 で書いたチェックがエラーを出します。中身が Publishable key でも、変数名は本書のコードに合わせて `ANON_KEY` にします（5.2.2）。`node --env-file=.env.local scripts/check-env.ts` で事前に確かめる習慣をつけると、起動前に気づけます。

問題6の解答

解答: (a) Server Component では `"abc123"` が読めます。(b) Client Component（ブラウザで動くとき）では `undefined` になります。

解説: Next.js は `.env.local` の値をサーバー側の `process.env` に入れます。ブラウザ用の JavaScript には、`NEXT_PUBLIC_` 付きの変数だけが**文字列として埋め込まれ**ます（5.2 の図）。`NEXT_PUBLIC_` が付いていない値は

ブラウザに届かないので、秘密にしたい値（Secret key など）は `NEXT_PUBLIC_` を付けずに、サーバーのコードだけで使います。逆に言えば、`NEXT_PUBLIC_` を付けた値は誰でも見られる前提で扱う必要があります。

問題7の解答

解答: Tailwind CSS v3 の古い書き方になっているのが原因です。`app/globals.css` を `@import "tailwindcss";` の1行に書き換えます。

解説: create-next-app が入れる Tailwind CSS v4 では、色や文字サイズの見本帳（テーマ）を `@import "tailwindcss";` で読み込みます。古い3行ではテーマが読み込まれないため、エラーは出ないのに `bg-white` や `text-3xl` などのクラスが作られません（7.2.1 の実験では、できた CSS が 12,332 バイトから 1,236 バイトに減っていました）。